UTN San Nicolás

UTN - FRSN

ALGORITMOS Y ESTRUCTURAS DE DATOS

Carrera: Ingeniería en Sistemas de Información
Materia: Algoritmos y estructuras de datos
AGOSTO 2026 – Versión 3.0

CONTENIDO

CAPITULO 1 Introducción a Algoritmos y Estructuras de Datos 6
Definición de Algoritmo 6
Características principales de un algoritmo: 6
Resolución de problemas 7
Representaciones de un algoritmo: 7
Importancia de los algoritmos: 9
Estrategias de Resolución de Problemas 10
Estado, Acción, Procesador Y Proceso 12
Tablas de Decisión 14
CAPITULO 2 Diagramas de Chapin o Nassi-Shneiderman 17
CAPITULO 3 Lenguajes de programación 18
Lenguajes de alto nivel y lenguajes de bajo nivel 18
Programa Fuente 18
Programa Objeto 19
Herramientas de Traducción 19
CAPITULO 4 Paradigmas de programación 21
CAPITULO 5 Instalación de CodeBlocks 22
CAPITULO 6 Variables, constantes y tipos de datos 32
Tipos de Datos 34
CAPITULO 7 Operadores y expresiones 36
CAPITULO 8 Errores 39
CAPITULO 9 Teorema Fundamental de la Programación Estructurada 41
CAPITULO 10 Primeros pasos en C++ 42
CAPITULO 11 Estructuras de control – Estructuras secuenciales 44
Salida de Datos en Algoritmia y C++ 45
Ingreso de Datos en Algoritmia y C++ 47
Asignación en Algoritmia y C++ 48
SCANF Y PRINTF 49
Ventajas de scanf y printf: 49
Sintaxis 49
Ejemplos básicos 50
CAPITULO 12 Estructuras de control – Estructuras de decisión 52

CAPITULO 13 Estructuras de control – Estructuras de selección múltiple. 55
CAPITULO 14 Contadores y acumuladores 56
Contadores 56
Acumuladores 56
CAPITULO 15 Estructuras de control – Estructuras repetitivas 57
Estructura WHILE 57
Estructura DO - WHILE 57
Estructura FOR 59
CAPITULO 16 Conversión entre Tipos de Datos Primitivos en C++ 61
CAPITULO 17 Arreglos 62
Arrays de dos dimensiones 63
Ordenamiento de Arreglos 65
Método de Ordenamiento por Intercambio 65
Método de Ordenamiento de Selección por intercambio Directo. 66
Método de Ordenamiento de Selección (SelectionSort) 68
Método de Ordenamiento de Inserción (InsertionSort) 70
Método de Ordenamiento Quick Sort 71
Búsquedas en arrays. 76
Búsqueda secuencial (Lineal) 76
Búsqueda binaria (Dicotómica) 78
CAPITULO 18 Funciones y procedimientos 81
Funciones y procedimientos 82
Procedimiento 82
Función 83
Parámetros y argumentos 84
Pasaje de parámetros 84
Pasaje por valor 84
Pasaje por referencia 85
Funciones que trabajan con arreglos 86
Parámetros de entrada, salida y entrada/salida 86
Prototipos de funciones en C++ 89
Ámbitos De Una Variable 89
CAPITULO 19 Recursividad 91
CAPITULO 20 Registros o Struct 93

Introducción a los registros 93
Definición de registro 94
Convención utilizada en la asignatura 94
Registro como modelo de una entidad 95
Declaración de una variable de tipo registro 95
Acceso a los campos 96
Vectores de registros 96
Definición de un vector de registros 97
CAPITULO 21 Archivos 100
Archivos en C++ 100
Escritura en archivos de texto 101
Verificación de la apertura de un archivo 102
Lectura de archivos de texto 103
Lectura de varios datos 104
Fin de archivo 105
Lectura de líneas completas con getline() 107
Agregar información a un archivo sin borrar su contenido 108
Modos de apertura de un archivo 109
Combinación de modos de apertura 110
Uso de fstream 110
Archivos de registros 110
Registros almacenados en un archivo de texto 111
Lectura de registros desde un archivo de texto 112
Archivos binarios 113
Apertura de un archivo binario 114
Escritura en un archivo binario 114
Lectura desde un archivo binario 115
Escritura de un registro completo 115
Lectura de un registro completo 117
Archivos binarios con varios registros 118
Grabación de varios registros 119
Carga de registros desde teclado 120
Lectura de varios registros 122
Agregar nuevos registros sin borrar los existentes 123

Acceso directo o aleatorio a un archivo 123
La función remove() 127
La función rename() 127
CAPITULO 22. Gestión Dinámica de Variables 128
CAPITULO 23. Punteros 130
Gestión dinámica de arreglos. Relación entre punteros y arreglos 131
Punteros y registros 132
CAPITULO 24. Estructuras de datos abstracta 134
CAPITULO 25. Listas 135
Lista Simple 135
Creación dinámica de un nodo 136
Acceso a los campos del nodo 136
Representación gráfica de una lista 137
Recorrido de una lista simplemente enlazada 137
Recorrer toda la lista 138
Lista vacía 139
Inserción del primer nodo 140
Algoritmo para insertar al inicio 141
Inserción al final de una lista 141
Inserción en una posición intermedia 143
Eliminación de nodos 145
Bibliografía 149

CAPITULO 1 Introducción a Algoritmos y Estructuras de Datos

En la era de la información, los algoritmos y las estructuras de datos se encuentran en el corazón de la informática y el desarrollo de software. Estas disciplinas son fundamentales para diseñar soluciones eficientes a problemas complejos, optimizar el uso de recursos computacionales y garantizar que las aplicaciones sean rápidas y escalables.
Un **algoritmo** es una secuencia finita de pasos lógicos destinados a resolver un problema o realizar una tarea específica. Es el núcleo de cualquier programa informático y define cómo se procesa la información para alcanzar un objetivo. Por otro lado, las **estructuras de datos** son formas organizadas de almacenar, gestionar y acceder a la información. Desde listas simples hasta árboles y grafos complejos, estas estructuras permiten que los algoritmos manipulen datos de manera efectiva.
La relación entre algoritmos y estructuras de datos es inseparable: la elección adecuada de una estructura puede simplificar el diseño de un algoritmo y mejorar su rendimiento. Por ejemplo, encontrar un elemento en una lista ordenada se resuelve más eficientemente con una búsqueda binaria, que requiere que los datos estén organizados de manera específica.
En este apunte, exploraremos conceptos clave como:

- **Diseño de algoritmos**: Estrategias y técnicas para construir soluciones óptimas.
- **Análisis de algoritmos**: Evaluación de su eficiencia en términos de tiempo y espacio.
- **Estructuras de datos básicas y avanzadas**: Desde arreglos y listas enlazadas hasta árboles y grafos.
- **Aplicaciones prácticas**: Cómo estos conceptos se aplican en el desarrollo de software y resolución de problemas reales.

El estudio de algoritmos y estructuras de datos no solo nos permiten crear programas eficientes, sino que también fomenta habilidades críticas como el pensamiento lógico, la resolución de problemas y la capacidad de modelar situaciones complejas. Al dominar estos temas, estaremos mejor preparados para enfrentar los desafíos del mundo tecnológico actual y del futuro.

Definición de Algoritmo

Un **algoritmo** es una secuencia **finita, ordenada y bien definida** de pasos o instrucciones diseñadas para resolver un problema o llevar a cabo una tarea específica. Es un concepto fundamental en la informática y las matemáticas, ya que constituye la base sobre la cual se construyen los programas y sistemas computacionales.

Características principales de un algoritmo:

1. **Finitud**:
   Un algoritmo debe tener un número limitado de pasos. Esto significa que debe terminar después de ejecutarse completamente, evitando bucles infinitos o procesos interminables.
2. **Definición precisa**:
   Cada paso del algoritmo debe estar claramente especificado, sin ambigüedades. Esto asegura que cualquier persona o máquina que lo interprete lo entienda de la misma manera.
3. **Entrada**:
   Un algoritmo *puede* requerir datos iniciales, conocidos como entradas, para operar. Estas entradas son los parámetros que definen el problema a resolver.
4. **Salida**:
   El algoritmo debe producir al menos un resultado o solución como salida, que es la respuesta al problema planteado.
5. **Efectividad**:
   Cada paso del algoritmo debe ser realizable en un tiempo razonable y con los recursos disponibles, asegurando que las operaciones sean prácticas y ejecutables.

6. **Generalidad**:
   Un algoritmo debe ser aplicable a una clase general de problemas, no solo a un caso específico. Esto lo hace reutilizable y versátil.

Eficiencia de un algoritmo

Un algoritmo es eficiente cuando logra llegar a sus objetivos planteados utilizando la menor cantidad de recursos posibles, es decir, minimizando el uso memoria, de pasos y de esfuerzo humano.

Eficacia de un algoritmo

Un algoritmo es eficaz cuando alcanza el objetivo primordial, el análisis de resolución del problema se lo realiza prioritariamente.

Diferencia entre eficiencia y eficacia

La eficacia difiere de la eficiencia en el sentido que la eficiencia hace referencia a la mejor utilización de los recursos, en tanto, la eficacia hace referencia en la capacidad para alcanzar un objetivo sin importar si se les ha dado el mejor uso posible a los recursos.

Cuando un algoritmo es *EFICIENTE* y *EFICAZ* se dice que es **EFECTIVO**

Resolución de problemas

La resolución de un problema exige el diseño de un algoritmo que resuelva el problema propuesto.
Para lograr resolver un problema, se pueden seguir los siguientes pasos:

1. Análisis del problema: en este paso se define el problema, se lo comprende y se lo analiza con el mayor detalle.
2. Diseño del algoritmo: se debe elaborar un algoritmo que refleje paso a paso la resolución del problema.
3. Resolución del algoritmo: se debe expresar el algoritmo como un programa en un lenguaje de programación adecuado.



Representaciones de un algoritmo:

Un algoritmo puede expresarse de varias formas dependiendo del contexto y el nivel de detalle requerido:

- **Narrativa**: Explicación en lenguaje natural, adecuado para comunicar ideas de manera sencilla.

Para poder sumar 2 números debe primero obtener el valor de los números a sumar, luego realizar la operación algebraica suma con estos valores y posteriormente exhibir el resultado de la misma

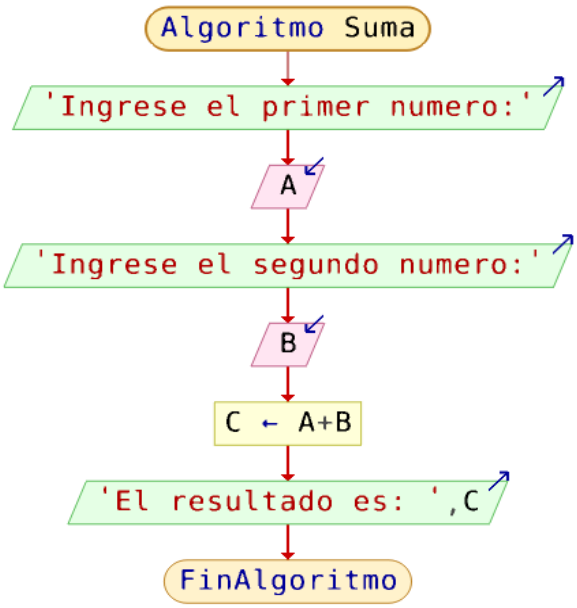
Otra opción

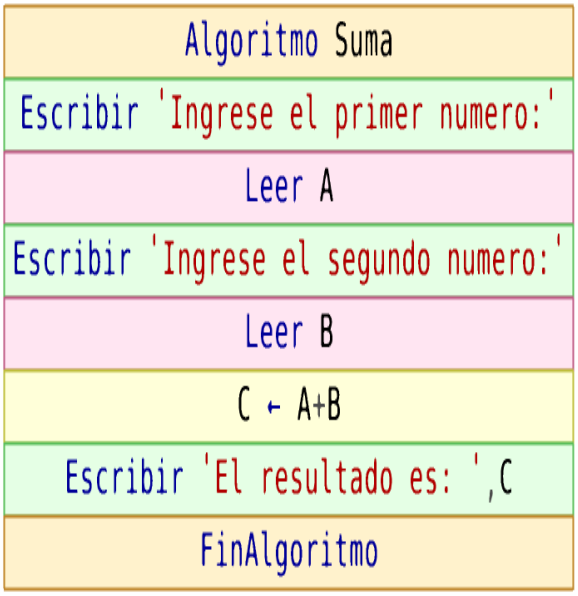
- Leer dos números.
- Sumar ambos números
- Mostrar el resultado de la suma.

- **Pseudocódigo**: Representación estructurada que combina lenguaje natural con notación matemática o lógica.

```
Algoritmo Suma
   Escribir "Ingrese el primer número:"
   Leer A
   Escribir "Ingrese el segundo numero:"
   Leer B
   C <- A+B
   Escribir "El resultado es: ", C
FinAlgoritmo
```

- **Diagramas de flujo**: Representación gráfica que utiliza símbolos y flechas para describir el flujo de ejecución.





- **Lenguaje de programación**: Implementación concreta del algoritmo en un lenguaje computacional específico.

```
int main() {
```

```
    float a;
    float b;
    float c;
    cout << "Ingrese el primer número:" << endl;
    cin >> a;
    cout << "Ingrese el segundo numero:" << endl;
    cin >> b;
    c = a+b;
    cout << "El resultado es: " << c << endl;
    return 0;
}
```

Importancia de los algoritmos:

Los algoritmos son esenciales para abordar problemas computacionales y prácticos. Permiten diseñar soluciones que son eficientes, reutilizables y comprensibles. Además, el análisis de algoritmos, en términos de complejidad temporal (tiempo de ejecución) y espacial (uso de memoria), es crucial para garantizar la optimización de recursos en sistemas modernos.

En resumen, un algoritmo es el punto de partida para transformar ideas abstractas en soluciones concretas que se pueden implementar y ejecutar en máquinas o sistemas reales.

Estrategias de Resolución de Problemas

Las estrategias de resolución de problemas son enfoques sistemáticos utilizados para analizar un problema, descomponerlo en partes más manejables y diseñar soluciones efectivas. Estas estrategias son fundamentales en la informática y la matemática aplicada, ya que permiten resolver problemas de manera lógica, estructurada y eficiente.
Cada estrategia se adapta a distintos tipos de problemas y contextos, y la elección de la más adecuada depende de las características del problema y los recursos disponibles. A continuación, se describen las principales estrategias:

---

1. Dividir y Vencerás

Esta estrategia consiste en dividir un problema grande y complejo en subproblemas más pequeños y manejables, resolver cada uno de estos subproblemas de forma independiente y, finalmente, combinar las soluciones para obtener la solución al problema original.

- Ejemplo clásico:
    - Merge Sort: Un algoritmo de ordenamiento que divide un arreglo en mitades, ordena cada mitad recursivamente y luego combina las mitades ordenadas.
    - Proceso:
        1. Dividir el arreglo en dos mitades hasta que queden arreglos de un solo elemento.
        2. Ordenar y combinar las mitades repetidamente.
- Ventajas:
    - Facilita la resolución de problemas complejos.
    - Es eficaz en algoritmos recursivos.

---

2. Backtracking (Vuelta Atrás)

El backtracking es una técnica basada en explorar todas las posibles soluciones a un problema, retrocediendo cuando se detecta que una opción no conduce a una solución válida.

- Ejemplo clásico:
    - Problema de las N-Reinas: Colocar N reinas en un tablero de ajedrez de tamaño NxN de forma que ninguna ataque a otra.
    - Proceso:
        1. Colocar una reina en la primera fila.
        2. Probar posiciones para la siguiente reina.
        3. Si no hay posiciones válidas, retroceder y probar otra opción para la reina anterior.
- Ventajas:
    - Permite explorar soluciones en problemas con múltiples combinaciones posibles.
    - Es útil para problemas de búsqueda y optimización.

---

3. Algoritmos Codiciosos

Los algoritmos codiciosos toman decisiones óptimas locales en cada paso, con la esperanza de que estas decisiones conduzcan a una solución global óptima.

- Ejemplo clásico:

    - Problema del Cambio de Monedas: Encontrar el menor número de monedas para dar un cambio específico.

- Proceso:
    1. Seleccionar la moneda de mayor valor que no exceda el cambio restante.
    2. Repetir hasta completar el cambio.

- Limitaciones:
    - No siempre garantizan la solución óptima en todos los problemas. (
    - Funcionan bien en problemas donde la decisión local conduce naturalmente a la global, como en grafos con pesos uniformes.

Estas estrategias constituyen herramientas esenciales en la resolución de problemas computacionales, y su estudio permite a los estudiantes diseñar algoritmos eficientes y elegantes para problemas complejos en el ámbito académico y profesional.

Estado, Acción, Procesador Y Proceso

Dentro de la temática de algoritmos, los conceptos de acción, estado, procesador y proceso son fundamentales para comprender cómo se describe, ejecuta y evalúa una secuencia de instrucciones orientada a la resolución de problemas. A continuación, se explican cada uno de ellos:

1. Acción

Definición:
Una acción es una operación elemental o instrucción que realiza un algoritmo como parte de su ejecución. Las acciones son los pasos básicos que conforman el flujo de un algoritmo, y pueden incluir cálculos, asignaciones, decisiones, o cualquier tarea que transforme los datos o avance en la resolución del problema.

Características:

- Son indivisibles en el nivel de abstracción del algoritmo.
- Se ejecutan de forma secuencial o condicional según el diseño del algoritmo.

Ejemplo:
En un algoritmo que calcula la suma de dos números:

- Acción 1: Leer el primer número.
- Acción 2: Leer el segundo número.
- Acción 3: Calcular la suma.
- Acción 4: Mostrar el resultado.

---

2. Estado
Definición:
El estado de un algoritmo o sistema es la información completa que describe su situación en un momento específico durante su ejecución. En términos de algoritmos, el estado está determinado por los valores actuales de las variables y parámetros que utiliza el algoritmo.

Características:

- Evoluciona a medida que el algoritmo ejecuta sus acciones.
- Es fundamental para determinar el comportamiento futuro del algoritmo.

Ejemplo:
En un algoritmo que calcula la suma de dos números:

- **El estado del valor a sumar estará “vacío” al inicio del algoritmo y luego contendrá el valor de uno de los operandos.**

3. Procesador
Definición:
Un procesador es el componente, ya sea físico (hardware) o lógico (software), encargado de interpretar y ejecutar las instrucciones de un algoritmo. En el contexto de algoritmos, el procesador puede ser considerado como el "ejecutor" que realiza las acciones definidas en el algoritmo.
Características:

- Puede ser una CPU, una unidad especializada, o un programa que simula la ejecución del algoritmo.
- Su eficiencia afecta directamente el tiempo de ejecución del algoritmo.

Ejemplo:
En un programa computacional:

- El procesador sería la CPU que ejecuta el código generado a partir del algoritmo.
- Para un robot que sigue un conjunto de instrucciones, el procesador sería el controlador interno que interpreta y actúa sobre esas instrucciones.

4. Proceso

Definición:
Un proceso es la ejecución dinámica de un algoritmo. Representa el conjunto de acciones realizadas por un procesador siguiendo las instrucciones del algoritmo en un entorno específico. Es el aspecto "activo" del algoritmo en ejecución.

Características:

- Se inicia con el estado inicial y finaliza al alcanzar un estado final o una condición de término.
- Puede involucrar recursos como memoria, tiempo de CPU y dispositivos de entrada/salida.
- Puede ser interactivo o autónomo, dependiendo del diseño del algoritmo.

Ejemplo:
Un algoritmo que ordena una lista:

- El proceso es la ejecución concreta de dicho algoritmo para una lista dada.
- Durante el proceso, los elementos de la lista se comparan, intercambian y reordenan según las reglas del algoritmo.

Relación entre los conceptos

1. Acciones son los bloques básicos que un algoritmo ejecuta.
2. El conjunto de valores en un momento dado define el estado del sistema o algoritmo.
3. El procesador es quien interpreta y ejecuta estas acciones, llevando al algoritmo de un estado a otro.
4. El proceso es la ejecución concreta del algoritmo en tiempo real, utilizando un procesador para llevar a cabo las acciones que transforman los estados.

Analogía para clarificar:

Imaginemos un chef que sigue una receta:

- Acción: Cortar verduras, calentar agua, agregar sal.
- Estado: La cantidad de ingredientes preparados y el nivel de cocción del plato en un momento dado.
- Procesador: El chef que realiza las acciones descritas en la receta.
- Proceso: La secuencia completa de cocinar el plato, desde reunir los ingredientes hasta servirlo.

Tablas de Decisión

Las tablas de decisión son una herramienta estructurada que permite representar de forma clara y sistemática la lógica detrás de un conjunto de reglas o condiciones y sus respectivas acciones. Se utilizan principalmente en la resolución de problemas complejos que involucran múltiples condiciones y decisiones interdependientes.

Componentes de una Tabla de Decisión

Condiciones:
Representan los criterios o factores que determinan las decisiones.

- Se expresan como preguntas de tipo sí/no o valores específicos.

Acciones:
Son las actividades o resultados que se ejecutan en función de las condiciones evaluadas.

Reglas:
Combinaciones de condiciones que conducen a una o más acciones.
Cada regla es una fila (o columna, según el diseño) que define un escenario específico.

Estructura Básica

Condiciones	**Regla 1**	**Regla 2**	**Regla 3**	**Regla 4**
Condición 1	Sí	Sí	No	No
Condición 2	Sí	No	Sí	No
Acciones				
Acción 1	X			
Acción 2		X		X
Acción 3			X	

- X: Indica que una acción se ejecuta para la regla correspondiente.
- Cada regla es una combinación específica de condiciones.

Ejemplo Práctico: Sistema de Descuentos en una Tienda
Enunciado:
Una tienda aplica descuentos según las siguientes reglas:

1. Si el cliente es miembro del programa de fidelidad y compra más de $1000, recibe un descuento del 20%.
2. Si el cliente no es miembro, pero compra más de $1000, recibe un descuento del 10%.

3. Si el cliente es miembro y compra $1000 o menos, recibe un descuento del 5%.
4. Si el cliente no es miembro y compra $1000 o menos, no recibe descuento.

Tabla de Decisión:

Condiciones	**Regla 1**	**Regla 2**	**Regla 3**	**Regla 4**
Es miembro del programa	Sí	No	Sí	No
Compra > $1000	Sí	Sí	No	No
Acciones				
Descuento 20%	X			
Descuento 10%		X		
Descuento 5%			X	
Sin descuento				X

Ventajas de las Tablas de Decisión

1. Claridad:
    - Representan de forma visual y organizada todas las combinaciones posibles de condiciones y sus resultados.
2. Precisión:
    - Ayudan a identificar reglas contradictorias, redundantes o ausentes.
3. Facilidad de mantenimiento:
    - Si se modifica una condición o acción, la tabla permite evaluar rápidamente el impacto en el sistema.
4. Versatilidad:
    - Son útiles para diseñar algoritmos, modelar sistemas de reglas en software o definir políticas de negocio.

Limitaciones

1. Escalabilidad:
    - Para problemas con muchas condiciones, las tablas pueden volverse demasiado grandes y difíciles de manejar.
2. Ambigüedad en reglas complejas:
    - Si las condiciones son interdependientes o tienen múltiples estados posibles, puede ser complicado representarlas de manera sencilla.

Conclusión

Las tablas de decisión son una herramienta poderosa para modelar decisiones lógicas en problemas estructurados. Su capacidad para simplificar la complejidad de múltiples condiciones las convierte en un recurso indispensable en el diseño y análisis de algoritmos, especialmente en entornos donde la claridad y la consistencia son cruciales

CAPITULO 2 Diagramas de Chapin o Nassi-Shneiderman

Representación de Algoritmos

Los diagramas de Chapin o Nassi-Shneiderman son una forma gráfica de representar algoritmos que se basa en el modelo estructurado de flujo de control. Su objetivo principal es proporcionar una representación clara y concisa de la lógica de un algoritmo, eliminando la ambigüedad que puede surgir en otras técnicas, como los diagramas de flujo.

Características Principales

1. Estructuración jerárquica:
   Los procesos se representan como bloques anidados, organizados de forma que reflejan el flujo de control del algoritmo.
2. Ausencia de líneas de flujo:
   A diferencia de los diagramas de flujo, los diagramas de Nassi-Shneiderman no utilizan flechas ni conexiones para indicar el flujo. Todo se deduce de la disposición espacial de los bloques.
3. Clara separación de estructuras:
   Representan explícitamente estructuras de control como:
   - Secuencia
   - Selección (decisiones)
   - Iteración (bucles)
4. Diseño lógico:
   Fomenta el pensamiento estructurado y la programación modular.

En esta cátedra se utilizará los diagramas de Chapin para poder representar las soluciones algorítmicas.

CAPITULO 3 Lenguajes de programación

Para que un procesador realice un proceso se le debe suministrar, en primer lugar, un **algoritmo** adecuado. El procesador debe ser capaz de interpretar el algoritmo, lo que significa:

• comprender las instrucciones de cada paso
• realizar las operaciones correspondientes.

Cuando el procesador es una computadora, el algoritmo se ha de expresar en un formato que se denomina programa, ya que el pseudocódigo o el diagrama de Chapin no son comprensibles por la computadora, aunque pueda entenderlos cualquier programador. Un programa se escribe en un lenguaje de programación. El proceso de traducir un algoritmo en pseudocódigo a un lenguaje de programación se denomina **codificación**. Hoy en día, la mayoría de los programadores emplean lenguajes de programación como C++, C#, Java, HTML, PHP, JavaScript, Python, Kotlin, Swift.

Lenguajes de alto nivel y lenguajes de bajo nivel

Los programadores escriben instrucciones en diversos lenguajes de programación.
La computadora puede entender directamente algunos de ellos, pero otros requieren pasos de traducción intermedios. Hoy día se utilizan cientos de lenguajes de computadora, los cuales pueden dividirse en tres tipos generales:

Lenguaje de máquina

Es un lenguaje escrito en **formato binario** (formado por 1 y 0), propio de cada computadora y relacionado con el diseño del hardware de esta. El lenguaje de máquina ordena a la computadora realizar sus operaciones fundamentales una por una. Es el único lenguaje capaz de entender por una computadora.
A+B -> 10100101010010

Lenguaje de bajo nivel (ensamblador)

Consiste en abreviaturas similares al inglés, llamadas instrucciones mnemotécnicas, que permiten representar las operaciones elementales de la computadora (también es dependiente de la computadora).

Lenguaje de alto nivel

Los lenguajes de alto nivel permiten a los programadores escribir instrucciones que asemejan el inglés cotidiano y contiene notaciones matemáticas de uso común. Cualquier programa escrito debe poder ejecutarse en cualquier computadora después de compilarlo. A esto se le conoce como portabilidad de programas.

Programa Fuente

El **programa fuente** es el conjunto de instrucciones escritas por el programador en un lenguaje de programación de alto nivel o de bajo nivel. Este código es generalmente legible para los humanos y sigue las reglas sintácticas y semánticas del lenguaje utilizado.

Lenguajes comunes:

- Lenguajes de alto nivel: Python, Java, C, C++, JavaScript.
- Lenguajes de bajo nivel: Ensamblador.

Ejemplo en C++ (alto nivel):

```
#include <iostream>

int main() {
   std::cout << "Hola, mundo!" << std::endl;
   return 0;
}
```

Programa Objeto

El **programa objeto** es el resultado del proceso de traducción de un programa fuente. Contiene código que la máquina puede interpretar o ejecutar directamente, pero a menudo requiere una etapa adicional de vinculación (linking) para crear un ejecutable final.

- En el caso de lenguajes compilados, el programa objeto está en un formato intermedio (código máquina o binario) que es entendido por el procesador.
- **Ejemplo:** Al compilar el código en C++ anterior, se genera un archivo intermedio como `main.o` (en sistemas UNIX) o un archivo ejecutable como `main.exe` (en Windows).

Herramientas de Traducción

a. Compiladores

Un **compilador** es un programa que traduce el código fuente escrito en un lenguaje de alto nivel a código máquina (programa objeto) en una sola etapa o en múltiples fases.

- **Proceso:**
  1. Traducción del programa fuente al programa objeto.
  2. Opcionalmente, el vinculador combina múltiples programas objeto en un ejecutable.
- **Ventajas:**
  - El código ejecutable generado es rápido.
  - La traducción se realiza una sola vez.
- **Ejemplos de lenguajes compilados:**
  - C, C++, Fortran, Rust.
- **Ejemplo de compilador:** GCC (GNU Compiler Collection).
- **Flujo de trabajo de compilación:**

Programa Fuente (.cpp) → Compilador → Programa Objeto (.o) → Vinculador → Ejecutable (.exe o binario)

b. Ensambladores

Un **ensamblador** convierte el código fuente en lenguaje ensamblador (bajo nivel) al código máquina ejecutable.

- **Ventajas:**
  - Permite un control detallado del hardware.

- **Ejemplo:** Un programa en ensamblador para sumar dos números:

```
section .data
   num1 db 5
   num2 db 3
   result db 0

section .text
   mov al, [num1]
   add al, [num2]
   mov [result], al
```

Ensamblador utilizado: NASM (Netwide Assembler).

c. Intérpretes

Un **intérprete** ejecuta el programa fuente directamente, traduciendo las instrucciones una por una y compilándolas en tiempo de ejecución.

- Ventajas:
  - Permite una prueba y depuración rápida.
  - No requiere un paso previo de compilación.
- Desventajas:
  - La ejecución suele ser más lenta que en lenguajes compilados.
- Ejemplos de lenguajes interpretados:
  - Python, Ruby, JavaScript, PHP.

Ejemplo en Python:

```
print("Hola, mundo!")
```

d. Lenguajes Híbridos (Compilación e Interpretación)

Algunos lenguajes combinan compilación e interpretación:

1. Primero, el código fuente se compila en un bytecode intermedio.
2. Luego, un intérprete o máquina virtual ejecuta este bytecode.

- **Ejemplos:**
  - Java: Compilado a bytecode ejecutado por la JVM (Java Virtual Machine).
- **Ventajas:**
  - Ofrecen portabilidad y buen rendimiento.

CAPITULO 4 Paradigmas de programación

Los **paradigmas de programación** son enfoques o estilos fundamentales para resolver problemas y diseñar programas en el desarrollo de software. Cada paradigma tiene sus principios, técnicas y metodologías, y puede ser más adecuado para ciertos tipos de problemas o proyectos. Diversos paradigmas han surgido desde los inicios de la programación que se verán en otros espacios de la carrera.
A continuación, se presentan los paradigmas más comunes:

Paradigma Imperativo

Se centra en describir cómo un programa debe realizar tareas mediante instrucciones paso a paso que modifican el estado del sistema.
Su característica principal es el uso de variables, asignaciones y estructuras de control como bucles y condicionales. El control del flujo es explícito. (C, C++, Pascal)

Paradigma Declarativo

Se enfoca en qué se quiere lograr en lugar de como hacerlo. El programador describe el resultado deseado y el lenguaje se encarga de los detalles.
Hace uso de expresiones y funciones en lugar de instrucciones detalladas.(SQL, Prolog).

Paradigma Orientado a Objetos (OOP)

Organiza el código en "objetos" que encapsulan datos (atributos) y comportamientos (métodos). Utiliza clases, herencia, polimorfismo y encapsulación y promueve la reutilización y modularidad. (Java, Python, Ruby)

CAPITULO 5 Instalación de CodeBlocks

Introducción

Codeblocks es un programa de desarrollo integrado libre y multiplataforma para el desarrollo de programas en C y C++. Este manual esta creado con la finalidad de guiar al alumno en la instalación del software Codeblocks. Antes de comenzar, asegurarse de leer cuidadosamente estas instrucciones y seguir cada paso con atención para que la instalación se realice correctamente.

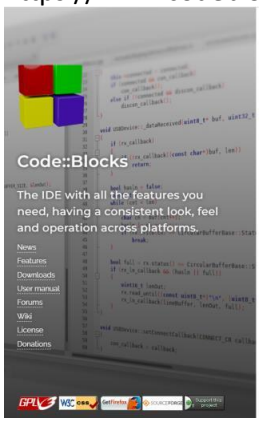
Requisitos del sistema

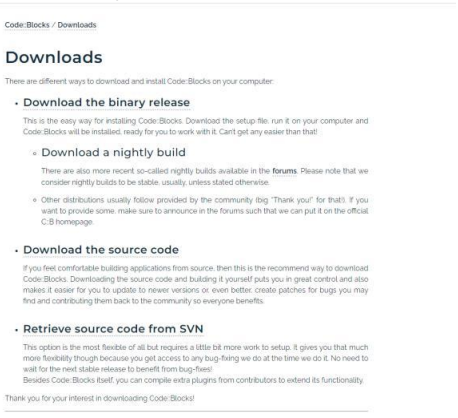
Para instalar el software Codeblocks, el ordenador debe cumplir con las siguientes especificaciones:

- Procesador de al menos 1 GHz
- Memoria RAM de al menos 2 GB
- Espacio en disco duro de al menos 5 GB
- Sistema operativo Windows 10 o superior, o macOS 10.13 o superior.

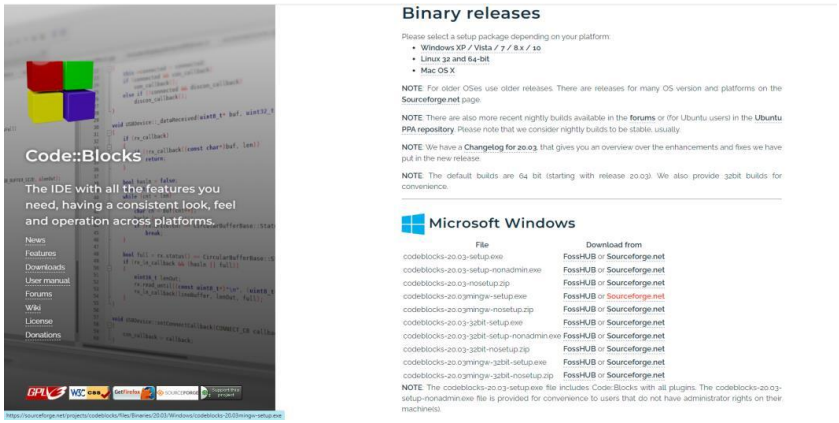
Descarga

Ingresar a la siguiente página y buscar el enlace de descarga. https://www.codeblocks.org/downloads/





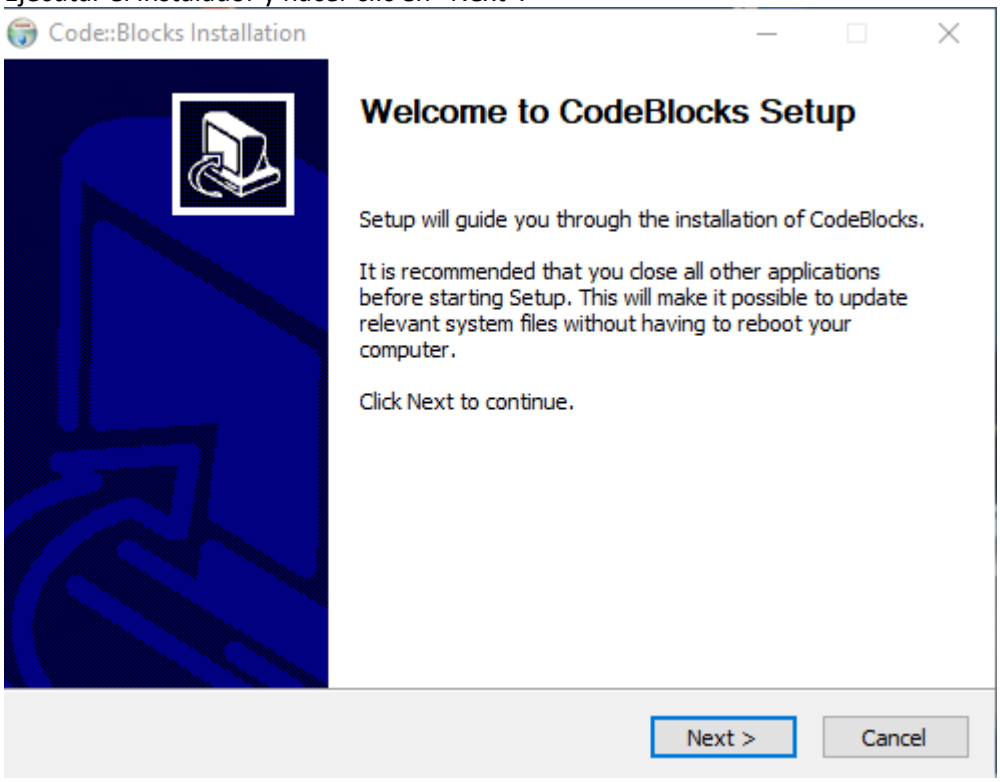
Hacer clic en el enlace para iniciar la descarga.



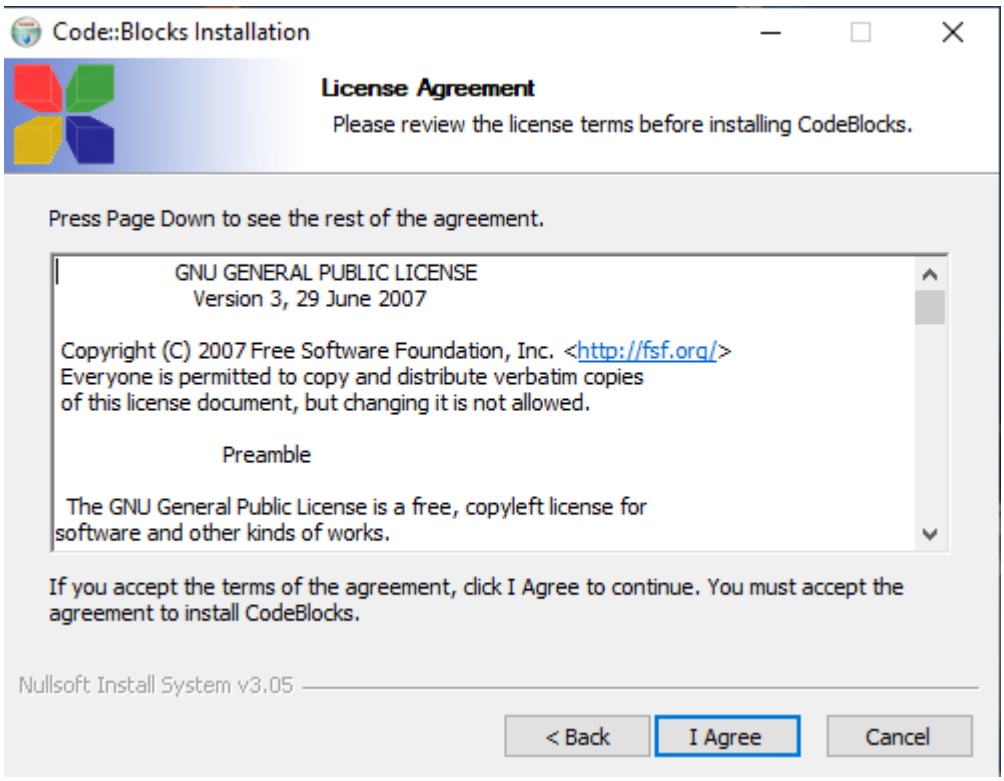
En la mayoría de los casos (Usuarios con Windows y permisos de administrador de sus equipos) se deberá descargar la opción codeblocks-20.03mingw-setup.exe (depende el momento que lea esto, el nro puede cambiar)
Esta opción incluye el compilador de C++.

INSTALACIÓN

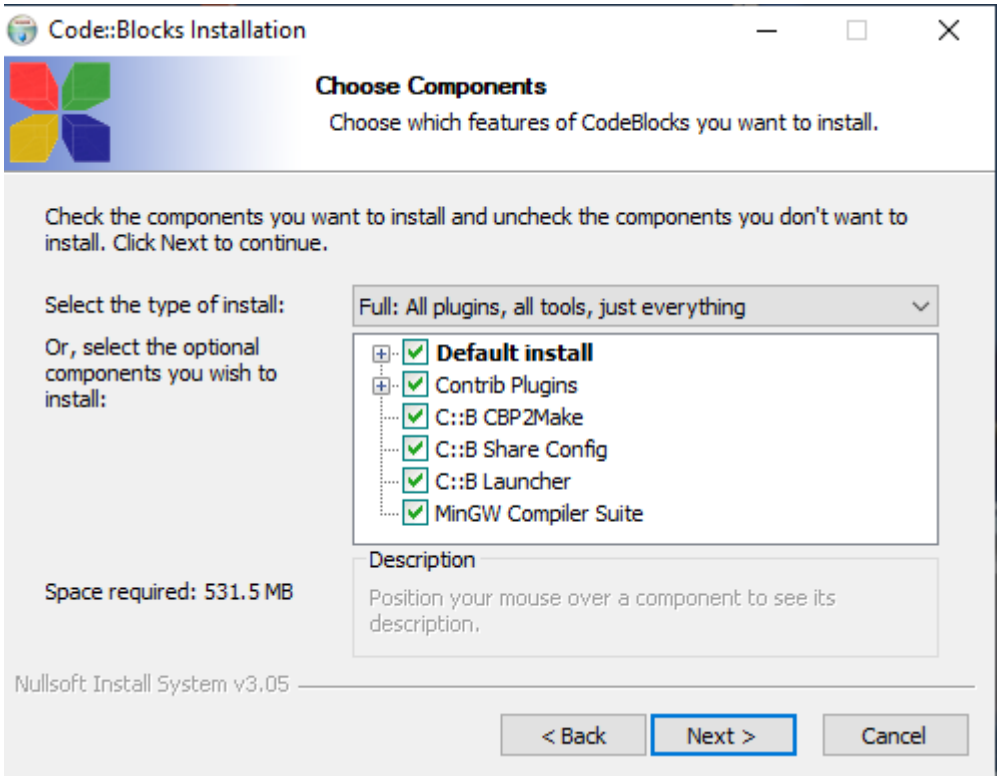
Ejecutar el instalador y hacer clic en “Next”.

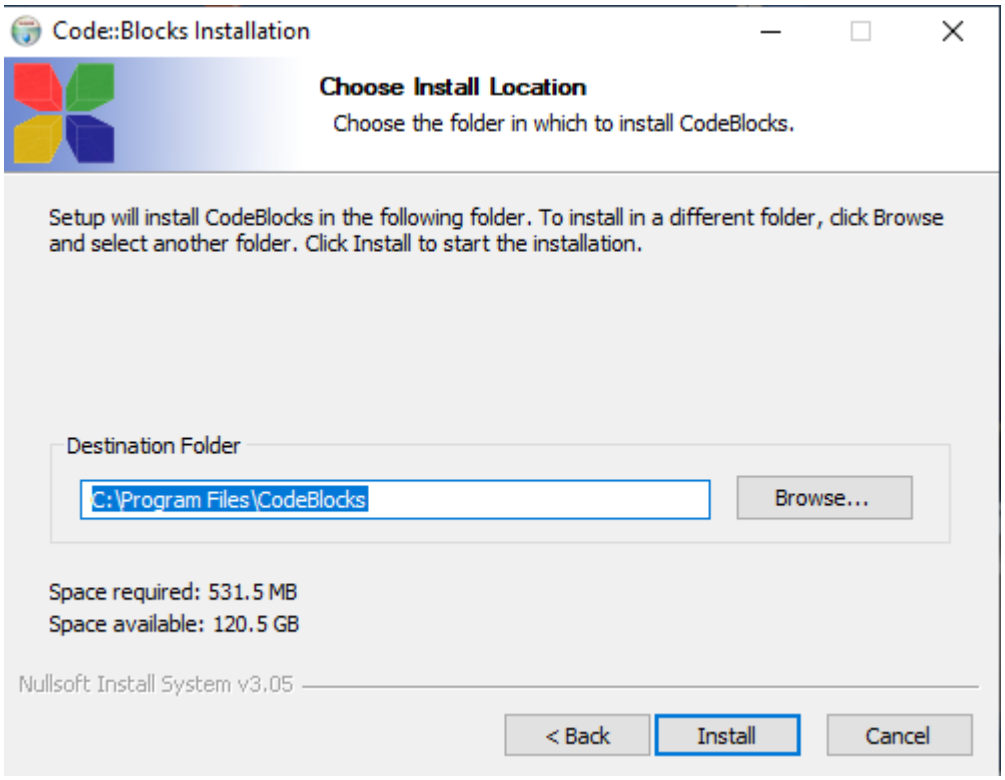


Aceptar los términos y condiciones, hacer clic en **“I Agree”**.

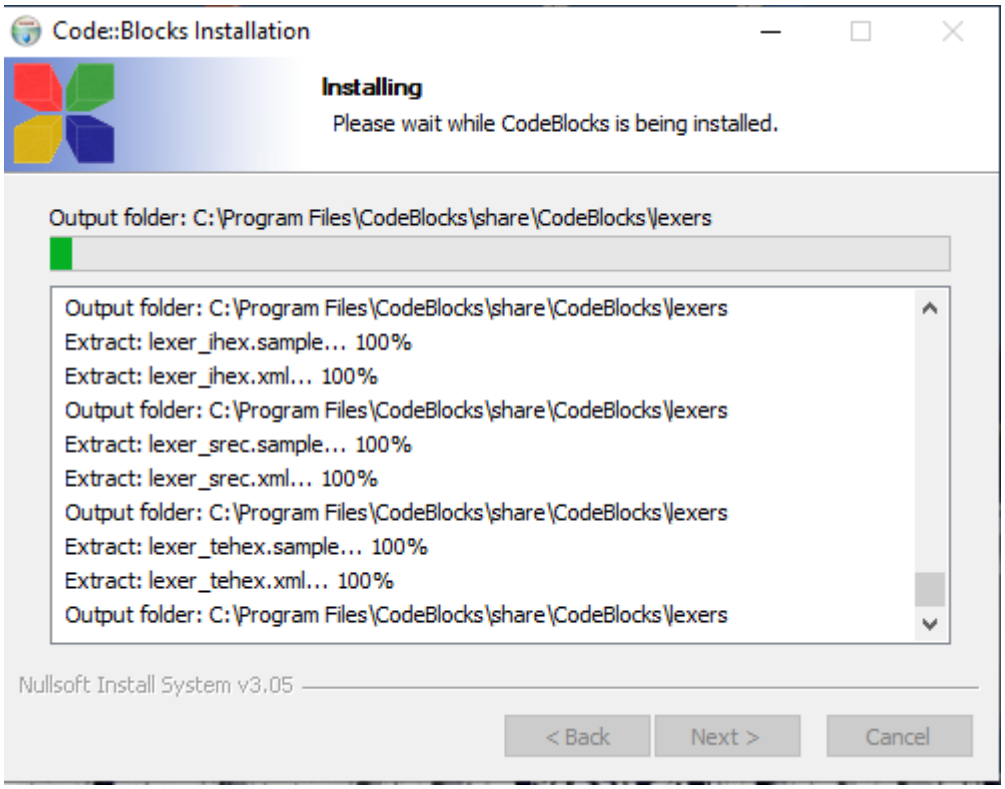


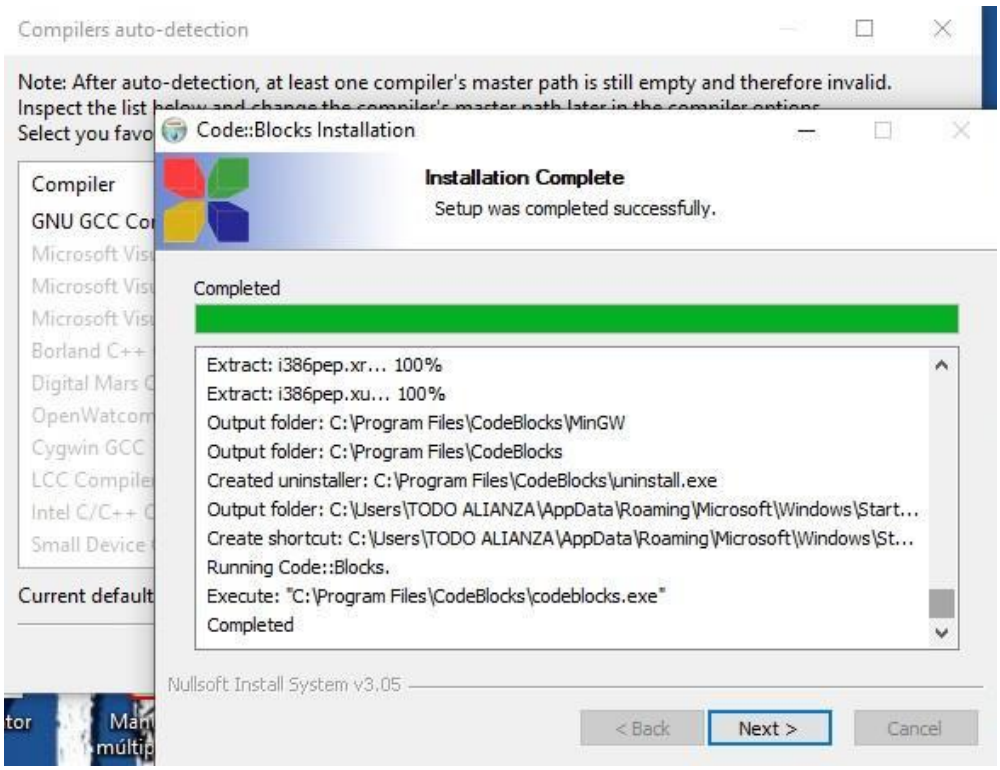
Seleccionar los componentes que se va instalar (por defecto), y luego hacer clic en "**Next**".



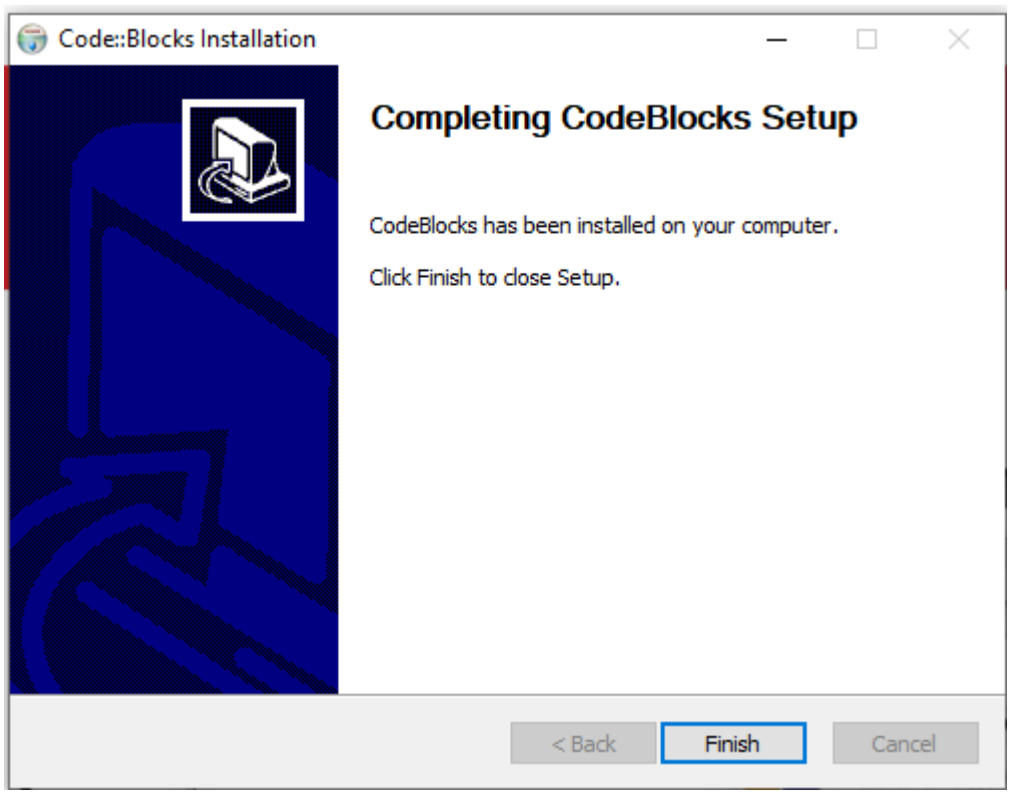


Seleccionar la ubicación donde se va guardar los archivos del programa a instalar. Hacer clic en "**Install**".
Esperar que la instalación culmine
Luego hacer clic en "**Next**".

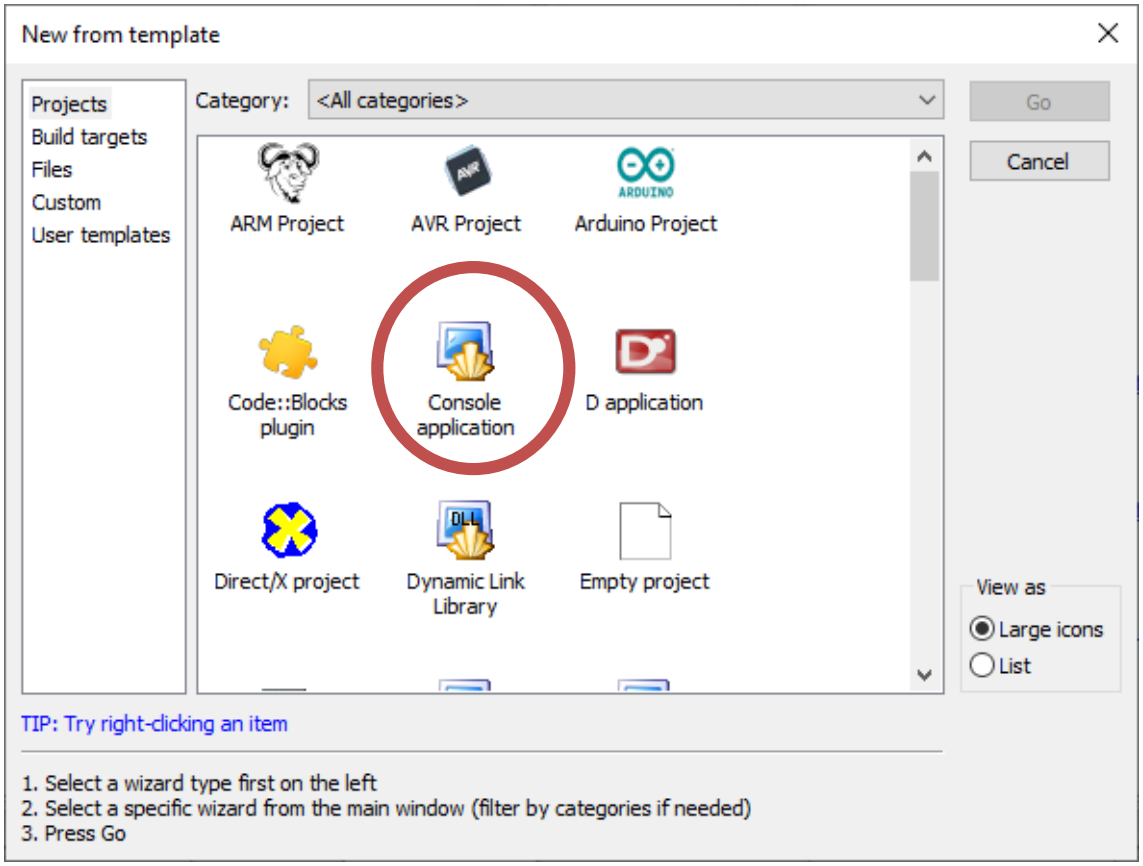




Al finalizar la instalación y hacer clic en Finish.
Finalmente podrá usar el software.

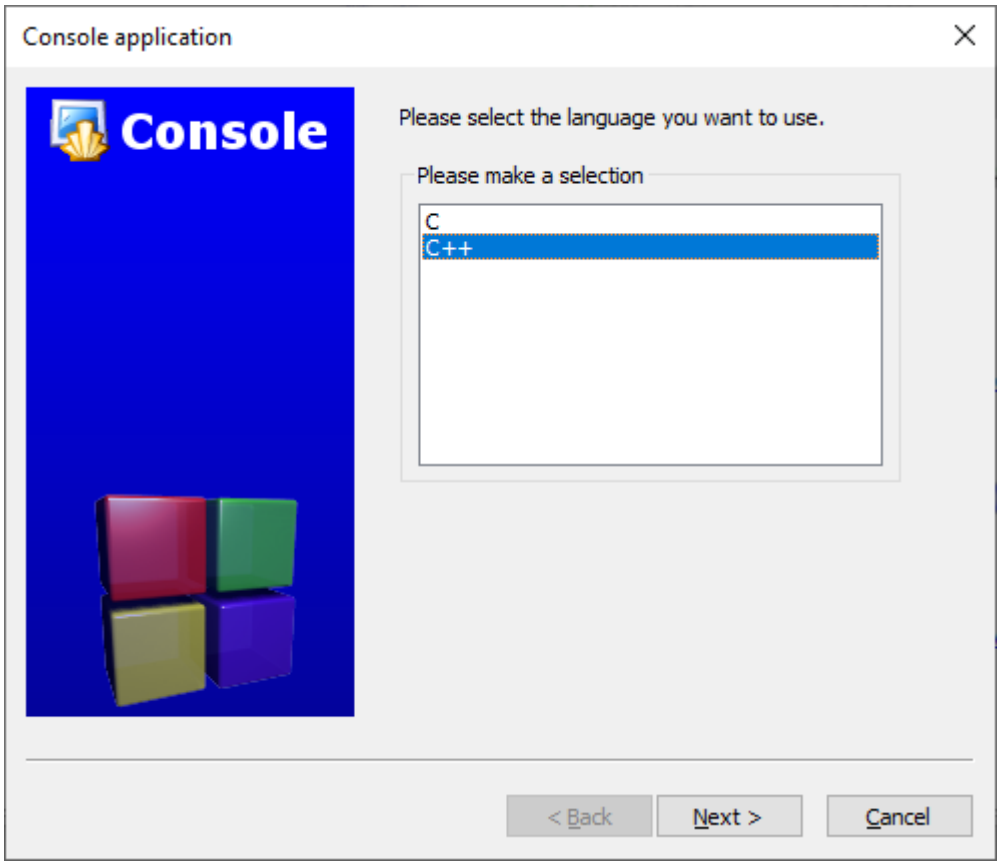


En Code:Blocks, las fuentes y las configuraciones para el proceso de compilación se almacenan en un archivo de proyecto <name> .cbp . Las fuentes de C/C++ y los archivos de encabezado correspondientes son los componentes típicos de un proyecto. La forma más fácil de crear un nuevo proyecto es ejecutar el comando 'Archivo' → ' Proyecto' y seleccionar un asistente. En nuestro caso

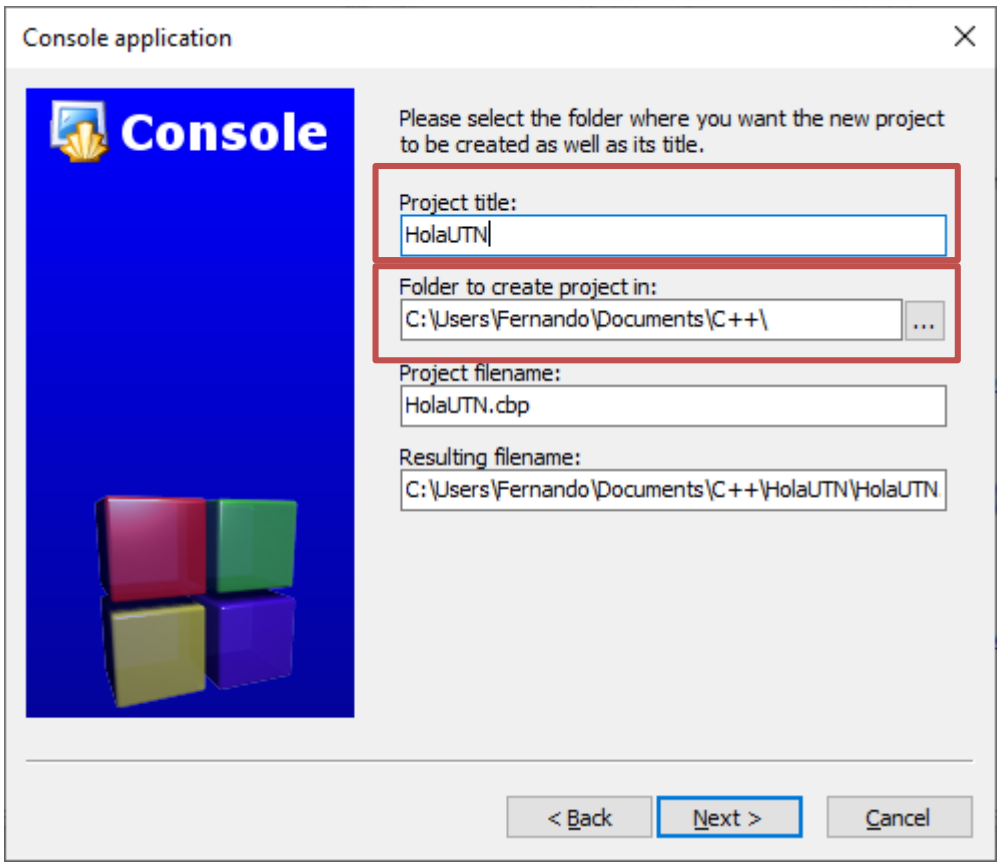


siempre seleccionaremos CONSOLE APPLICATION

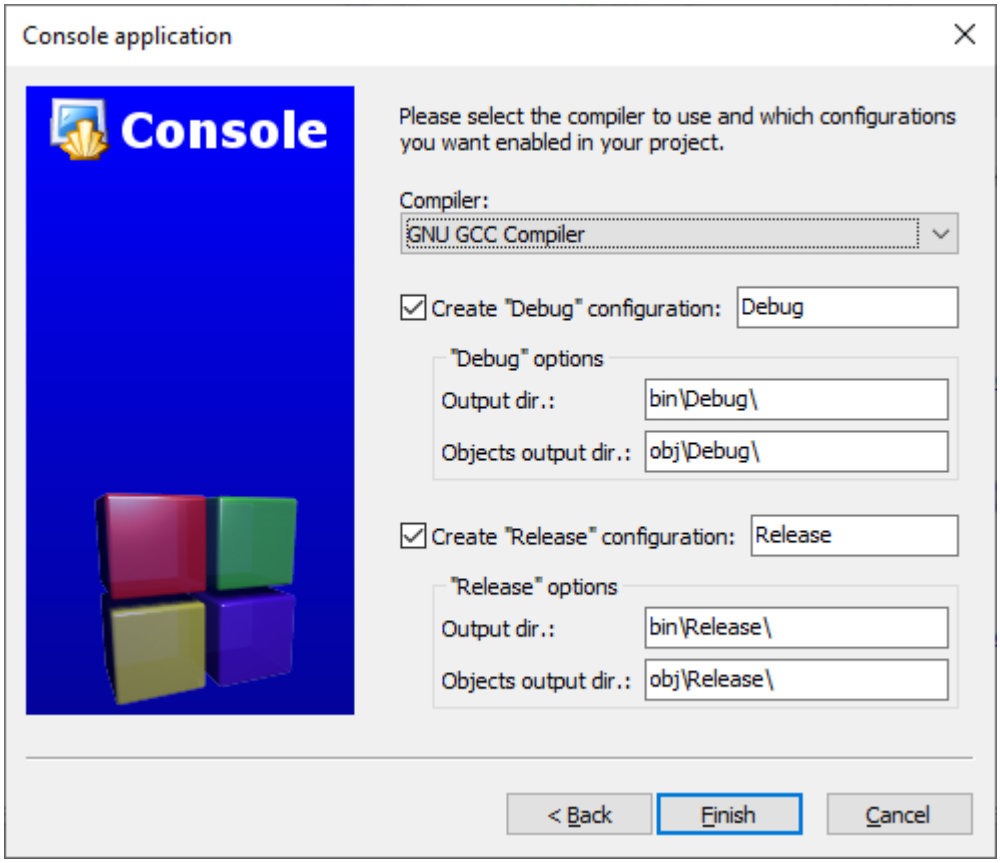
Luego elegiremos el lenguaje C++



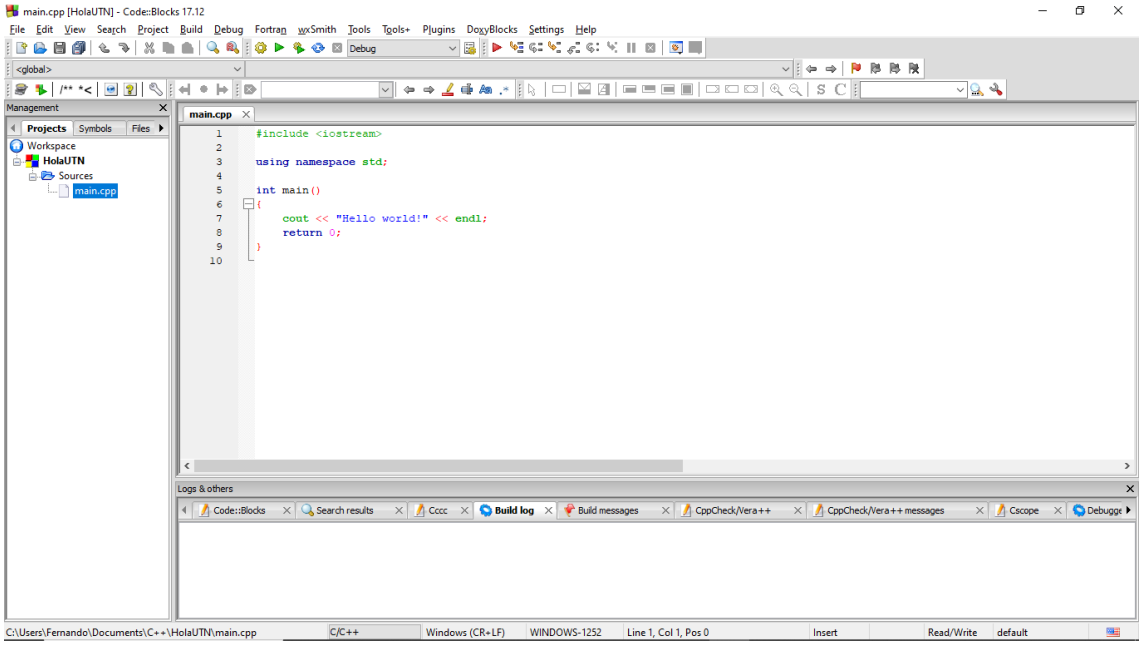
Y elegiremos un **nombre** y una **ubicación** para nuestro proyecto



En la siguiente pantalla se seleccionará automáticamente el compilador a utilizar, que si instalamos correctamente será GNU GCC Compiler



Al presionar **Finish** se creará nuestro proyecto

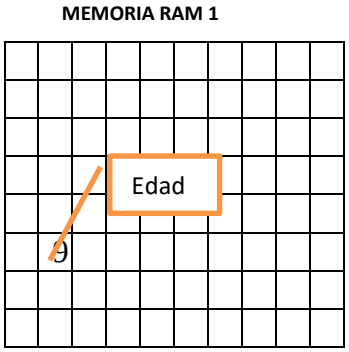


CAPITULO 6 Variables, constantes y tipos de datos

Variables

Definición

Un programa necesita almacenar los diferentes datos que utiliza durante su ejecución. Para ello existen las **variables,** ubicaciones en la memoria de la computadora en las cuales se puede grabar un valor y por la cual se puede recuperar dicho valor más tarde.
En otras palabras, una variable es un espacio para guardar datos/información (valores). Las variables están formadas por un **espacio en el sistema de almacenaje** (memoria principal de un ordenador) y un **nombre simbólico** (identificador) que está asociado a dicho espacio.



¿Cómo funcionan las variables?

Cuándo creamos una variable lo que hace la pc es reservar un espacio de la memoria RAM. La **memoria RAM** de la computadora puede ser vista como una serie de pequeñas casillas, cada una de las casillas esta numerada secuencialmente, este número que se le asigna representa su dirección de memoria y su objetivo es identificarla.
Una variable reserva una o más casillas en las cuales es posible grabar datos y se accede a ellas mediante el nombre (identificador) asignado a dicha variable. El **nombre de las variables** (por ejemplo, Edad) es una etiqueta en una sola casilla, para que se pueda encontrarla fácilmente sin saber su actual dirección de memoria.

Tipos de variables

Existen variables de diferentes tipos, tales como *int* (almacena nros enteros), *float* (almacena nros con decimales) o *char* (almacena caracteres). Esta información le dice al compilador cuanto de espacio debe separar o reservar, y que tipo de valor se va a guardar en la variable.
Cada **casilla de memoria** tiene un byte de capacidad. Si el tipo de variable que se crea es de dos bytes de tamaño, este necesita de dos bytes de memoria, o de dos casillas. El tipo de variable (por ejemplo, entero) le dice al compilador cuanta memoria (o cuantas casillas) tiene que reservar para la variable.

Variables, Constantes, Identificadores y Tipos de Datos en C++
En el lenguaje de programación C++, los conceptos de variables, constantes, identificadores y tipos de datos son pilares fundamentales que permiten estructurar y organizar un programa.

Declaración e Inicialización

Como dijimos, las **variables** son elementos que se utilizan para almacenar datos en memoria durante la ejecución de un programa. Cada variable tiene asociado un **tipo de dato**, un **nombre único** (identificador) y un **valor** que puede cambiar a lo largo de la ejecución del programa.

Para usar una variable, primero se debe declarar especificando su tipo y nombre, y opcionalmente asignarle un valor inicial:

```
int edad;           // Declaración de una variable de tipo entero
edad = 25;          // Asignación posterior

float altura = 1.75; // Declaración e inicialización
```

Reglas para Declarar Variables

- El nombre debe comenzar con una letra o un guion bajo (_).
- Puede contener letras, números y guiones bajos, pero no otros caracteres especiales.
- No debe coincidir con palabras clave del lenguaje (como `int`, `if`, `while`).
- C++ distingue entre mayúsculas y minúsculas, por lo que `Edad` y `edad` son identificadores distintos.

Constantes

Una constante es similar a una variable, pero su valor no puede ser modificado una vez definida. En C++, se utiliza la palabra clave `const` para declarar constantes.

Declaración de Constantes

```
const double PI = 3.14159; // Definición de una constante
```

El uso de constantes mejora la legibilidad y seguridad del código, al evitar que ciertos valores sean modificados accidentalmente. Por ejemplo, al trabajar con fórmulas matemáticas, valores como PI o e suelen declararse como constantes.

Identificadores

Los identificadores son los nombres que se asignan a variables, constantes, funciones, clases u otros elementos del programa. Un identificador debe ser único dentro de su contexto (alcance) y seguir ciertas reglas para garantizar la validez del código.

Reglas para Identificadores

- El nombre debe comenzar con una letra o un guion bajo (_).
- Puede contener letras, números y guiones bajos, pero no otros caracteres especiales.
- El nombre del identificador debe ser significativo.
- No debe coincidir con palabras clave del lenguaje (como int, if, while).
- C++ distingue entre mayúsculas y minúsculas, por lo que Edad y edad son identificadores distintos.

Ejemplo de Identificadores Válidos

```
int numero;      // Válido
float _resultado; // Válido
```

Errores Comunes
Usar espacios en los nombres:

int mi variable; // Inválido

Empezar con un número:

float 1dato; // Inválido

Tipos de Datos

En C++, los tipos de datos definen el tipo de valor que puede almacenar una variable y las operaciones que se pueden realizar sobre ella. C++ es un tipo de lenguaje de programación "**fuertemente tipado**", es decir *no se permiten violaciones de los tipos de datos, dado el valor de una variable de un tipo concreto, no se puede usar como si fuera de otro tipo distinto a menos que se haga una conversión.*
Los tipos de datos se dividen principalmente en:

Tabla de Tipos de Datos en C++

Tipo de Dato	Tamaño Aproximado	Rango
`bool`	1 byte	`true` o `false`
`char`	1 byte	`-128` a `127` o `0` a `255`
`unsigned char`	1 byte	`0` a `255`
`short`	2 bytes	`-32,768` a `32,767`
`unsigned short`	2 bytes	`0` a `65,535`
`int`	4 bytes	`-2,147,483,648` a `2,147,483,647`
`unsigned int`	4 bytes	`0` a `4,294,967,295`
`long`	4 u 8 bytes	De -2.147.483.648 a 2.147.483.647
`unsigned long`	4 u 8 bytes	De 0 a 4.294.967.295
`long long`	8 bytes	`-9,223,372,036,854,775,808` a `9,223,372,036,854,775,807`
`unsigned long long`	8 bytes	`0` a `18,446,744,073,709,551,615`
`float`	4 bytes	Aproximadamente `±3.4e-38` a `±3.4e+38`
`double`	8 bytes	Aproximadamente `±1.7e-308` a `±1.7e+308`

Ámbito de las Variables

El ámbito determina la región del programa donde una variable es accesible. En C++ existen tres tipos principales de ámbito:

1. **Local**: Variables declaradas dentro de una función o bloque. Solo son accesibles dentro de ese bloque.
2. **Global**: Variables declaradas fuera de cualquier función. Son accesibles desde cualquier parte del programa.
3. **Estático**: Variables locales que mantienen su valor entre ejecuciones de la función donde están declaradas.

```
#include <iostream>
int global = 10; // Variable global
int main() {
int local = 5; // Variable local
```

```
    std::cout << "Global: " << global << std::endl;
        std::cout << "Local: " << local << std::endl;
        return 0;
}
```

Buenas Prácticas con Variables

- Usa nombres descriptivos que reflejen el propósito de la variable.
  ```
  double velocidadPromedio; // Mejor que "v" o "x"
  ```

- Inicializa las variables antes de usarlas cuando sea necesario.
- Usa constantes para valores que no deben cambiar.
- Minimiza el uso de variables globales para evitar conflictos y errores difíciles de rastrear.

CAPITULO 7 Operadores y expresiones

Un operador es un carácter o grupo de caracteres que actúa sobre una, dos o más variables para realizar una determinada operación con un determinado resultado. Ejemplos típicos de operadores son: suma (+), resta (-), multiplicación (*), etc.

Operadores aritméticos

Los operadores aritméticos son los más sencillos de entender y de utilizar. Todos ellos son operadores binarios.

- Suma: +
- Resta: -
- Multiplicación: *
- División: /
- Resto: % (resto de la división entera). Este operador se aplica solamente a constantes, variables o expresiones de tipo entero.

22 % 4 es = 2, puesto que el resto de dividir de forma entera 22 por 4 es 2. (5 * 4 = 20 + resto = 2 → 22).

Operadores de asignación

Los operadores de asignación atribuyen a una variable el resultado de una expresión o el valor. El operador de asignación más utilizado es el operador de igualdad (=), que no debe ser confundido con la igualdad lógica (==). Su forma general es: **nombre_variable = expresión;**

Operadores relacionales

Una característica imprescindible de cualquier lenguaje de programación es la de considerar alternativas, esto es, la de proceder de un modo u otro según se cumplan o no ciertas condiciones. Los operadores relacionales permiten estudiar si se cumplen o no esas condiciones.
En un programa si una condición se cumple, el resultado es cierto; en caso contrario, el resultado es falso. Un 0 representa la condición de falso, y cualquier número distinto de 0 equivale a la condición cierto. Los operadores relacionales son los siguientes:

- Igual que → ==
- Menor que → <
- Mayor que → >
- Menor o igual que → <=
- Mayor o igual que → >=
- Distinto que → !=

Todos los operadores relacionales son operadores binarios (tienen dos operandos).
Algunos ejemplos:

1. (2 == 1) → falso (0)
2. (2 == 2) → verdadero (1)
3. (3 < 5) → verdadero (1)

4. (3 <= 3) → verdadero (1)
5. (5 != 5) → falso (0)

Operadores lógicos

Los operadores lógicos son operadores que permiten combinar, modificar o negar expresiones lógicas, es decir, expresiones cuyo resultado es Verdadero o Falso.

Se utilizan para construir condiciones compuestas a partir de una o más comparaciones relacionales.

TABLA DE OPERADORES LOGICOS

OPERADOR	OPERACION LOGICA
&&	AND
||	OR
!	NOT

TABLA VERDAD DEL OPERADOR AND

A	B	a AND b
Falso	Falso	Falso
Falso	Verdadero	Falso
Verdadero	Falso	Falso
Verdadero	Verdadero	Verdadero

TABLA VERDAD DEL OPERADOR OR

A	B	a OR b
Falso	Falso	Falso
Falso	Verdadero	Verdadero
Verdadero	Falso	Verdadero
Verdadero	Verdadero	Verdadero

TABLA VERDAD DEL OPERADOR NOT

A	NOT A
Falso	Verdadero
Verdadero	Falso

Los operadores && y || se pueden combinar entre sí mediante paréntesis.
Por ejemplo:
(2==1) || (-1==-1) // el resultado es Verdadero
(2==2) && (3<=1) // el resultado es Falso

Expresiones

Una expresión es todo aquello que se evalúa y devuelve un valor. Existen varios tipos de expresiones de acuerdo lo que contienen.
Las **expresiones aritméticas** consisten en una secuencia de operadores y operandos que especifican una operación determinada. Los operandos pueden ser variables, constantes y los operadores aritméticos son (+ - * / %). Es más sencillo pensar que una expresión aritmética es como una ecuación o una fórmula matemática.
Una expresión aritmética sencilla es: **area = base * altura**
En la anterior línea de código, el resultado de la expresión base * altura se guarda en área

Precedencia de los operadores

Nivel	Operadores	Descripción
1	`*, /, %`	Multiplicación, división, módulo
2	`+, –`	Suma, resta
3	`<, <=, >, >=`	Comparaciones
4	`==, !=`	Igualdad
5	`&&`	AND lógico
6	`||`	OR lógico

Uso de Paréntesis
Los paréntesis tienen la más alta precedencia y se utilizan para alterar el orden de evaluación predeterminado.

CAPITULO 8 Errores

Errores en la Programación y el Proceso de Prueba

El desarrollo de software involucra la identificación y corrección de errores, así como la validación de que un programa cumple con sus requisitos funcionales. Comprender los diferentes tipos de errores y la importancia del proceso de prueba es fundamental para escribir código robusto y confiable.

Tipos de Errores en la Programación

Errores de Sintaxis

Son errores relacionados con la gramática del lenguaje de programación. Ocurren cuando el código no sigue las reglas sintácticas del compilador o intérprete.

Ejemplo:

int x = 10 // Falta el punto y coma al final

Errores Semánticos

Son errores en la lógica del programa, donde las instrucciones no producen el resultado esperado, aunque sean válidas sintácticamente.

Ejemplo:

```
int x = 5, y = 10;
int suma = x - y; // Error lógico: se usa la resta en lugar de la suma
```

Errores de Ejecución

Aparecen cuando el programa se ejecuta y produce resultados inesperados o falla. Estos errores suelen estar relacionados con problemas como divisiones por cero, accesos a memoria inválida, entre otros.

Ejemplo:

```
int a = 10, b = 0;
int resultado = a / b; // Error: División por cero
```

Solución: Implementar verificaciones antes de realizar operaciones críticas.

```
if (b != 0) {
    int resultado = a / b;
} else {
    std::cout << "Error: División por cero" << std::endl;
}
```

El Proceso de Prueba

Las pruebas son esenciales para garantizar que el software funcione según lo esperado. El proceso de prueba incluye varias etapas y enfoques diseñados para detectar y corregir errores.

Tipos de Pruebas

1. Pruebas Unitarias
    - Evalúan componentes individuales (funciones, métodos) de manera aislada.
    - Aseguran que cada unidad del programa funcione correctamente.
2. Pruebas de Integración
    - Verifican cómo interactúan entre sí los diferentes módulos o componentes del sistema.
3. Pruebas Funcionales
    - Evalúan si el software cumple con los requisitos especificados.
4. Pruebas de Estrés
    - Analizan el comportamiento del sistema bajo condiciones extremas o de alta carga.

El manejo de errores y la implementación de pruebas son habilidades críticas en la programación. Detectar y corregir errores en diferentes etapas del desarrollo garantiza la calidad y confiabilidad del software. Por ello, es importante adoptar una metodología rigurosa y apoyarse en herramientas automatizadas cuando sea posible para optimizar estos procesos.

CAPITULO 9 Teorema Fundamental de la Programación Estructurada

El teorema fundamental de la programación estructurada, formulado por Böhm y Jacopini en 1966, establece que cualquier algoritmo puede ser implementado utilizando únicamente tres estructuras de control básicas:

1. Secuencia: Ejecución de instrucciones una tras otra en un orden definido.
2. Selección (Decisión): Elección entre diferentes bloques de código basados en una condición (como `if`, `else`).
3. Iteración (Repetición): Ejecución repetitiva de un bloque de código mientras se cumple una condición (`for`, `while`).

Este teorema eliminó la necesidad del uso indiscriminado de goto, promoviendo la escritura de código más comprensible, estructurado y fácil de mantener.

Unicidad de Puntos de Entrada y Salida

El principio de unicidad de puntos de entrada y salida establece que cada bloque estructurado de código debe tener:

- Un único punto de entrada: La ejecución siempre comienza desde un único lugar.
- Un único punto de salida: La ejecución termina en un único lugar, evitando múltiples `return`, `break` o `goto` que compliquen el flujo del programa.

Beneficios de la Unicidad

1. Legibilidad: El código es más claro y fácil de entender.
2. Mantenibilidad: Facilita la depuración y las modificaciones futuras.
3. Evita errores lógicos: Reduce la probabilidad de dejar instrucciones sin ejecutar o de crear bucles incontrolados.

CAPITULO 10 Primeros pasos en C++

Durante el cursado de esta cátedra se utilizará el lenguaje C++ para la codificación de los algoritmos. A su vez, se diagramarán soluciones utilizando el diagrama de chapín o diagrama de Nassi-Schneiderman cuando se quiera expresar la solución de manera más práctica sin tener en cuenta el lenguaje.
Es por esto que, al conocer las estructuras de control, veremos cómo se escribe en ambas metodologías.
En capítulos anteriores se planteó como comenzar un proyecto en C++ con el IDE CodeBlocks.
Obteniendo lo siguiente:

```
#include <iostream>

using namespace std;

int main()
{
    cout << "Hello world!" << endl;
    return 0;
}
```

Analizaremos estas líneas.

#include <iostream>: #include es una directiva de preprocesador en C++. Que le indica que debe incluir el contenido de un archivo especificado antes de que comience la compilación del programa. En este caso, iostream (iostream.h en otras versiones) archivo de cabecera estándar en C++. Que forma parte de la biblioteca estándar y proporciona definiciones para realizar operaciones de entrada y salida (E/S) a través de flujos, como lectura desde el teclado y escritura en la consola.

using namespace std; Un espacio de nombres es una forma de organizar y agrupar identificadores (como nombres de variables, funciones, clases, etc.) para evitar conflictos de nombres. En C++, muchos elementos estándar, como cout, cin, endl, están definidos dentro del espacio de nombres llamado std (abreviatura de standard). Esto le indica al compilador que, a partir de ese punto, todos los identificadores dentro del espacio de nombres std se pueden usar sin el prefijo std::.Esto simplifica el código y lo hace más legible en ciertos casos.

int main() { } : es la declaración de la función principal de un programa. Es el punto de entrada donde comienza la ejecución de cualquier programa en este lenguaje.
En este caso **int** indica que la función principal devolverá un entero al sistema operativo al finalizar su ejecución. Generalmente, un valor de retorno de 0 indica que el programa se ejecutó correctamente, mientras que valores diferentes a 0 suelen indicar errores.
El nombre **main** es obligatorio como nombre de la función principal en C++ y no se puede cambiar ya que es el punto de inicio del programa que busca el compilador
El resto del código que está entre las {} es un ejemplo que genera el IDE.
Este ejemplo

```
cout << "Hello world!" << endl;
```

Permite probar si todo funciona bien y luego debe ser borrado.

Para probar cualquier código que realicemos utilizaremos el botón que compila y ejecuta el proyecto entero.

Comentarios en C++

En C++, los comentarios son fragmentos de código que no son interpretados ni ejecutados por el compilador. Se utilizan para mejorar la legibilidad del código, documentar su funcionamiento y facilitar su mantenimiento.

Tipos de comentarios en C++

Comentarios de una sola línea
Se escriben usando `//` y todo el texto que sigue en la misma línea es ignorado por el compilador.

```
#include <iostream>
using namespace std;

int main() {
   // Esto es un comentario de una sola línea
   cout << "Hola, mundo!" << endl; // También puede estar al final de una línea de código
   return 0;
}
```

Comentarios de varias líneas (o comentarios en bloque)

Se escriben usando `/*` para abrir y `*/` para cerrar el comentario. Todo el texto dentro de este bloque es ignorado por el compilador.

```
#include <iostream>
using namespace std;

int main() {
   /* Este es un comentario de varias líneas
      Puede extenderse a más de una línea sin problema */
   cout << "Hola, mundo!" << endl;
   return 0;
}
```

Buenas prácticas de los comentarios

- Explicar el propósito del código:
  - Ayuda a otros desarrolladores (o a ti mismo en el futuro) a entender el objetivo del código.
- Documentar funciones y variables:
  - Describir la funcionalidad de funciones y variables ayuda a mejorar la mantenibilidad del código.
- Desactivar temporalmente fragmentos de código:
  - Se puede usar para probar o depurar código sin eliminarlo.

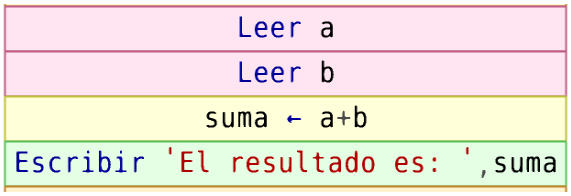
CAPITULO 11 Estructuras de control – Estructuras secuenciales

Las estructuras secuenciales ejecutan instrucciones en el orden en que aparecen en el código, sin bifurcaciones ni repeticiones. Son el tipo de control más simple y directo en la programación.

Características de las estructuras secuenciales

- Se ejecutan de manera **lineal**.
- No hay bifurcaciones ni ciclos.
- Son la base de cualquier programa antes de incluir decisiones o repeticiones.

Ejemplo de un algoritmo secuencial en Chapin y pseudocódigo
Este algoritmo sigue una estructura secuencial porque todas las instrucciones se ejecutan en el orden en que aparecen, sin condiciones ni bucles.




```
Inicio
    Leer A
    Leer B
    Suma ← A + B
    Escribir "El resultado es: ", Suma
Fin
```

Aquí, las instrucciones se ejecutan de manera secuencial, sin ninguna bifurcación o repetición.

Casos de uso comunes:

- Cálculos matemáticos básicos.
- Lectura y escritura de datos
- Procesos que no requieren toma de decisiones.

Implementación en C++

En C++, una estructura secuencial simplemente se traduce en líneas de código ejecutadas en orden.
Ejemplo de código en C++:

```
#include <iostream>
using namespace std;

int main() {
   int a, b, suma;
   cout << "Ingrese el primer número: ";
   cin >> a;
   cout << "Ingrese el segundo número: ";
   cin >> b;
   Suma = A + B;
   cout << "El resultado es: " << Suma << endl;
   return 0;
}
```

Importancia de las Estructuras Secuenciales

Las estructuras secuenciales son esenciales porque:

- Son la base de cualquier algoritmo.
- Son fáciles de entender y depurar.
- Forman la estructura básica antes de introducir condiciones o bucles.

Un programa sin estructuras secuenciales sería caótico, ya que no habría un orden lógico de ejecución.

Salida de Datos en Algoritmia y C++

La **salida de datos** en programación se refiere a la manera en que un programa muestra información al usuario. En **algoritmia** y **C++**, esto se realiza mediante instrucciones específicas que imprimen valores en la pantalla.

En **Chapin**, la salida de datos se logra con la instrucción `Escribir o Mostrar`, que permite mostrar mensajes o valores almacenados en variables.

Ejemplo en Chapin

```
MOSTRAR "Hola" , nombre , " tienes ", edad , " años."
```

```
MOSTRAR "Ingrese su nombre"
```

Ejemplo en C++

En **C++**, la salida de datos se realiza con el objeto **`cout`** (Console Output), que pertenece a la biblioteca `<iostream>`.

```
cout << "Hola, mundo!";
cout<< “Ingrese su nombre”;

cout<< ”Hola ” << nombre << ” tienes ” << edad << “ años”;
```

Caracteres Especiales en la Salida

En C++, podemos usar secuencias de escape para modificar la salida.

Secuencia	Significado
`\n`	Salto de línea
`\t`	Tabulación

\"	Comillas dobles
\\	Barra invertida

Ejemplo con Caracteres Especiales

```
cout << "Hola,\nBienvenido a \"C++\"!\n";
cout << "Esto es una tabulación:\t1\t2\t3\n"
```

La salida sería

Hola,
Bienvenido a "C++"!
Esto es una tabulación: 1 2 3

Formateo de Salida con setprecision

La función **`setprecision()`** se utiliza en C++ para controlar la cantidad de **decimales** que se muestran en la salida de números de tipo **`float`** o **`double`**. Forma parte de la biblioteca **`<iomanip>`** y se usa junto con **`cout`**.

Uso básico de `setprecision()`

Por defecto, `setprecision(n)` muestra **n dígitos en total**, contando los números enteros y decimales.

```
#include <iostream>
#include <iomanip>  // Necesario para setprecision()
using namespace std;

int main() {
   double numero = 123.456789;

   cout << "Sin setprecision(): " << numero << endl;
   cout << "Con setprecision(4): " << setprecision(4) << numero << endl;

   return 0;
}
```

La salida sería

Sin setprecision(): 123.457
Con setprecision(4): 123.5

Uso con fixed para fijar decimales

Si queremos que `setprecision(n)` controle **solamente los decimales**, debemos usar `fixed`.

```
#include <iostream>
```

```
#include <iomanip>
using namespace std;

int main() {
   double numero = 123.456789;

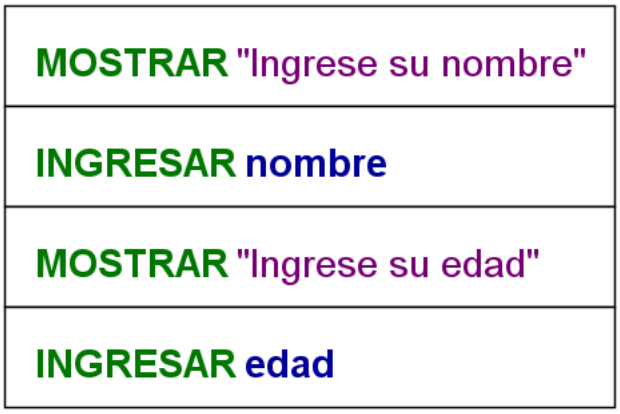
   cout << fixed << setprecision(2);
   cout << "Con fixed y setprecision(2): " << numero << endl;
   return 0;
}
```

La salida sería
Con fixed y setprecision(2): 123.46

Ingreso de Datos en Algoritmia y C++

En Chapin, el ingreso de datos se realiza con la instrucción `Leer o Ingresar`, que permite capturar valores introducidos por el usuario y almacenarlos en variables.

Ejemplo en Chapin



Ejemplo en C++

En C++, la entrada de datos se realiza mediante el objeto **`cin`** (Console Input), que forma parte de la biblioteca `<iostream>`.

```
#include <iostream>
using namespace std;

int main() {
   string nombre;
   int edad;
   cout << "Ingrese su nombre: ";
   cin >> nombre;
```

```
    cout << "Ingrese su edad: ";
    cin >> edad;
    return 0;
}
```

Problema con cin **y cadenas de texto:**

Si se quiere leer una cadena con espacios, `cin >>` no funciona correctamente, ya que solo captura la primera palabra. Para solucionar esto, se usa `getline(cin, variable)`.

Ejemplo con `getline()` para cadenas con espacios

```
#include <iostream>
using namespace std;

int main() {
    string nombreCompleto;
    cout << "Ingrese su nombre completo: ";
    cin.ignore();  // Limpia el buffer de entrada
    getline(cin, nombreCompleto);
    cout << "Hola, " << nombreCompleto << "." << endl;
    return 0;
}
```

Diferencia entre cin >> y getline()

Método	¿Cuándo usarlo?	Características
`cin >>`	Para leer una palabra o número.	No admite espacios en cadenas.
`getline(cin, variable)`	Para leer una línea completa de texto.	Permite capturar nombres completos o frases.

Conclusión

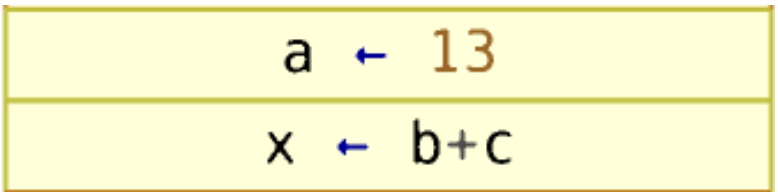
- En **algoritmia**, el ingreso de datos se realiza con la instrucción `Leer`.
- En **C++**, la entrada de datos se maneja con `cin`, pero para capturar cadenas completas se recomienda `getline()`.
- Se deben considerar los diferentes tipos de datos y sus particularidades al ingresar información en un programa.

Asignación en Algoritmia y C++

La **asignación** en programación es el proceso de **darle un valor a una variable**. Se usa el operador = para asignar un valor en memoria que luego puede ser usado o modificado.

En **CHAPIN** , la asignación de valores a una variable se hace con el operador ← (flecha hacia la izquierda).

Ejemplo en Chapin



Ejemplo en C++

```
a = 13;
x = b + c;
```

Otros ejemplo

```
int numero = 10;
double precio = 99.99;
char letra = 'A';
string nombre = "Juan";
string mensaje = saludo + ", " + nombre + "!";
```

SCANF Y PRINTF

Aunque en C++ solemos usar **cin** y **cout** para la entrada y salida de datos, C mantiene sus funciones tradicionales: **scanf** para leer y **printf** para escribir datos. Estas funciones siguen siendo ampliamente usadas en entornos donde se requiere mayor control sobre el formato o compatibilidad con C.

Ventajas de scanf y printf:

- Control total sobre el formato de entrada y salida.
- Muy utilizadas en bibliotecas de C y programación de sistemas.
- Compatibles con C y C++.

Sintaxis

`scanf`

```
scanf("formato", &variable);
```

Nota: Es obligatorio usar el operador `&` para pasar la dirección de memoria de la variable (excepto en arreglos de caracteres, como strings en C).

`printf`

```
printf("formato", variable);
```

Especificadores de formato

Tipo de dato	Especificador
`int`	`%d`
`float`	`%f`
`double`	`%lf`
`char`	`%c`
`char[]` (cadena)	`%s`

Ejemplos básicos

Ingreso de datos con `scanf`

```
#include <cstdio>

int main() {
    int edad;
    printf("Ingrese su edad: ");
    scanf("%d", &edad);
    printf("Su edad es: %d\n", edad);
    return 0;
}
```

Salida de datos con `printf`

```
#include <cstdio>

int main() {
    float pi = 3.1416;
    printf("El valor de pi es: %.2f\n", pi);
    return 0;
}
```

Nota: `%.2f` muestra 2 cifras decimales.

Ejemplo completo

```
#include <cstdio>

int main() {
    int entero;
    float decimal;
    char letra;
    char nombre[20];

    printf("Ingrese un numero entero: ");
    scanf("%d", &entero);

    printf("Ingrese un numero decimal: ");
    scanf("%f", &decimal);

    printf("Ingrese una letra: ");
    scanf(" %c", &letra);   // OJO: espacio antes de %c para limpiar buffer
```

```
    printf("Ingrese su nombre: ");
    scanf("%s", nombre); // No se usa & porque nombre es un arreglo

    printf("\n---- Datos ingresados ----\n");
    printf("Entero: %d\n", entero);
    printf("Decimal: %.2f\n", decimal);
    printf("Letra: %c\n", letra);
    printf("Nombre: %s\n", nombre);

    return 0;
}
```

Atención: detalles importantes

✓ **Espacio en `scanf(" %c", &letra);`**
Se recomienda colocar un espacio antes de `%c` para ignorar el salto de línea pendiente en el buffer.

✓ **Uso de `&` en `scanf`**
Necesario para variables simples porque `scanf` necesita la dirección de memoria donde guardar el dato.

✓ **Strings**
Los arreglos de `char` se pasan sin `&` porque el nombre del arreglo ya es la dirección.

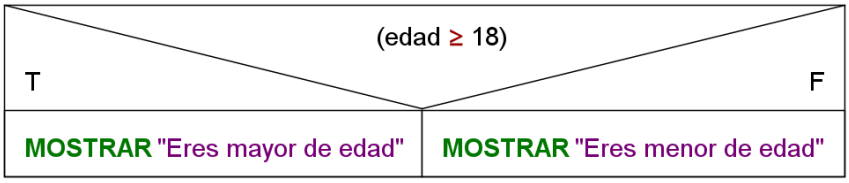
Control de formato en `printf`

Formato	Significado	Ejemplo
`%.2f`	2 decimales en un float	`3.14`
`%5d`	Ancho de 5 caracteres para un entero	`42`
`%-5d`	Ancho de 5, alineado a la izquierda	`42`
`%10s`	Ancho de 10 para un string	`nombre`

CAPITULO 12 Estructuras de control – Estructuras de decisión

Las **estructuras de decisión** permiten que un programa **tome decisiones** en función de ciertas condiciones. En **algoritmia** y **C++**, esto se logra con sentencias como `Si - Entonces` y `if - else`. En **CHAPIN**, la estructura de decisión más usada es `Si - Entonces`, que evalúa una condición y ejecuta instrucciones si se cumple.

Ejemplo en Chapin



Ejemplo en C++

En **C++**, la estructura de decisión básica es `if - else`

```
if (edad >= 18) {
   cout << "Eres mayor de edad." << endl;
} else {
   cout << "Eres menor de edad." << endl;
}
```

Uso de if - else if - else en C++

```
if (edad < 13) {
    cout << "Eres un niño." << endl;
} else if (edad < 18) {
    cout << "Eres un adolescente." << endl;
} else {
    cout << "Eres un adulto." << endl;
}
```

Operadores Relacionales y Lógicos

Para lograr un correcto uso de la estructura condicional es conveniente realizar un repaso de los operadores ya que son fundamentales para poder expresar la decisión/condición correcta.

Las condiciones pueden usar operadores **relacionales** y **lógicos.**

OPERADORES RELACIONALES

Operador	Significado
`==`	Igual a
`!=`	Diferente de
`>`	Mayor que
`<`	Menor que
`>=`	Mayor o igual que
`<=`	Menor o igual que

OPERADORES LOGICOS

OPERADOR	OPERACION LOGICA
&&	AND
||	OR
!	NOT

TABLA VERDAD DEL OPERADOR && / AND

A	B	a AND b
Falso	Falso	Falso
Falso	Verdadero	Falso
Verdadero	Falso	Falso
Verdadero	Verdadero	Verdadero

TABLA VERDAD DEL OPERADOR || / OR

A	B	a OR b
Falso	Falso	Falso
Falso	Verdadero	Verdadero
Verdadero	Falso	Verdadero
Verdadero	Verdadero	Verdadero

TABLA VERDAD DEL OPERADOR ! / NOT

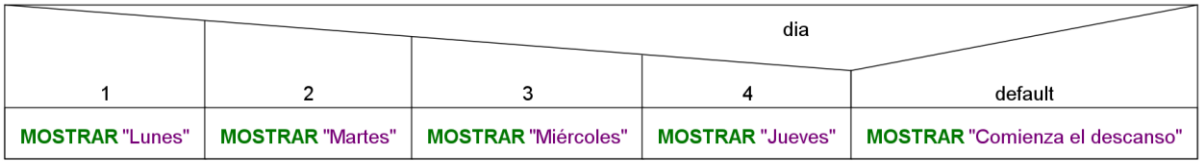
A	NOT A
Falso	Verdadero

Verdadero	Falso

CAPITULO 13 Estructuras de control – Estructuras de selección múltiple.

La **estructura de decisión múltiple** `switch` se utiliza cuando se quiere comparar **un valor con múltiples opciones** y ejecutar código según la coincidencia.

Ejemplo en Chapin



Ejemplo en C++

```
switch (dia) {
    case 1:
      cout << "Lunes" << endl;
      break;
    case 2:
      cout << "Martes" << endl;
      break;
    case 3:
      cout << "Miércoles" << endl;
      break;
    case 4:
      cout << "Jueves" << endl;
      break;
    default:
      cout << "Comienza el descanso." << endl;
  }
```

Diferencia entre if-else y switch

Característica	if-else	switch
Condiciones	Permite cualquier comparación (<, >, ==, etc.)	Solo compara valores exactos (==)
Tipos de datos	Funciona con cualquier tipo (int, float, string)	Solo con int, char, enum
Legibilidad	Bueno para comparaciones complejas	Mejor para muchas opciones fijas
Eficiencia	Más lento en muchas comparaciones	Más rápido para casos específicos

CAPITULO 14 Contadores y acumuladores

Contadores

Llamamos contadores a una variable que iremos **incrementando manualmente de 1 en 1** dentro de un ciclo de repetición. Para incrementar una variable numérica, tendremos que asignarle su valor actual más 1.
Hay que tener en cuenta que la variable debe ser inicializada. En general, esta inicialización comienza con el valor 0.

Ejemplo en C++

```
cont = cont + 1;
cont++;
```

Acumuladores

Llamamos acumuladores a una variable que iremos incrementando en cantidades variables dentro de un ciclo de repetición. Para incrementar una variable numérica, tendremos que asignarle su valor actual más 1.

Ejemplo en C++

```
acum = acum + edad;
acum +=edad;
```

CAPITULO 15 Estructuras de control – Estructuras repetitivas

Las estructuras repetitivas o bucles permiten ejecutar un bloque de código varias veces mientras se cumpla una condición específica. Estas estructuras son fundamentales en la programación porque permiten automatizar tareas repetitivas sin necesidad de escribir el mismo código múltiples veces.

Las estructuras repetitivas se pueden clasificar en 2 tipos:

- Aquellas que se utilizan cuando se conoce la cantidad de repeticiones a realizar (PARA o FOR)
- Aquellas que se utilizan cuando se NO conoce la cantidad de repeticiones a realizar (MIENTRAS/WHILE o HACER MIENTRAS (DO WHILE)

Estructura WHILE

La estructura **while/mientras** Se utiliza cuando no sabemos cuántas veces se repetirá el bucle y queremos que se ejecute mientras se cumpla una condición.
El bucle **while/mientras** evalúa una condición antes de ejecutar el bloque de instrucciones. Si la condición es verdadera, el código dentro del bucle se ejecuta; de lo contrario, el bucle finaliza y el programa continúa con la siguiente instrucción después del bucle.

Ejemplo en Chapin

```
int respuesta
INGRESAR respuesta
MIENTRAS respuesta == 1
    MOSTRAR "Hola mundo"
    INGRESAR respuesta
```

Ejemplo en C++

```
#include <iostream>
using namespace std;

int main() {
   int respuesta;
  cin>>respuesta;
   while (respuesta == 1) {

      cout << "Hola Mundo " << endl;
      cin>>respuesta;

   }

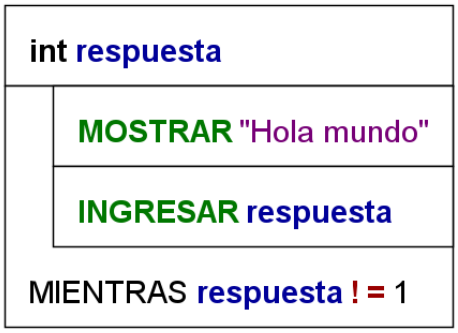
   return 0;
}
```

Estructura DO - WHILE

La estructura **do-while/hacer-mientras** Se utiliza cuando no sabemos cuántas veces se repetirá el bucle y queremos que se ejecute mientras se cumpla una condición.

Asegura que el código se ejecute al menos una vez, incluso si la condición es falsa desde el principio. Esto se debe a que la evaluación de la condición ocurre después de la ejecución del bloque de código.

Ejemplo en Chapin



Ejemplo en C++

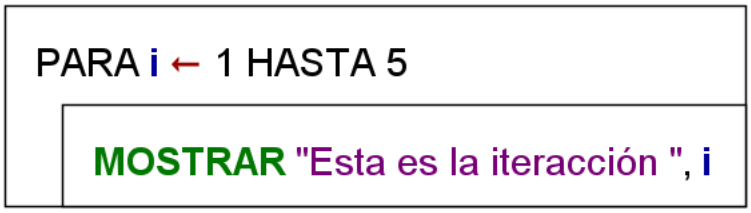
```
#include <iostream>
using namespace std;

int main() {
   int respuesta;
   do {
      cout << "Hola Mundo " << endl;
      cin>>respuesta;
   } while (respuesta !=  1);
   return 0;
}
```

Estructura FOR

Se usa cuando conocemos la cantidad exacta de repeticiones y queremos recorrer un rango de valores con un control preciso de la variable de iteración. Es útil para recorrer arreglos, realizar cálculos iterativos y ejecutar bloques de código un número determinado de veces.

Ejemplo en Chapin



Ejemplo en C++

```
for (int i = 1; i <= 5; i++) {
    cout << "Iteración: " << i << endl;
}
```

Explicación:

1. La variable `i` se inicializa en 1.
2. Se verifica si `i <= 5`. Si es verdadero, se ejecuta el bloque de código.
3. Después de ejecutar el bloque, `i` se incrementa en 1.
4. Se repiten los pasos 2 y 3 hasta que `i` sea mayor que 5.

Variaciones del `for`:

- La inicialización o el valor final del for pueden ser variables

```
int valInicial = 10
int valFinal = 20
for (int i = valInicial; i <= valFinal; i++) {
    cout << "Iteración: " << i << endl;
}
```

- Se pueden usar decrementos o pasos personalizados.

```
for (int i = 5; i >= 1; i--) {
   cout << "Cuenta regresiva: " << i << endl;
}
```

```
for (int i = 0; i <= 10; i += 2) {
   cout << "Número par: " << i << endl;
}
```

Buen uso de la estructura FOR

Aunque el lenguaje lo permite, no es aconsejable modificar el valor de la variable de control. Tampoco es recomendable colocar en el control de finalización de la estructura (ej i<=10) una condición más compleja.

En ambos casos no se respeta la esencia de la estructura de realizar la iteración un número determinado de veces. Si se considera que no se sabe la cantidad de iteraciones se debe realizar con otra estructura (while, por ejemplo)

Estructuras de repetición anidadas

Cuando una estructura de repetición está dentro de otra se denomina estructuras de repetición anidadas.

Y la estructura interior se repetirá tantas veces como indique la estructura exterior.

CAPITULO 16 Conversión entre Tipos de Datos Primitivos en C++

En C++, la conversión entre tipos de datos primitivos puede ser explícita o implícita.

Conversión Implícita

Ocurre automáticamente cuando el compilador convierte un tipo de dato a otro de manera segura.

```
int entero = 42;
double decimal = entero;
cout << decimal; // Salida: 42.0
```

Conversión Explícita (Casting)

En C++, el **casting** (conversión de tipos) permite transformar un valor de un tipo de dato a otro. Esto es especialmente útil cuando necesitas realizar operaciones entre tipos diferentes o cuando deseas manejar un valor de una manera específica.

Tipos de Casting en C++

Existen varios tipos de casting en C++ de los cuales veremos los 2 principales:

1. **Casting C-style:** Utiliza la sintaxis `(tipo_dato)valor`.

```
#include <iostream>
using namespace std;

int main() {
   double numero = 9.75;
   int entero = (int)numero;  // C-style cast
   cout << "Número convertido: " << entero << endl;
   return 0;
}
```

Desventaja: No es seguro porque no distingue entre tipos de casting y puede generar errores si se usa incorrectamente.

2. **Casting con static_cast**

Es el tipo de casting más seguro y recomendado para conversiones en tiempo de compilación.

```
#include <iostream>
using namespace std;

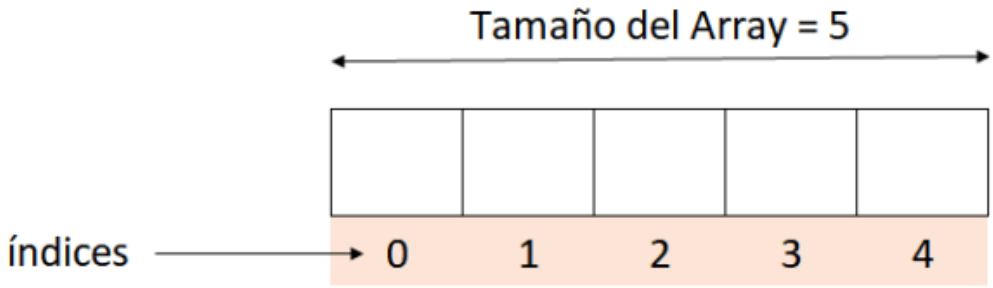
int main() {
   double numero = 9.75;
   int entero = static_cast<int>(numero);  // Conversión explícita
   cout << "Número convertido con static_cast: " << entero << endl;
   return 0;
}
```

Ventaja: Verifica la validez de la conversión en tiempo de compilación.

CAPITULO 17 Arreglos

Los **arreglos** son estructuras de datos que permiten almacenar múltiples valores del mismo tipo bajo un mismo nombre. Son fundamentales en la programación porque permiten organizar y manipular grandes cantidades de datos de manera eficiente.
Los arreglos son estructuras de datos **homogéneas**, es decir, todas sus componentes son del mismo tipo. Como una estructura de datos estática su espacio debe definirse en tiempo de compilación
¿Por qué utilizar este tipo de estructura de datos?. Supongamos la siguiente situación; se requiere tener en memoria interna y en forma **simultánea** los datos de los Apellidos, Nombres de 10 alumnos. Mantener todos estos datos al mismo tiempo en la memoria interna requerirá de 10 variables, pero cada una de ellas con un nombre diferente, EJ: ApeNom1, ApeNom2, ..., ApeNom10.
Notamos que esta forma de notación haría muy compleja la tarea de programar, imaginemos ingresar estos datos, emitirlos, procesarlos y si en vez de 10 fueran 100 ó 1000 o más, **¡mucho más complicado!**. Entonces, bajo estas situaciones, cuando contemos con una colección de datos todas del mismo tipo emplearemos los arreglos.
Podemos decir, entonces que los arreglos son colecciones o disposiciones de datos homogéneas ubicadas en la memoria interna y de manera **continua** la cual se le asigna un **nombre único o genérico** que hace referencia a esa región de memoria reservada en tiempo de compilación. Ahora bien, para poder referirnos a un elemento o componente se utiliza un **subíndice**, siendo la notación formada por el nombre genérico seguido del subíndice encerrado entre corchetes.
Por ejemplo, si tomamos el caso anterior, el nombre genérico podría ser *ApeNom* y el subíndice podría ser *i*; por lo tanto, para indicar un elemento del arreglo lo escribiríamos como se indica a continuación ***ApeNom[i]***. De esta manera al tener que emitir por ejemplo cada uno de los nombres de los alumnos, basta con variar el valor del subíndice para hacer referencia a cada elemento del arreglo.

Gráficamente lo podemos representar así:



En C++, los **arreglos** se declaran de manera similar a las variables, pero agregando corchetes `[]` para indicar la cantidad de elementos.

```
int miArreglo[5];
```

Aquí, `miArreglo` es un arreglo que puede almacenar 5 enteros.

Inicialización y Asignación

```
miArreglo[0] = 10;
miArreglo[1] = 20;
miArreglo[2] = 30;
miArreglo[3] = 40;
miArreglo[4] = 50;
```

También podemos inicializar un arreglo en una sola línea:

```
int miArreglo[5] = {10, 20, 30, 40, 50};
```

Si al declarar el array, se omite el tamaño, el array tendrá el tamaño de la lista inicializadora.

```
int miArreglo[] = {10, 20, 30, 40, 50, 60, 70}; // Tiene un tamaño
de 7
```

Recorrido de un Arreglo con un Bucle

```
#include <iostream>
using namespace std;

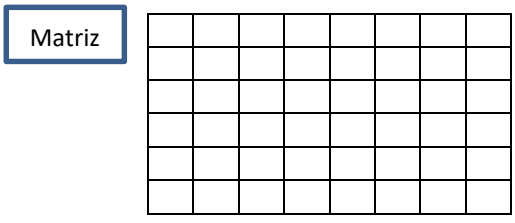
int main() {
   int miArreglo[5] = {10, 20, 30, 40, 50};
   for (int i = 0; i < 5; i++) {
      cout << "Elemento en la posición " << i << ": " << miArreglo[i] << endl;
   }
   return 0;
}
```

Características y Consideraciones

- Los arreglos tienen **un tamaño fijo** que debe definirse al declararlos.
- **Los índices comienzan en 0**, es decir, el primer elemento es `miArreglo[0]`.
- Intentar acceder a un índice fuera del rango del arreglo puede generar errores.
- En C++, los arreglos pueden ser de **cualquier tipo de dato** (`int`, `float`, `char`, etc.).

Arrays de dos dimensiones

Un array en C++ puede tener una, dos o más dimensiones.
Por ejemplo, un array de dos dimensiones también denominado **matriz**, es interpretado como un array (unidimensional) de dimensión "f" (número de filas), donde cada componente es un array (unidimensional) de dimensión "c" (número de columnas).
Un array de dos dimensiones, contiene, pues, "f*c" componentes.



C (8 columnas)

El formato para declarar un array multidimensionales: int nombre[f][c]
Ej

```
int matriz [6][8];
```

Para acceder a un elemento del array multidimensional: nombre[i][j]
Ej

```
matriz [3][4] = 63;
```

Durante la declaración de un array multidimensional también podemos inicializar sus componentes indicando la lista de los valores entre llaves. En el interior de la lista, los componentes de cada línea del array son encerrados nuevamente entre llaves. Para hacer más cara la visibilidad de los elementos del array, podemos indicarlos en varias líneas.

```
int matrizB [3][4] ={{ 0,1,2,3},
                     { 1,2,3,4},
                      { 2,3,4,5}
                     };
```

Sin embargo, es mucho más conveniente anidar dos ciclos para inicializar un array de dos dimensiones:

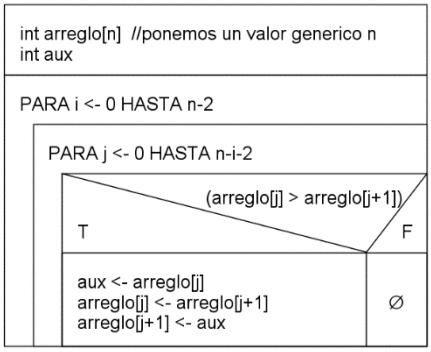
```
for (i = 0; i< 3; i++) {
    for (j = 0; j< 4; j++){
                matrizB [i][j] = i+j;
    }
}
```

Ordenamiento de Arreglos

Los métodos de ordenamiento son algoritmos que realizan la operación de arreglar los registros de una tabla en algún orden secuencial de acuerdo a un criterio de ordenamiento. El ordenamiento se efectúa con base en el valor de algún campo en un grupo de datos. El ordenamiento puede estar dado de forma iterativa o recursiva según la naturaleza y forma de ejecución del mismo.
Estos son los ejemplos más comunes de métodos de ordenamiento junto con sus características principales

Método de Ordenamiento por Intercambio

El **ordenamiento por intercambio** es uno de los algoritmos más simples para ordenar elementos en un arreglo. La idea principal es comparar cada par de elementos adyacentes e intercambiarlos si están en el orden incorrecto.



Ejemplo en C++

```
for (i = 0; i < n - 1; i++) {
    for (j = 0; j < n - i - 1; j++) {
        if (arreglo[j] > arreglo[j + 1]) {
            aux = arreglo[j];
            arreglo[j] = arreglo[j + 1];
            arreglo[j + 1] = aux;
        }
    }
}
```

Representación

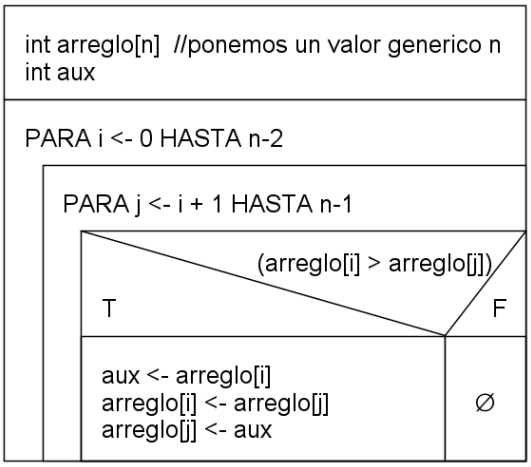
Sub	Estados intermedios del Vector															
0	7	7	7	7	7	7	3	3	3	3	3	3	3	1	1	**1**
1	10	10	3	3	3	3	7	6	6	6	6	1	1	3	3	**3**
2	3	3	10	6	6	6	6	7	1	1	1	6	5	5	5	**5**
3	6	6	6	10	1	1	1	1	7	5	5	5	6	6	6	**6**
4	1	1	1	1	10	5	5	5	5	7	7	7	7	7	7	**7**
5	5	5	5	5	5	10	10	10	10	10	10	10	10	10	10	**10**
i	0					1				2			3		4	
j	0	1*	2*	3*	4*	0*	1*	2*	3*	0	1*	2*	0*	1*	0	

En la tabla, cada vez que hay un * significa que hay un intercambio entre los valores sombreados, lo que se ve reflejado en la siguiente iteración de j

Método de Ordenamiento de Selección por intercambio Directo.

Para cada posición i del arreglo (excepto la última):

- Compara arreglo[i] con todos los elementos siguientes (j = i+1 hasta n-1).
- Si encuentra uno más chico (arreglo[j] < arreglo[i]), los intercambia inmediatamente.



Ejemplo en C++

Dado un arreglo de cuyo máximo subíndice es n (o sea n+1 elementos)

```
for (int i = 0; i < n - 1; i++) {
    for (int j = i + 1; j < n; j++) {
        if (arreglo[i] > arreglo[j]) {
            int temp = arreglo[i];
            arreglo[i] = arreglo[j];
            arreglo[j]= temp;
        }
    }
}
```

Este método hace **más intercambios que el selection sort clásico**, porque intercambia cada vez que encuentra un valor menor, en lugar de esperar al mínimo.

EJEMPLO

Sub	Estados intermedios del Vector															
0	7	7	3	3	1	1	1	1	1	1	1	1	1	1	1	**1**
1	10	10	10	10	10	10	7	6	3	3	3	3	3	3	3	**3**
2	3	3	7	7	7	7	10	10	10	10	7	6	5	5	5	**5**
3	6	6	6	6	6	6	6	7	7	7	10	10	10	7	6	**6**
4	1	1	1	1	3	3	3	3	6	6	6	7	7	10	10	**7**
5	5	5	5	5	5	5	5	5	5	5	5	5	6	6	7	**10**
i	0					1				2			3		4	
j	1	2 *	3	4*	5	2*	3*	4*	5	3*	4*	5*	4*	5*	5*	

En la tabla, cada vez que hay un * significa que hay un intercambio entre los valores sombreados, lo que se ve reflejado en la siguiente iteración de j

Método de Ordenamiento de Selección (SelectionSort)

El **ordenamiento por selección clásica (Selection Sort)** es un algoritmo que ordena un arreglo buscando, en cada pasada, el valor mínimo entre los elementos no ordenados y colocándolo en su posición definitiva mediante un solo intercambio. A diferencia de otros métodos por intercambio, como el de burbuja o el intercambio inmediato, este algoritmo **minimiza la cantidad de intercambios** al realizar solo uno por iteración externa, lo que mejora su eficiencia relativa.

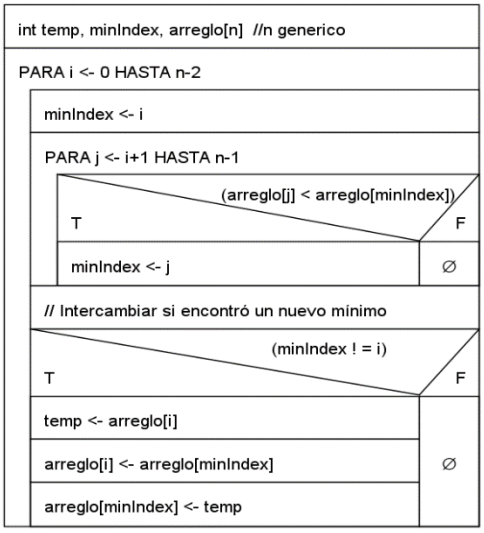
Ejemplo en C++

```
for (int i = 0; i < n - 1; i++) {
    int minIndex = i;

    // Encontrar el índice del elemento restante más pequeño

    for (int j = i + 1; j < n; j++) {
        if (arreglo[j] < arreglo[minIndex]) {
            minIndex = j;
        }
    }

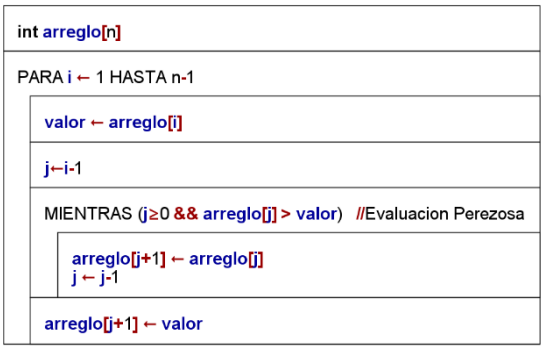
    // si el índice minindex cambio a j hago el intercambio de valores
    if (minIndex != i) {
        int temp = arreglo[i];
        arreglo[i] = arreglo[minIndex];
        arreglo[minIndex] = temp;
    }
}
```



Sub	Estados intermedios del Vector																			
0	7	7	7	7	7	1	1	1	1	1	1	1	1	1	1	1	1	1	1	**1**
1	10	10	10	10	10	10	10	10	10	10	3	3	3	3	3	3	3	3	3	**3**
2	3	3	3	3	3	3	3	3	3	3	10	10	10	10	5	5	5	5	5	**5**
3	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	**6**
4	1	1	1	1	1	7	7	7	7	7	7	7	7	7	7	7	7	7	7	**7**
5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	10	10	10	10	10	**10**
i	0						1					2				3			4	
j	1	2	3	4	5	Interc.	2	3	4	5	Interc.	3	4	5	Interc.	4	5	Interc.	5	Interc.
minIndex	0	0	2	2	4	0 - 4	1	2	2	3	1-2	2	3	3	2 - 5	3	3	Sin	4	Sin

Método de Ordenamiento de Inserción (InsertionSort)

El ordenamiento por inserción (Insertion Sort) es un algoritmo que construye la lista ordenada de izquierda a derecha, tomando cada nuevo elemento y comparándolo con los anteriores para insertarlo en su posición correcta. En cada paso, los elementos mayores al valor actual se desplazan hacia la derecha para hacerle espacio. Este método es eficiente con listas pequeñas o parcialmente ordenadas, y es de tipo estable, ya que no altera el orden relativo de los elementos iguales.



Ejemplo en C++

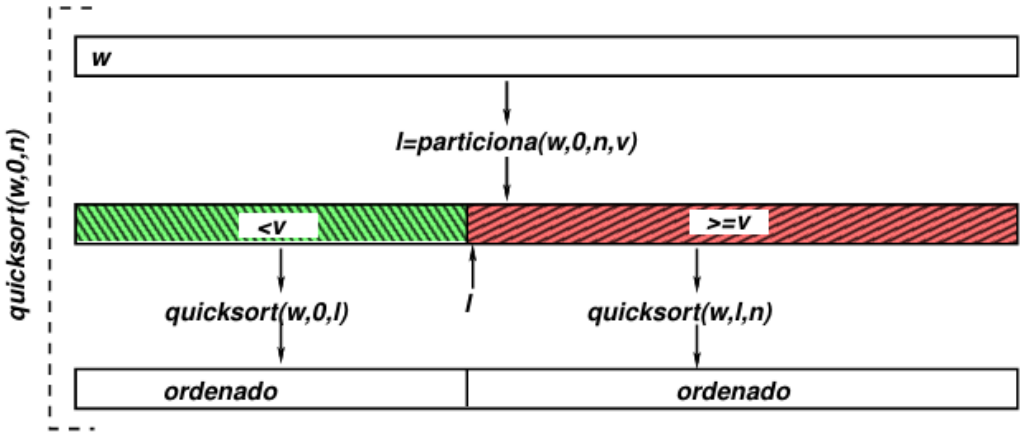
```
for (int i = 1; i < n; i++) {
    int valor = arreglo[i];
    int j = i - 1;

    // Desplazar elementos mayores al valor hacia la derecha
    while (j >= 0 && arreglo[j] > valor) {
        arreglo[j + 1] = arreglo[j];
        j--;
    }

    // Colocar el valor en su posición correcta
    arreglo[j + 1] = valor;
  }
}
```

Sub	Estados intermedios del Vector																					
0	7	7	7	7	7	3	3	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
1	10	10	10	10	7	7	7	7	7	6	6	6	6	6	3	3	3	3	3	3	3	3
2	3	3	3	10	10	10	10	10	7	7	7	7	7	6	6	6	6	6	6	6	5	5
3	6	6	6	6	6	6	6	10	10	10	10	10	7	7	7	7	7	7	7	6	6	6
4	1	1	1	1	1	1	1	1	1	1	1	10	10	10	10	10	10	10	7	7	7	7
5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	10	10	10	10	10
Valor	10	10	3	3	3	3	6	6	6	6	1	1	1	1	1	1	5	5	5	5	5	

Método de Ordenamiento Quick Sort



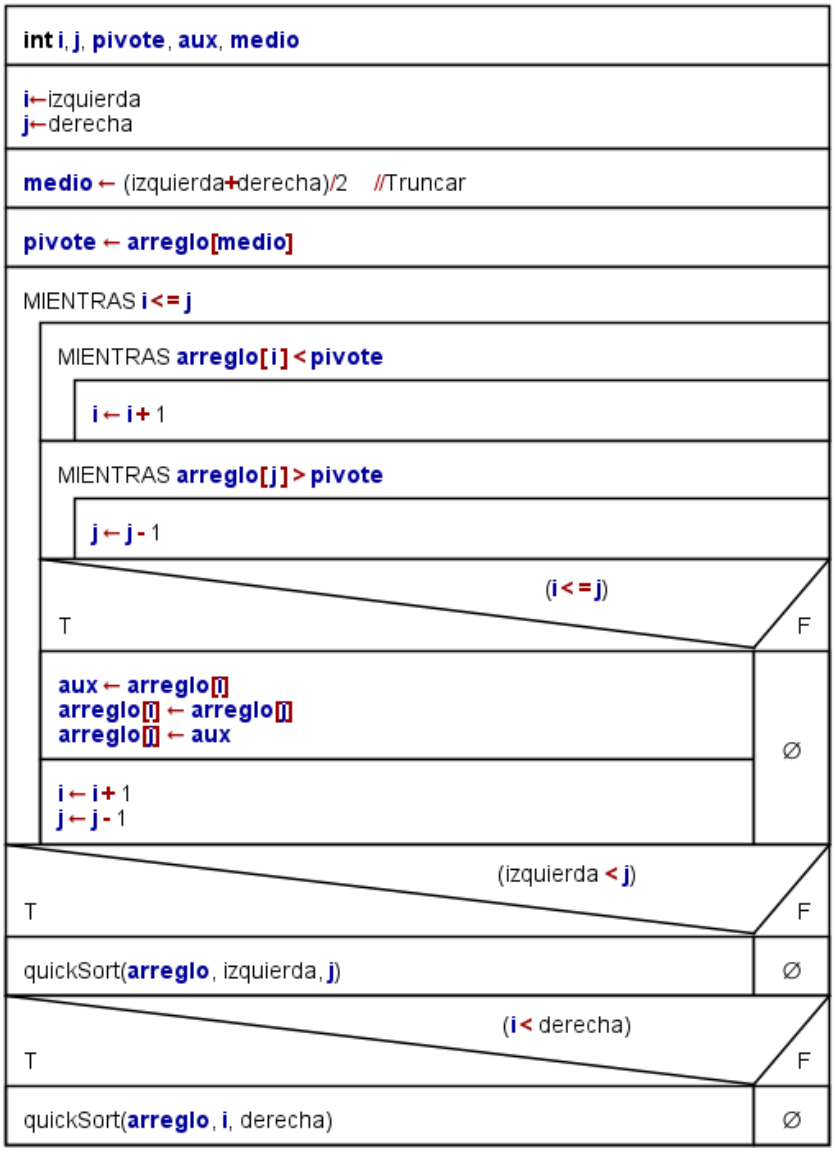
Este es probablemente uno de los algoritmos de ordenamiento más usados y se basa en la estrategia de tipo **divide y vencerás**. Se elige un elemento del vector v llamado "pivote" y se "particiona" el vector de manera de dejar los elementos >= v a la derecha (rango [l n), donde l es la posición del primer elemento de la partición derecha) y los < v a la izquierda (rango [0 l)). Está claro que, a partir de entonces, los elementos en cada una de las particiones quedarán en sus respectivos rangos, ya que todos los elementos en la partición derecha son estrictamente mayores que los de la izquierda. Podemos entonces aplicar quick-sort recursivamente a cada una de las particiones.

Quick Sort (ordenamiento rápido) es un algoritmo de tipo **divide y vencerás**. Funciona de forma **recursiva** y se basa en:

1. Elegir un **pivote** (un valor del arreglo).
2. Reordenar (particionar) los elementos:
    a. A la izquierda del pivote van los elementos **menores**.
    b. A la derecha, los **mayores**.
3. Aplicar el mismo proceso a cada mitad.

Ejemplo en Chapin

Procedimiento QuickSort(int arreglo[ ] porRef, int izquierda, int derecha)



Ejemplo en C++

```
void quicksort(int arreglo[], int izquierda, int derecha) {
   int i = izquierda;
   int j = derecha;
   int medio = (izquierda + derecha) / 2;
   int pivote = arreglo[medio];
   int temp;

   while (i <= j) {
      while (arreglo[i] < pivote) {
         i++;
      }
      while (arreglo[j] > pivote) {
         j--;
      }
      if (i <= j) {
         // Intercambiar
         temp = arreglo[i];
         arreglo[i] = arreglo[j];
         arreglo[j] = temp;
         i++;
         j--;
      }
   }

   if (izquierda < j) {
      quicksort(arreglo, izquierda, j);
   }
   if (i < derecha) {
      quicksort(arreglo, i, derecha);
   }
}
```

EJEMPLO VISUAL

Con color verde se muestra el valor de **i**, con rojo el de **j** y se le coloca borde al sector del vector que está ordenando el algoritmo.

IZQUIERDA= 0 , DERECHA = 6

	Indice	0	1	2	3	4	5	6
Pivote 7		8	3	1	7	0	10	2
		2	3	1	7	0	10	8
		2	3	1	7	0	10	8
		2	3	1	7	0	10	8
		2	3	1	7	0	10	8
		2	3	1	7	0	10	8
		2	3	1	7	0	10	8
		2	3	1	0	7	10	8
Pivote 3		2	3	1	0	7	10	8
		2	3	1	0	7	10	8
		2	3	1	0	7	10	8
		2	0	1	3	7	10	8
		2	0	1	3	7	10	8
		2	0	1	3	7	10	8
Pivote 0		2	0	1	3	7	10	8
		2	0	1	3	7	10	8
		0	2	1	3	7	10	8
Pivote 2		0	2	1	3	7	10	8
		0	1	2	3	7	10	8
Pivote 10		0	1	2	3	7	10	8
		0	1	2	3	7	10	8
		0	1	2	3	7	8	10
		0	1	2	3	7	8	10
Pivote 7		0	1	2	3	7	8	10
		0	1	2	3	7	8	10
		0	1	2	3	7	8	10
		0	**1**	**2**	**3**	**7**	**8**	**10**

El **pivote** es **clave** en el algoritmo Quick Sort porque determina **cómo se divide el arreglo** en cada paso. Su elección **afecta directamente la eficiencia** del algoritmo.

Si el pivote **divide en mitades iguales**, el rendimiento es **óptimo**: `O(n log n)`. Si el pivote **es el mayor o menor valor**, una de las mitades queda vacía → **peor caso**: $O(n^2)$.
Existen estrategias para la elección del pivote. Por ejemplo, **mediana de tres** y **selección aleatoria**, son técnicas inteligentes para elegir el **pivote** en Quick Sort y **mejorar su rendimiento promedio**, especialmente cuando el arreglo tiene ciertos patrones.

Mediana de tres consiste en tomar **tres elementos** del arreglo: el **primero, el último y el del medio.** Y usar la **mediana** como pivote.

Selección aleatoria consiste en elegir el pivote **al azar** dentro del subarreglo que se está ordenando. Evita el sesgo de entrada: no importa si el arreglo está ordenado, invertido o tiene duplicado

Estrategias comunes para elegir el pivote

Estrategia	Comentario
Primer o último elemento	Simple, pero propenso al peor caso
Elemento del medio	Mejor, pero aún sensible al orden
Aleatorio (`rand()`)	Evita patrones, mejora rendimiento promedio
Mediana de tres (inicio, medio, fin)	Muy usada en implementaciones reales

Búsquedas en arrays.

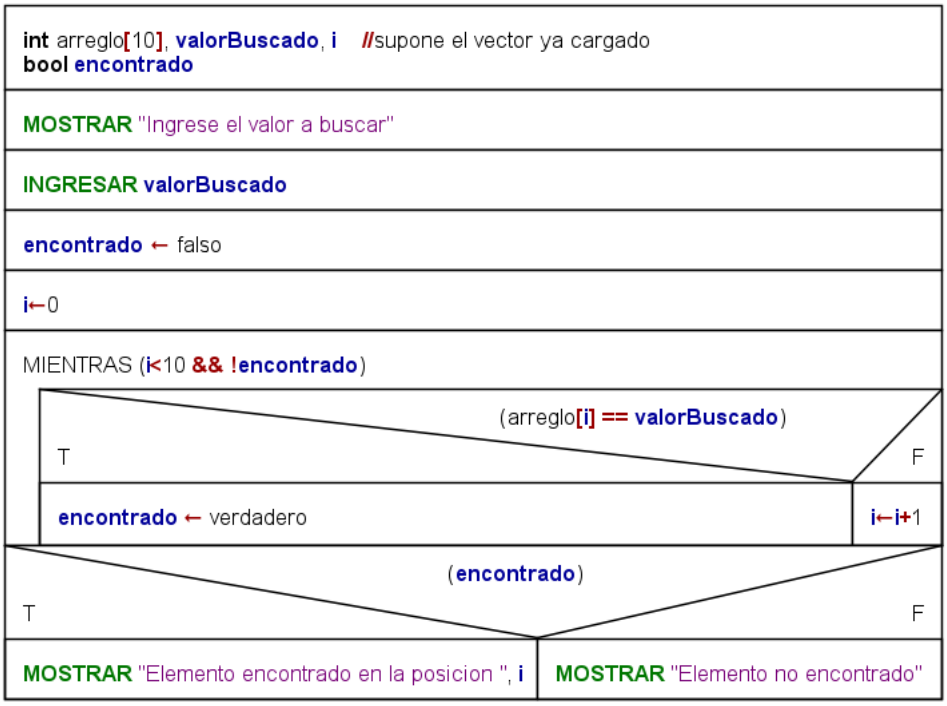
La búsqueda de elementos en estructuras de datos es una operación fundamental en programación. En este apunte abordaremos dos algoritmos clásicos aplicados a **arreglos unidimensionales (arrays)**: **la búsqueda secuencial** y **la búsqueda binaria o dicotómica**. Se analizarán su funcionamiento, eficiencia, condiciones de uso

Búsqueda secuencial (Lineal)

La búsqueda secuencial recorre el arreglo desde el inicio hasta el final, comparando uno por uno los elementos con el valor buscado. Su complejidad está directamente relacionada con la cantidad de elementos. El peor caso es cuando el elemento buscado está al final del arreglo o cuando no está.

No requiere que el arreglo esté ordenado y es conveniente para arreglos con pocos elementos.

Ejemplo en Chapin



Ejemplo en C++

```
int arreglo[10];

.....
bool encontrado;
int i;
int valorbuscado;

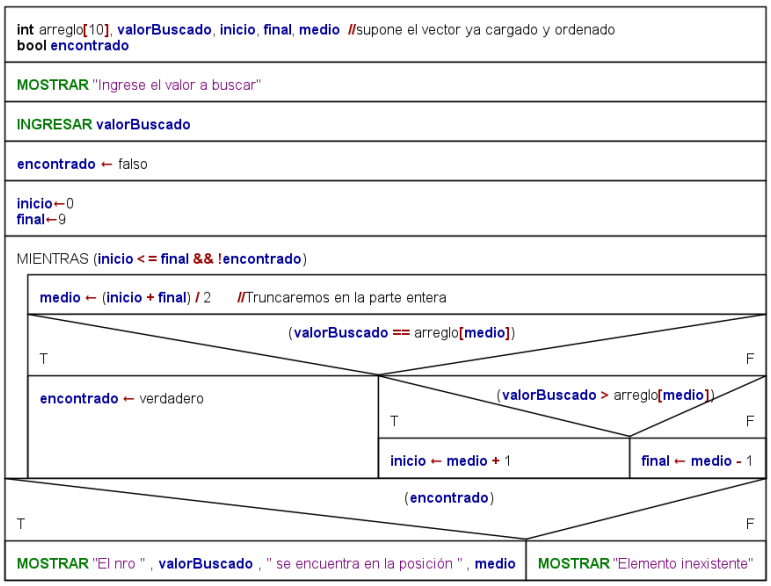
cout << "Ingrese el valor a buscar:" << endl;
cin >> valorbuscado;
encontrado = false;
i = 0;
while (i<10 && !encontrado) {
        if (arreglo[i]==valorbuscado) {
                encontrado = true;

        }
        else{
                i = i+1;
        }
}
if (encontrado) {
        cout << "Elemento encontrado en la posición " << i << endl;
} else {
        cout << "Elemento no encontrado" << endl;
}
```

Búsqueda binaria (Dicotómica)

La búsqueda binaria es un método eficiente que **divide el arreglo a la mitad en cada paso**, descartando una mitad del conjunto en cada iteración. El arreglo debe estar **ordenado de forma ascendente**.
Es muy eficiente para arreglos grandes y tiene rápida convergencia al valor buscado.

Ejemplo en Chapín



Ejemplo en C++

```
int arreglo[10];

....
bool encontrado;
int final;
int inicio;
int medio;
int nrobusc;
cin >> nrobusc;
encontrado = false;
inicio = 0;
final = 9;
while (!encontrado && inicio<=final) {
        medio = (inicio+final)/2;
        if ((nrobusc==arreglo[medio])) {
```

```
                encontrado = true;
        } else {
                if ((nrobusc>arreglo[medio])) {
                        inicio = medio+1;
                } else {
                        final = medio-1;
                }
        }
}
if (encontrado) {
        cout << "EL nro  " << nrobusc << " se encuentra en la posición " << medio << endl;
} else {
        cout << "Nro Inexistente" << endl;
}
```

EJEMPLO CON UN VALOR EXISTENTE

Sub	VALOR BUSCADO = 23		
0	1	1	1
1	3	3	3
2	4	4	4
3	6	6	6
4	7	7	7
5	9	9	9
6	11	11	11
7	15	15	15
8	16	16	16
9	19	19	19
10	23	23	23
11	33	33	33
12	35	35	35
13	45	45	45
14	56	56	56
15	58	58	58
16	71	71	71
17	79	79	79
inicio	0	9	9
fin	17	17	12
Medio	8	13	10

El elemento se encuentra en la posición 10 por lo que sale del MIENTRAS y muestra la posición.

EJEMPLO CON UN VALOR INEXISTENTE

Sub	VALOR BUSCADO = 25				
0	1	1	1	1	1
1	3	3	3	3	3
2	4	4	4	4	4
3	6	6	6	6	6
4	7	7	7	7	7
5	9	9	9	9	9
6	11	11	11	11	11
7	15	15	15	15	15
8	16	16	16	16	16
9	19	19	19	19	19
10	23	23	23	23	23
11	33	33	33	33	33
12	35	35	35	35	35
13	45	45	45	45	45
14	56	56	56	56	56
15	58	58	58	58	58
16	71	71	71	71	71
17	79	79	79	79	79
inicio	0	9	9	11	**11**
fin	17	17	12	12	**10**
Medio	8	13	10	11	

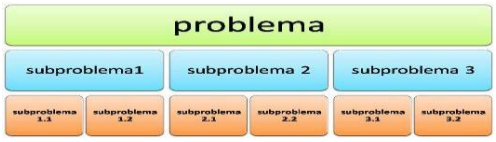
Sale de la estructura repetitiva por ser el inicio > fin.

CAPITULO 18 Funciones y procedimientos

Cuando un programa comienza a crecer, escribir todas las instrucciones dentro de un único bloque principal vuelve el algoritmo difícil de leer, corregir y mantener. Para evitar esto se utiliza la **programación modular**, que consiste en dividir un problema grande en partes más pequeñas, llamadas **módulos**, **subprogramas** o **subalgoritmos**.

Entre las distintas metodologías de resolución de problemas encontramos este enfoque que permite desarrollar algoritmos complejos de manera ordenada, comenzando por una visión global del problema y refinando progresivamente hasta obtener instrucciones simples y ejecutables

En algoritmia, los subalgoritmos más habituales son:



Procedimientos
Funciones

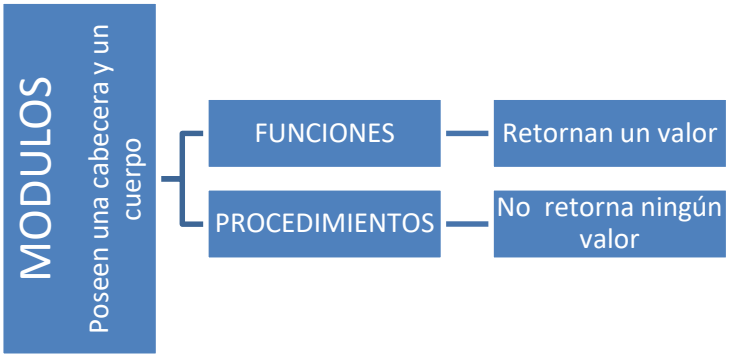
Ambos permiten agrupar instrucciones bajo un nombre, de manera que puedan ser llamadas desde otra parte del programa.

Un módulo o subalgoritmo es una unidad lógica de código e independiente de otros módulos, el cual es posible analizar, codificar y probar por separado.

CRITERIOS PARA PENSAR UN MÓDULO:

- Un módulo no debe ser demasiado extenso ni demasiado breve. La correcta organización y estructura del programa es un factor prioritario: no debemos agrupar contenidos inconexos ni dividir un proceso que se entiende mejor agrupado.
- Se debe buscar la independencia del módulo. En general es preferible que cada módulo realice un único proceso claramente identificable.
- Se debe buscar la facilidad de comprensión de la estructura.

Funciones y procedimientos



Un módulo que presenta las siguientes partes: Una cabecera y un cuerpo

La cabecera presenta las siguientes partes:

• El nombre del módulo

• Una lista de parámetros, que podría estar vacía

• El tipo de valor que devolverá (si corresponde).

Aunque ambos, funciones y procedimientos, son módulos o subalgoritmos, no cumplen exactamente el mismo rol.

Procedimiento

Un **procedimiento** es un subalgoritmo que ejecuta una acción, pero NO devuelve un valor directamente como resultado.

Estructura de un procedimiento en Chapin

```
procedimiento <nombreProcedimiento> ([<tipoarg2 arg1 [por Ref], tipoArg2 arg2 [por Ref], ...])
    ...
    ...
    Acciones
    ...
    ...
```

Estructura general de un procedimiento en C++

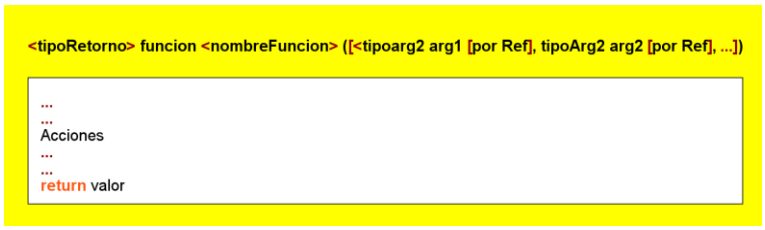
```
void nombreProcedimiento(listaDeParametros) {
  …
 instrucciones;
 ….

}
```

Función

Una **función** es un subalgoritmo que ejecuta una acción, pero SI devuelve un valor directamente como resultado. El programa que invoque a la función deber realizar alguna acción con dicho resultado.

Estructura de una función en Chapin



Ejemplo

int funcion sumar(int a, int b)

```
int suma ← a + b
return suma
```

Estructura general de una función en C++

```
tipoDeRetorno nombreFuncion(listaDeParametros) {
  …
 instrucciones;
 ….
  return valor;
}
```

Valor de retorno

Las funciones devuelven un resultado mediante la instrucción return.

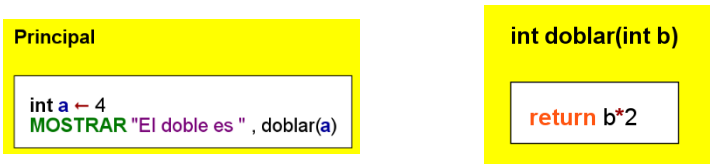
El tipo del valor devuelto debe ser compatible con el tipo declarado en la función. Por ejemplo, si la función dice que devuelve int, debería retornar un valor entero.

Parámetros y argumentos

Un subalgoritmo puede recibir datos desde el programa principal. Para eso se usan **parámetros**.

Hay que distinguir dos conceptos:

- Parámetros formales: son los que aparecen en la definición de la función o procedimiento.
- Argumentos o parámetros reales: son los valores que se envían al llamar a la función.



En este caso, el Parámetro Formal es **b** y el argumento al llamar la función es **a.**

Pasaje de parámetros

Cuando se llama a una función o procedimiento, los datos pueden pasarse de distintas maneras.

Las formas más importantes son:

- **Pasaje por valor**
- **Pasaje por referencia**

Pasaje por valor

Los valores contenidos en los parámetros actuales son automáticamente copiados en las variables formales en el momento en que la función se invoca, y de esta manera la función sólo trabaja con la copia. Por lo tanto, cualquier cambio en el parámetro formal no afectará el valor del parámetro actual o argumento.

Principal

```
int a←10
MOSTRAR "Antes de llamar al procedimiento. Valor de a: " , a
incrementar(a)
MOSTRAR "Después de llamar al procedimiento. Valor de a: " , a
```

Procedimiento incrementar(int a)

```
a← a+1
```

La salida sería:
Antes de llamar al procedimiento. Valor de a: 10
Después de llamar al procedimiento. Valor de a: 10

Pasaje por referencia

En el ejemplo siguiente, al usar el **por Ref** (o en C++ el &) en un parámetro formal x, lo que hace el lenguaje es introducir el identificador **x** como un sinónimo o segundo nombre del identificador que entra como parámetro actual (en nuestro ejemplo, **a**), lo que técnicamente significa que ambos identificadores están ubicados en el mismo lugar de la memoria (es decir que **x** y **a** hacen referencia a la misma variable). Por lo tanto, cualquier cambio en el valor del parámetro formal **x** afectará directamente al valor del parámetro actual **a**.

Principal

```
int a←10
MOSTRAR "Antes de llamar al procedimiento. Valor de a: " , a
incrementar(a)
MOSTRAR "Después de llamar al procedimiento. Valor de a: " , a
```

procedimiento incrementar(int x por Ref)

```
x ← x+1
```

Resumen del pasaje de parámetros

El pasaje por valor y el pasaje por referencia definen cómo se comunican los datos entre el programa principal y sus funciones o procedimientos.

En el **pasaje por valor**, el subalgoritmo recibe una copia del dato, por lo que cualquier modificación interna no afecta a la variable original.

En el **pasaje por referencia**, el subalgoritmo trabaja directamente sobre la variable original, por lo que los cambios realizados dentro de la función permanecen luego de la llamada.

Funciones que trabajan con arreglos

En C++, cuando se pasa un arreglo a una función, normalmente se pasa la dirección del primer elemento. Por eso, las modificaciones hechas sobre el arreglo dentro de la función afectan al arreglo original.

Ejemplo:

```
#include <iostream>
using namespace std;

void inicializarVector(int v[], int n) {
   for (int i = 0; i < n; i++) {
      v[i] = 0;
   }
}

int main() {
   int numeros[5];

   inicializarVector(numeros, 5);

   for (int i = 0; i < 5; i++) {
      cout << numeros[i] << " ";
   }

   return 0;
}
```

Salida:

0 0 0 0 0

El vector fue modificado dentro del procedimiento

Parámetros de entrada, salida y entrada/salida

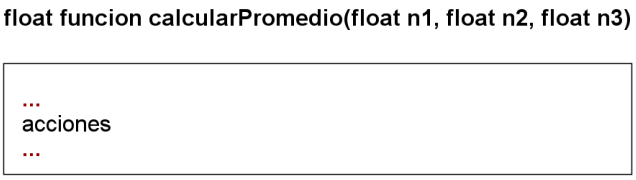
Desde el punto de vista algorítmico, los parámetros pueden clasificarse según su uso.

Parámetro de entrada

Sirve para enviar información al subalgoritmo.

El subalgoritmo lo usa, pero no necesita modificarlo.

Ejemplo:

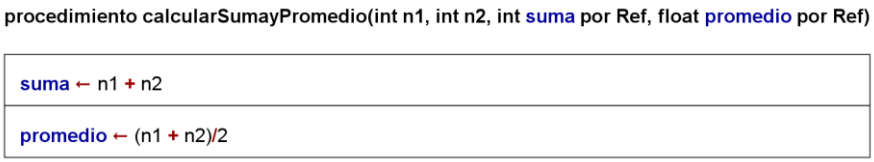


n1, n2 y n3 son parámetros de entrada.

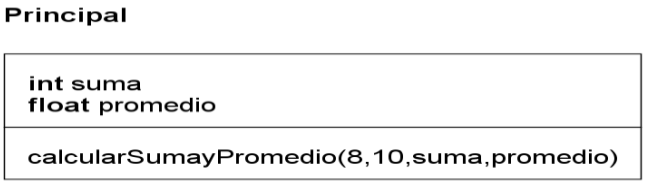
Parámetro de salida

Sirve para que el subalgoritmo devuelva información modificando una variable recibida por referencia.

Ejemplo:



Uso:



Después de la llamada:

suma = 18
promedio = 9

Parámetro de entrada/salida

Es un parámetro que entra con un valor inicial y sale modificado.

Ejemplo:

procedimiento aumentar(int numero por Ref)

```
numero ← numero + 1
```

Uso:

Principal

```
int edad ← 20
aumentar(edad)
```

Después:

edad = 21

Prototipos de funciones en C++

En C++, una función debe estar declarada antes de usarse.

Hay dos formas comunes.

Opción 1: definir la función antes del main

```
#include <iostream>
using namespace std;
int sumar(int a, int b) {
   return a + b;
}
int main() {
   cout << sumar(4, 5);
   return 0;
}
```

Opción 2: usar prototipo

El prototipo anuncia la función antes del main, aunque su desarrollo aparezca después.

```
#include <iostream>
using namespace std;

int sumar(int a, int b);

int main() {
   cout << sumar(4, 5);
   return 0;
}

int sumar(int a, int b) {
   return a + b;
}
```

El prototipo es:

```
int sumar(int a, int b);
```

Indica:

- nombre de la función
- tipo de retorno
- cantidad y tipo de parámetros

Ámbitos De Una Variable

VARIABLES GLOBALES

El uso de este tipo de variables suele considerarse como una mala práctica. En un programa largo, complejo una variable global es muy difícil de controlar.

VARIABLES LOCALES

Se declaran dentro de un módulo, solo pueden usarse dentro del módulo y se destruyen al salir del módulo que las declaró, por lo que no es accesible desde fuera de dicho módulo.

Ámbito	Dónde se declara	Vida útil	Visibilidad
Global	Fuera de **todas** las funciones o procedimientos	Desde el inicio hasta el fin del programa	Todas las partes del programa (incluyendo subprogramas), salvo que exista variable local con el mismo nombre (sombreado).
Local	Dentro de una función o procedimiento	Sólo mientras ese subprograma se ejecuta	Sólo dentro de ese subprograma.
Parámetro	En la cabecera de una función o procedimiento	Igual que la ejecución del subprograma	Sólo dentro del subprograma; actúa como variable local.

CAPITULO 19 Recursividad



Una función es recursiva cuando “se define en función de si misma.

Una función es **recursiva** cuando, para resolver un problema, se invoca a sí mismo una y otra vez hasta que el problema queda resuelto

El concepto de recursividad se utiliza con mucha frecuencia en el campo de las matemáticas.

Ejemplo.

Los números naturales se definen de la siguiente forma:

(1) 1 es un número natural, y

(2) el siguiente de un número natural es también un número natural.

Vemos cómo esta definición es, efectivamente, recursiva, ya que para definir lo que es un número natural estamos usando, de hecho, el concepto de número natural.

Es importante observar como en el ejemplo anterior la definición recursiva presenta dos partes claramente diferenciadas. Por un lado, tenemos un caso trivial o elemental del cálculo, como por ejemplo el 1 es un nro. natural.

Después tenemos la parte propiamente recursiva de la definición. Es importante ver como en esta segunda parte se define el problema inicial, en función de una versión más “sencilla' del mismo problema.

CASO BASE

Para que la RECURSIVIDAD no sea infinita, es necesario comprender el concepto de caso base.

El caso base (o condición de corte) es la situación más simple del problema, aquella que no necesita más llamadas recursivas para resolverse. Es el punto en que la función deja de llamarse a sí misma y comienza a “retornar” los resultados hacia arriba en la pila de llamadas.

CARACTERÍSTICAS PRINCIPALES PARA FUNCIONES RECURSIVAS:

Caso base (condición de corte):

Es la condición que detiene la recursión. Sin esto, la función se llamaría infinitamente y causaría un desbordamiento de la memoria.

Llamada recursiva:

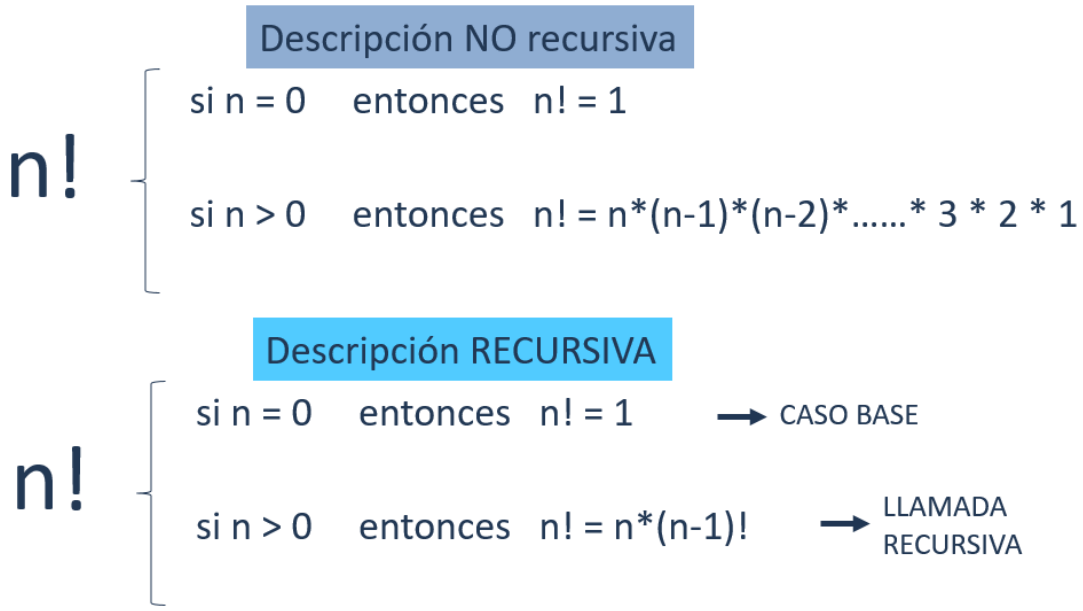
Es cuando la función se llama a sí misma con un argumento más simple o reducido, acercándose al caso base.

ANALISIS DE EJEMPLO: factorial

En las Matemáticas, se usa con frecuencia, debido a que existen problemas cuya descripción es más coherente con recursividad. Típico es el caso del factorial de un número n.

DEFINICION DE FACTORIAL

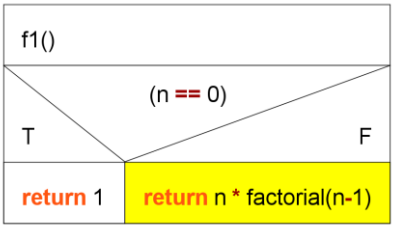
Si n es un entero positivo o cero, entonces el factorial de n (denotado como n!), se puede definir:

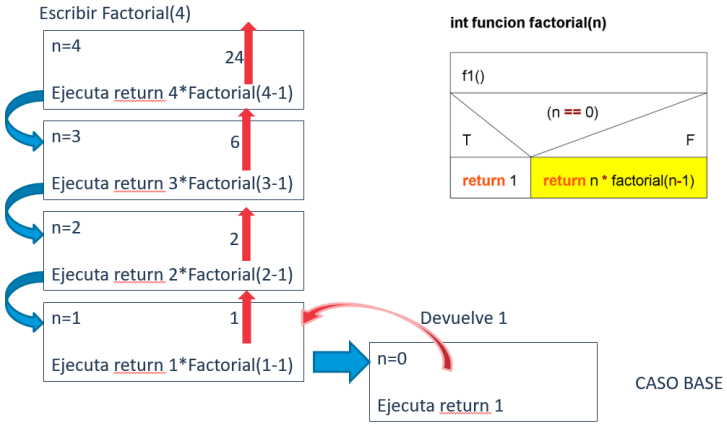


Principal

```
MOSTRAR factorial(4)
```

int funcion factorial(n)





CAPITULO 20 Registros o Struct

Hasta este momento, los algoritmos trabajaron principalmente con variables simples, vectores y matrices. Estas estructuras permiten almacenar datos, pero presentan una limitación cuando se desea representar entidades reales compuestas por varios atributos.
Por ejemplo, un alumno no se describe con un único dato. Para representarlo correctamente necesitamos guardar información como nombre, legajo, notas, fecha de nacimiento, entre otros datos.
Si cada dato se almacena en una variable independiente, el algoritmo se vuelve más difícil de leer, mantener y ampliar. Para resolver este problema aparecen los registros.
Un registro permite agrupar varios datos relacionados bajo un mismo nombre, formando un nuevo tipo de dato definido por el programador.

Introducción a los registros

¿Qué problema resuelven los registros?
Supongamos que queremos representar un alumno.
Una primera solución podría ser declarar variables separadas:

string nombre
int legajo
float nota

Esta solución funciona para un solo alumno, pero presenta dificultades cuando el problema crece.
Por ejemplo, si necesitamos trabajar con varios alumnos, rápidamente aparecen muchas variables o varios vectores separados:

string nombres[35]
int legajos[35]
float notas[35]

Aunque esto permite almacenar los datos, la información de cada alumno queda dispersa. El nombre, el legajo y la nota pertenecen a la misma entidad, pero están almacenados en estructuras separadas. El registro permite resolver este problema agrupando todos los datos de una misma entidad en una única estructura.

Definición de registro

Un registro es una estructura de datos compuesta que permite agrupar bajo un único nombre varios datos relacionados entre sí, aunque sean de distintos tipos.
Cada dato que forma parte del registro recibe el nombre de campo.
Un registro representa una entidad del problema.
Por ejemplo:

Registro Alumno {
string nombre,
int legajo,
float nota
}

En este caso, Alumno es un nuevo tipo de dato definido por el programador.
Sus campos son:
nombre, de tipo string.
legajo, de tipo int.
nota, de tipo float.
La definición del registro no almacena datos todavía. Solamente describe qué forma tendrán las variables que luego se declaren con ese tipo.

Convención utilizada en la asignatura

En esta materia trabajaremos con Diagramas de Chapin y con C++. Por eso utilizaremos una notación cercana a C++ para definir registros.
Usaremos tipos como:

- int
- float
- string
- char
- bool

La forma general será:

Registro NombreRegistro {
tipoDato nombreCampo,
tipoDato nombreCampo,
tipoDato nombreCampo
}

Ejemplo:

Registro Producto {
int codigo,
string descripcion,
float precio,

int stock
}

Esta notación facilita la posterior traducción a C++.
En Chapin escribiremos:

Registro Alumno {
string nombre,
int legajo,
float nota
}

En C++ se implementa como:

struct Alumno {
string nombre;
int legajo;
float nota;
};

La diferencia principal es que en C++ cada campo finaliza con punto y coma.

Registro como modelo de una entidad

Un registro debe pensarse como la representación de una entidad del problema.
Por ejemplo, si el problema habla de alumnos, productos, fechas o empleados, cada una de esas entidades puede modelarse mediante un registro.

Ejemplo:

Registro Fecha {
int dia,
int mes,
int anio
}

Ejemplo:

Registro Producto {
int codigo,
string descripcion,
float precio,
int stock
}

Ejemplo:

Registro Alumno {
string nombre,
int legajo,
float nota1,
float nota2,
float nota3
}

La clave está en identificar qué datos pertenecen a la misma entidad lógica.

Declaración de una variable de tipo registro

Una vez definido un registro, se puede declarar una variable de ese tipo.
Ejemplo:

Alumno a

Esto significa que **a** es una variable de tipo Alumno.

Internamente, esa variable contiene todos los campos definidos en el registro:

a.nombre
a.legajo
a.nota

Es importante diferenciar dos momentos:

Primero se define el tipo:

Registro Alumno { … }

Luego se declara una variable de ese tipo:

Alumno a

La definición del registro describe la estructura. La declaración de la variable reserva el espacio para almacenar datos.

Acceso a los campos

Para acceder a un campo de una variable de tipo registro se utiliza el operador punto.

Ejemplo:

a.nombre
a.legajo
a.nota

Cada campo puede utilizarse como una variable común.
Por ejemplo:

a.nombre ← "Juan"
a.legajo ← 1234
a.nota ← 8.5

También se puede ingresar o mostrar cada campo:

Ingresar a.nombre
Ingresar a.legajo
Ingresar a.nota
Mostrar a.nombre
Mostrar a.legajo
Mostrar a.nota

El operador punto indica que se está accediendo a un dato interno del registro.

Vectores de registros

Hasta el momento trabajamos con una variable de tipo registro. Eso permite representar una sola entidad.
Por ejemplo:

Alumno a

Pero en la mayoría de los problemas necesitamos representar muchas entidades del mismo tipo.

Por ejemplo:

- Una comisión con 35 alumnos.
- Un comercio con 100 productos.
- Una empresa con varios empleados.

Para estos casos se utiliza un vector de registros.
Un vector de registros permite almacenar varios registros del mismo tipo, uno en cada posición del vector.

Definición de un vector de registros

Supongamos que tenemos el siguiente registro:

Registro Alumno {
string nombre,
int legajo,
float promedio
}

Para almacenar 35 alumnos podemos declarar:

Alumno alumnos[35]

Esto significa que alumnos es un vector de 35 posiciones y cada posición contiene un registro completo de tipo Alumno.
Cada elemento del vector tiene sus propios campos:

alumnos[0].nombre
alumnos[0].legajo
alumnos[0].promedio
alumnos[1].nombre
alumnos[1].legajo
alumnos[1].promedio

ACCESO A CAMPOS EN UN VECTOR DE REGISTROS

Para acceder a un campo dentro de un vector de registros se combinan dos mecanismos:
Primero, el índice del vector.
Luego, el operador punto.
Ejemplo:

alumnos[i].nombre

Esto significa:
acceder al alumno ubicado en la posición i y, dentro de ese alumno, acceder al campo nombre.
Esta notación es fundamental porque permite recorrer el vector y trabajar con los campos de cada registro.

CARGA DE UN VECTOR DE REGISTROS

Para cargar o mostrar un vector de registros se utiliza una estructura repetitiva.
Ejemplo:



En cada vuelta del ciclo se carga un alumno completo.
Mostrar un vector de registros implica recorrerlo y mostrar los campos de cada elemento.

FUNCIONES Y PROCEDIMIENTOS CON REGISTROS

Un registro puede enviarse a una función o procedimiento de la misma manera que se envía un int, un float o un string.
Supongamos:

Registro Alumno {
string nombre,
int legajo,
float nota1,
float nota2,
float nota3
}

Si declaramos:

Alumno a

Podemos enviar a como parámetro:

mostrarAlumno(a)
calcularPromedio(a)

Funciones que modifican registros

Cuando un procedimiento o función debe modificar un registro, debe recibirlo por referencia.
En esta asignatura utilizaremos el símbolo **por Ref** y en C++ se usa &.
Ejemplo:

*procedimiento cargarAlumno(Alumno a **por Ref**)*

El **por Ref** indica que el procedimiento trabajará sobre la variable original.

*procedimiento cargarAlumno(Alumno a **por Ref**)*
Ingresar a.nombre
Ingresar a.legajo
Ingresar a.nota1
Ingresar a.nota2
Ingresar a.nota3

Invocación:

cargarAlumno(a)

Como a fue pasado por referencia, los datos ingresados dentro del procedimiento quedan almacenados en la variable original.
Si no se usara **por Ref**, el procedimiento trabajaría con una copia y los cambios podrían no reflejarse fuera del subalgoritmo.

Funciones sobre vectores de registros

También es posible enviar un vector de registros como parámetro.
Esto resulta útil cuando una operación necesita recorrer varios registros.
Por ejemplo, buscar un alumno dentro de un vector.
Firma:

int función buscarAlumno(Alumno alumnos[], int n, int legajoBuscado)

REGISTROS ANIDADOS

En algunos problemas, una entidad contiene dentro de sí otra entidad.
Por ejemplo, un alumno puede tener una fecha de nacimiento.
La fecha también está compuesta por varios datos:

- día
- mes
- año

Por eso puede definirse como otro registro.

```
Registro Fecha {
    int dia,
    int mes,
    int anio
}
```

Luego puede utilizarse como campo dentro de Alumno:

```
Registro Alumno {
    string nombre,
    int legajo,
    Fecha nacimiento,
    float nota1,
    float nota2,
    float nota3
}
```

En este caso, nacimiento es un campo de tipo Fecha.

ACCESO A CAMPOS ANIDADOS
Cuando un registro contiene otro registro, se accede mediante el operador punto más de una vez.
Ejemplo:

```
alumno.nacimiento.dia
alumno.nacimiento.mes
alumno.nacimiento.anio
```

Primero se accede al campo nacimiento del alumno.
Luego se accede al campo interno de la fecha.
En C++ se escribe de la misma manera.

CAPITULO 21 Archivos

Un archivo es un conjunto de datos almacenados en un dispositivo de almacenamiento, como un disco rígido, pendrive, memoria del celular o unidad externa.

A diferencia de una variable, un vector o una matriz, un archivo permite que la información permanezca guardada incluso después de que el programa termina.

Por ejemplo, si un programa carga los datos de alumnos en un vector, esos datos existen solo mientras el programa está funcionando. Pero si los guarda en un archivo, pueden recuperarse más tarde.
Un archivo es un conjunto de datos que esencialmente se define para ser almacenado en memoria externa (es decir, en discos, cintas, etc.), y por lo tanto el contenido de un archivo no se destruye al finalizar el programa que lo utiliza.
El uso de archivos permite el manejo de grandes volúmenes de datos sin tener que volverlos a cargar desde teclado cada vez que se desee procesarlos. Por otra parte, una vez que un archivo fue creado y grabado en un disco, ese archivo puede ser utilizado por cualquier otro programa, ya sea para obtener datos del archivo y/o para modificar los datos del mismo. En otras palabras: no sólo el archivo puede ser usado por el programa que lo creó, sino también por cualquier otro programa que realice las definiciones adecuadas.
También en un archivo los distintos componentes se ubican uno a continuación del otro, y al igual que en los arreglos, se puede considerar que los componentes de un archivo en C / C++ también son automáticamente numerados con índices, desde el 0 (cero) en adelante (si bien en un archivo estos números realmente no existen, veremos que el programador puede trabajar con ellos como si los tuviera, para facilitar la tarea de acceder a un componente individual). No obstante, como se verá, la sintaxis para manejar estos índices es diferente a la de un arreglo.
Por otra parte, los archivos se clasifican en dos tipos, de acuerdo a su contenido:

- Archivos de texto: poseen texto "puro" (incluyendo caracteres de salto de línea, tabuladores, etc.). Son esta clase, por ejemplo, los archivos fuente de cada programa .cpp.
  Estos archivos, no sólo pueden ser manejados desde un programa en C, sino que pueden ser abiertos con cualquier editor de textos que conozca.
- Archivos binarios: Contienen secuencias o bloques de valores binarios, que representan información grabada por el usuario. El lenguaje no distingue si los bytes grabados pertenecen a un número int o a un float o a una cadena. Es el programador el que deteremina cómo interpretar el contenido. Si bien es cierto que en un archivo binario pueden guardarse elementos del tipo que se desee, es común encontrarse con archivos que poseen registros, debido a que el registro es una estructura de datos que permite describir muy bien a cualquier entidad de la cual se quiera almacenar datos.

Archivos en C++

En C++, para trabajar con archivos se utiliza la biblioteca:

```
#include <fstream>
```

La palabra fstream viene de file stream, es decir, flujo de archivo.

Los tipos más usados son:

```
ifstream   // para leer archivos
ofstream   // para escribir archivos
```

fstream // para leer y escribir

Escritura en archivos de texto

Para escribir en un archivo usamos **`ofstream`**.
Ejemplo:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
   ofstream archivo;
   archivo.open("datos.txt");
   archivo << "Hola mundo" << endl;
   archivo << "Este texto se guarda en un archivo" << endl;
   archivo.close();
   return 0;
}
```

Este programa crea un archivo llamado:
datos.txt
y escribe dentro:
Hola mundo
Este texto se guarda en un archivo

Apertura, escritura y cierre de un archivo

Analicemos las instrucciones principales del ejemplo anterior.

```
ofstream archivo;
```

Esta instrucción declara un **objeto llamado archivo de la clase ofstream**.

La clase ofstream permite trabajar con archivos de salida, es decir, archivos sobre los cuales el programa va a **escribir información**.

El objeto archivo será utilizado por el programa para realizar las distintas operaciones sobre el archivo.

Es importante distinguir entre:
- archivo: es el **objeto que existe dentro del programa**.
- "datos.txt": es el **archivo que se encuentra almacenado en el disco**.

Luego aparece:

```
archivo.open("datos.txt");
```

El método open() permite **abrir un archivo y asociarlo al objeto** que hemos declarado.
Si el archivo no existe, será creado.
Si el archivo ya existe, al abrirlo normalmente mediante ofstream, su contenido anterior será eliminado y el archivo quedará preparado para recibir nueva información.

En este caso, el objeto archivo queda asociado al archivo físico llamado: datos.txt

También es posible declarar el objeto y abrir el archivo al mismo tiempo:

```
ofstream archivo("datos.txt");
```

A partir de ese momento, las operaciones realizadas mediante el objeto archivo se efectuarán sobre datos.txt.
Por ejemplo:

```
archivo << "Hola mundo" << endl;
```

El operador << funciona de manera similar a cuando utilizamos cout.
Con: cout << "Hola mundo"; la información se envía a la pantalla.
En cambio, con: archivo << "Hola mundo"; la información se envía al archivo.

Finalmente encontramos:

```
archivo.close();
```

El método close() permite **cerrar el archivo**.
Cuando el programa termina de utilizar un archivo es conveniente cerrarlo. De esta manera se finalizan correctamente las operaciones pendientes y se libera el recurso utilizado para trabajar con él.

Verificación de la apertura de un archivo

Cuando un programa intenta abrir un archivo, la operación puede fallar por diferentes motivos. Por ejemplo, puede ocurrir que la ubicación indicada no exista, que el programa no tenga permisos para acceder al archivo o que se haya especificado incorrectamente su nombre.

Por este motivo, es conveniente verificar que el archivo se haya abierto correctamente antes de comenzar a utilizarlo.

Para ello se puede utilizar el método:

```
is_open()
```

Este método devuelve verdadero si el archivo se encuentra abierto correctamente.

Por ejemplo:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
   ofstream archivo;

   archivo.open("datos.txt");

   if (archivo.is_open()) {
      archivo << "Hola mundo" << endl;
      archivo.close();
   }
   else {
      cout << "No se pudo abrir el archivo" << endl;
```

```
    }

    return 0;
}
```

En este caso, solamente se intenta escribir información si el archivo pudo abrirse correctamente.

Lectura de archivos de texto

Para leer información desde un archivo de texto se utiliza `ifstream`.

La palabra `ifstream` proviene de: input file stream es decir, flujo de entrada desde un archivo.

Por ejemplo:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream archivo;
    string nombre;

    archivo.open("datos.txt");

    if (archivo.is_open()) {
        archivo >> nombre;
        cout << nombre << endl;

        archivo.close();
    }
    else {
        cout << "No se pudo abrir el archivo" << endl;
    }

    return 0;
}
```

La instrucción:

```
archivo >> nombre;
```

lee información desde el archivo y la almacena en la variable `nombre`.

```
cout << nombre;
```

muestra en pantalla el dato leído.

Lectura de varios datos

Un archivo puede contener varios datos.

Por ejemplo, supongamos que `datos.txt` contiene:

Juan 20 8.5

El programa podría leerlos de la siguiente manera:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
   ifstream archivo;

   string nombre;
   int edad;
   float nota;

   archivo.open("datos.txt");

   if (archivo.is_open()) {
      archivo >> nombre;
      archivo >> edad;
      archivo >> nota;

      cout << "Nombre: " << nombre << endl;
      cout << "Edad: " << edad << endl;
      cout << "Nota: " << nota << endl;

      archivo.close();
   }
   else {
      cout << "No se pudo abrir el archivo" << endl;
   }

   return 0;
}
```

También podrían realizarse las tres lecturas en una misma instrucción:

```
archivo >> nombre >> edad >> nota;
```

El archivo posee una posición de lectura

Cuando se realiza una lectura, el programa no vuelve a comenzar desde el principio del archivo.

El archivo mantiene internamente una **posición de lectura**, que indica desde qué lugar se realizará la próxima lectura.

Supongamos que el archivo contiene:

10 20 30

y ejecutamos:

```
int nro;
archivo >> nro;
```

La variable `nro` recibe: 10

Si inmediatamente volvemos a ejecutar:

```
archivo >> nro;
```

ahora recibe: 20

Esto sucede porque, después de cada lectura, la posición del archivo avanza.

Esta característica permite recorrer un archivo de manera **secuencial**, leyendo sus datos uno después del otro.

Fin de archivo

Cuando se trabaja con archivos, generalmente no se conoce de antemano cuántos datos contiene el archivo.

Por ejemplo, un archivo podría contener cinco números, cien números o varios miles.

Por lo tanto, necesitamos alguna manera de detectar cuándo ya no quedan más datos para leer.

A esta situación se la denomina **fin de archivo**, o en inglés:

End Of File (EOF)

En C++ una forma muy utilizada de recorrer un archivo consiste en utilizar directamente la operación de lectura como condición.

Por ejemplo:

```
int nro;

while (archivo >> nro) {
   cout << nro << endl;
}
```

La expresión:

archivo >> nro

intenta leer un dato.

Mientras la lectura pueda realizarse correctamente, la condición se considera verdadera y el ciclo continúa.

Cuando ya no quedan datos para leer, la lectura falla y el ciclo termina.

Por ejemplo, si `datos.txt` contiene:

10
20
30
40

podemos escribir:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
   ifstream archivo;
   int nro;

   archivo.open("datos.txt");

   if (archivo.is_open()) {

      while (archivo >> nro) {
         cout << nro << endl;
      }

      archivo.close();
   }
   else {
      cout << "No se pudo abrir el archivo" << endl;
   }

   return 0;
}
```

El resultado será:

10
20
30
40

La lectura se realiza secuencialmente hasta que no existen más números en el archivo.

Esta forma:

```
while (archivo >> nro)
```

es especialmente importante, ya que evita tener que conocer previamente la cantidad de datos almacenados en el archivo.

Lectura de líneas completas con getline()

Cuando se utiliza el operador >> para leer una cadena de caracteres, la lectura se detiene al encontrar un espacio. Por ejemplo, supongamos que el archivo datos.txt contiene:

Juan Perez

Si escribimos:

```
string nombre;
archivo >> nombre;
```

la variable nombre contendrá solamente:
Juan
Esto sucede porque el operador >> considera al espacio como un separador entre datos.
Si se desea leer una línea completa, incluyendo los espacios, se puede utilizar la función:

```
getline()
```

Por ejemplo:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
   ifstream archivo;
   string nombre;
   archivo.open("datos.txt");
   if (archivo.is_open()) {

      getline(archivo, nombre);

      cout << nombre << endl;
      archivo.close();
   }
   else {
      cout << "No se pudo abrir el archivo" << endl;
   }
   return 0;
}
```

Si el archivo contiene: Juan Perez, la instrucción:

```
getline(archivo, nombre);
```

guardará en nombre:
Juan Perez

La función getline() lee desde la posición actual del archivo hasta encontrar un salto de línea.

También puede utilizarse para leer varias líneas.
Por ejemplo:

```
string linea;
while (getline(archivo, linea)) {
   cout << linea << endl;
}
```

En este caso, se leerá una línea completa en cada repetición del ciclo.
Si el archivo contiene:
David Marioni.
Pablo Frattini.
Nicolas Espinosa.
el ciclo realizará tres lecturas.

Agregar información a un archivo sin borrar su contenido

Como vimos anteriormente, cuando un archivo se abre normalmente con ofstream, si el archivo ya existe, su contenido anterior es eliminado.

Por ejemplo:

```
ofstream archivo;
archivo.open("datos.txt");
```

Si datos.txt ya contenía información, esta será reemplazada por la nueva información que se escriba.
En muchas situaciones, sin embargo, se desea conservar el contenido existente y agregar nuevos datos al final del archivo.
Para ello se utiliza el modo:

```
ios::app
```

La palabra app proviene de **append**, que significa agregar o anexar.
Por ejemplo:

```
ofstream archivo;

archivo.open("datos.txt", ios::app);
```

Supongamos que el archivo contiene:

Juan
Maria

y ejecutamos:

```
archivo << "Pedro" << endl;
```

Luego de cerrar el archivo, su contenido será:
Juan
Maria
Pedro

Los datos anteriores no fueron eliminados.

Un ejemplo completo sería:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
   ofstream archivo;
```

```
    archivo.open("datos.txt", ios::app);

    if (archivo.is_open()) {

        archivo << "Pedro" << endl;

        archivo.close();
    }
    else {
        cout << "No se pudo abrir el archivo" << endl;
    }

    return 0;
}
```

El modo ios::app hace que toda nueva escritura se realice al final del archivo.

Modos de apertura de un archivo

Cuando se abre un archivo se puede indicar de qué manera se desea trabajar con él.
Para ello existen diferentes modos de apertura.
Algunos de los más utilizados son:

- ios::in
  Permite abrir un archivo para lectura.
- ios::out
  Permite abrir un archivo para escritura.
- ios::app
  Permite escribir información al final del archivo, conservando el contenido anterior.
- ios::binary
  Permite trabajar con el archivo en modo binario.

  También existe:
- ios::trunc
  que indica que, si el archivo ya existe, su contenido debe ser eliminado al abrirlo.

Por ejemplo:

```
ofstream archivo;
archivo.open("datos.txt", ios::out);
```

abre el archivo para escritura. Mientras que:

```
ifstream archivo;
archivo.open("datos.txt", ios::in);
```

lo abre para lectura.

Sin embargo, en muchos casos no es necesario indicar explícitamente estos modos.

Por ejemplo:

```
ifstream archivo("datos.txt");
```

ya implica que el archivo será abierto para lectura.
Del mismo modo:

```
ofstream archivo("datos.txt");
```

lo abre para escritura.

Combinación de modos de apertura

También es posible combinar varios modos utilizando el operador: |
Por ejemplo:

```
fstream archivo;
archivo.open("datos.txt", ios::in | ios::out);
```

En este caso, el archivo se abre tanto para lectura como para escritura.
Otro ejemplo sería:

```
archivo.open("datos.dat", ios::in | ios::binary);
```

que abre un archivo para lectura en modo binario.
La combinación de modos será especialmente importante cuando estudiemos archivos binarios.

Uso de fstream

Hasta el momento vimos dos tipos principales de objetos para trabajar con archivos: **ifstream** para lectura, y **ofstream** para escritura.
Existe también **fstream** que permite realizar tanto operaciones de lectura como de escritura.

Por ejemplo:

```
fstream archivo;
archivo.open("datos.txt", ios::in | ios::out);
```

UNA CONSIDERACIÓN IMPORTANTE SOBRE EOF()

En C++ existe el método eof(). La sigla EOF significa **End Of File** es decir, fin de archivo.
Por este motivo puede parecer lógico escribir:

```
while (!archivo.eof()) {
   archivo >> dato;
   cout << dato << endl;
}
```

Sin embargo, esta forma de recorrer un archivo puede producir errores.
El motivo es que eof() no se activa simplemente cuando se llega al último dato del archivo.
Se activa cuando el programa **intenta realizar una lectura y ya no existen más datos disponibles**.
Por ejemplo, supongamos que el archivo contiene:
10
20
30
Cuando se lee el 30, la lectura fue correcta. Por lo tanto, todavía no se produjo un error de fin de archivo.
Recién cuando el programa intenta leer un dato después del 30, la lectura falla y se detecta el fin del archivo.

Archivos de registros

Hasta ahora trabajamos con archivos que almacenaban datos simples, como números, nombres o líneas de texto.

Sin embargo, muchas veces necesitamos guardar información formada por varios datos relacionados entre sí.

Por ejemplo, para representar a un alumno podríamos necesitar almacenar:

* Legajo
* Nombre
* Edad
* Nota

Estos datos pueden agruparse en una estructura o `struct`.

Por ejemplo:

```
struct Alumno {
   int legajo;
   string nombre;
   int edad;
   float nota;
};
```

Cada variable del tipo `Alumno` representa un alumno completo.

Por ejemplo:

```
Alumno a;

a.legajo = 125;
a.nombre = "Juan Perez";
a.edad = 20;
a.nota = 8.5;
```

En este caso, `a` es un **registro** que contiene varios campos.

Registros almacenados en un archivo de texto

También es posible almacenar los campos de un registro utilizando un archivo de texto.

Por ejemplo, podríamos guardar:

```
125 Juan 20 8.5
230 María 22 7
415 Pedro 19 9
```

Cada línea representa un alumno.

Un programa podría escribir estos datos de la siguiente manera:

```
ofstream archivo;
```

```
archivo.open("alumnos.txt");

archivo << 125 << " " << "Juan" << " " << 20 << " " << 8.5 << endl;
archivo << 230 << " " << "Maria" << " " << 22 << " " << 7 << endl;
archivo << 415 << " " << "Pedro" << " " << 19 << " " << 9 << endl;

archivo.close();
```

También podríamos utilizar una variable de tipo `Alumno`.

Por ejemplo:

```
struct Alumno {
   int legajo;
   string nombre;
   int edad;
   float nota;
};

Alumno a;

a.legajo = 125;
a.nombre = "Juan";
a.edad = 20;
a.nota = 8.5;

archivo << a.legajo << " "
      << a.nombre << " "
      << a.edad << " "
      << a.nota << endl;
```

En este caso, aunque utilizamos un `struct` dentro del programa, los campos son grabados en el archivo como **texto**.

El contenido del archivo seguirá siendo legible con cualquier editor de textos.

Lectura de registros desde un archivo de texto

Supongamos que el archivo `alumnos.txt` contiene:

125 Juan 20 8.5
230 María 22 7
415 Pedro 19 9

Podríamos leer todos los registros de la siguiente manera:

```
#include <iostream>
#include <fstream>
using namespace std;

struct Alumno {
```

```
    int legajo;
    string nombre;
    int edad;
    float nota;
};

int main() {

    ifstream archivo;
    Alumno a;

    archivo.open("alumnos.txt");

    if (archivo.is_open()) {

        while (archivo >> a.legajo >> a.nombre >> a.edad >> a.nota) {

            cout << "Legajo: " << a.legajo << endl;
            cout << "Nombre: " << a.nombre << endl;
            cout << "Edad: " << a.edad << endl;
            cout << "Nota: " << a.nota << endl;
            cout << endl;
        }

        archivo.close();
    }
    else {
        cout << "No se pudo abrir el archivo" << endl;
    }

    return 0;
}
```

En cada repetición del ciclo:

```
while (archivo >> a.legajo >> a.nombre >> a.edad >> a.nota)
```

se intenta leer un registro completo.

Si todos los campos pudieron ser leídos correctamente, el ciclo continúa.

Archivos binarios

Además de los archivos de texto, C++ permite trabajar con **archivos binarios**.

En un archivo de texto, los valores son almacenados utilizando caracteres.

Por ejemplo, el número 125 se guarda como los caracteres '1' '2' '5'

En cambio, en un archivo binario, el valor se almacena utilizando directamente su representación interna en memoria.

Por esta razón, si abrimos un archivo binario con un editor de textos, normalmente veremos símbolos o caracteres sin sentido.

Esto no significa que el archivo esté dañado. Simplemente los datos no fueron almacenados como texto.

Diferencia entre archivo de texto y archivo binario

Supongamos que queremos guardar:
int edad = 25;

En un archivo de texto podríamos escribir:

archivo << edad;

El archivo contendría: 25 es decir, los caracteres '2' y '5'.

En cambio, en un archivo binario se almacenan directamente los bytes que utiliza la variable `edad` en memoria.

Por lo tanto, los archivos binarios permiten guardar datos de una forma más parecida a como se encuentran representados internamente dentro del programa.

Apertura de un archivo binario

Para indicar que un archivo será utilizado en modo binario se agrega:

ios::binary

Por ejemplo, para escritura:

```
ofstream archivo;

archivo.open("alumnos.dat", ios::binary);
```

Y para lectura:

```
ifstream archivo;

archivo.open("alumnos.dat", ios::binary);
```

La extensión `.dat` es utilizada frecuentemente para archivos de datos, aunque la extensión por sí sola no determina si el archivo es de texto o binario.

Lo que determina el tratamiento es la forma en que el programa abre y procesa el archivo.

Escritura en un archivo binario

Para escribir datos binarios no se utiliza normalmente el operador << sino el método: **write()**.

Por ejemplo:

int numero = 25;

archivo.write(reinterpret_cast<char*>(&numero), sizeof(numero));

Esta instrucción puede parecer inicialmente más compleja que la escritura de un archivo de texto.

Por ahora debemos interpretar que:

write()

permite copiar al archivo una determinada cantidad de bytes que se encuentran en memoria.

La expresión:

sizeof(numero)

indica cuántos bytes ocupa la variable `numero`.

Por ejemplo, si un `int` ocupa 4 bytes, se escribirán esos 4 bytes en el archivo.

Lectura desde un archivo binario

Para leer información desde un archivo binario se utiliza: **read()**

Por ejemplo:

int numero;

archivo.read(reinterpret_cast<char>(&numero), sizeof(numero));*

El método `read()` toma los bytes almacenados en el archivo y los copia nuevamente en la variable `numero`.

Por lo tanto:

write() → escribe bytes en el archivo
read() → lee bytes desde el archivo

Escritura de un registro completo

Una de las principales ventajas de los archivos binarios es que permiten grabar un registro completo mediante una sola operación.
Supongamos la siguiente estructura:

```
struct Alumno {
   int legajo;
   char nombre[30];
   int edad;
   float nota;
};
```

Podemos crear un registro:

```
Alumno a;
```

y cargar sus datos:

```
a.legajo = 125;
a.edad = 20;
a.nota = 8.5;
```

Para el nombre podríamos utilizar, por ejemplo:

```
strcpy(a.nombre, "Juan Perez");
```

En este caso será necesario incluir:

```
#include <cstring>
```

Una vez cargado el registro completo, podemos escribirlo en el archivo mediante:

```
archivo.write(
   reinterpret_cast<char*>(&a),
   sizeof(a)
);
```

Esta instrucción significa: Copiar al archivo todos los bytes que ocupa el registro a.
No es necesario escribir campo por campo.
En lugar de hacer varias escrituras, se copia el registro completo.

¿POR QUÉ USAMOS CHAR NOMBRE[30] Y NO STRING?

Anteriormente utilizamos estructuras como:

```
struct Alumno {
   int legajo;
   string nombre;
   int edad;
   float nota;
};
```

Esta estructura es perfectamente válida para trabajar normalmente dentro de un programa.
Sin embargo, no es conveniente grabarla directamente con:

```
archivo.write(reinterpret_cast<char*>(&a), sizeof(a));
```

porque un objeto string no contiene necesariamente dentro de la estructura todos los caracteres del texto. Internamente administra memoria y posee información propia utilizada por C++.
Por este motivo, cuando se desea grabar directamente un registro completo mediante write(), suele utilizarse un arreglo de caracteres de tamaño fijo:

```
char nombre[30];
```

De esta manera, todos los datos que forman el registro se encuentran contenidos directamente dentro de la estructura.
Por ejemplo:

```
struct Alumno {
   int legajo;
   char nombre[30];
   int edad;
   float nota;
};

Ejemplo completo de grabación de un registro
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

struct Alumno {
   int legajo;
```

```
    char nombre[30];
    int edad;
    float nota;
};

int main() {
    ofstream archivo;
    Alumno a;

    a.legajo = 125;
    strcpy(a.nombre, "Juan Perez");
    a.edad = 20;
    a.nota = 8.5;

    archivo.open("alumnos.dat", ios::binary);

    if (archivo.is_open()) {

        archivo.write(
            reinterpret_cast<char*>(&a),
            sizeof(a)
        );

        archivo.close();
    }
    else {
        cout << "No se pudo abrir el archivo" << endl;
    }

    return 0;
}
```

La instrucción: sizeof(a) indica cuántos bytes ocupa el registro completo.
Por lo tanto:

```
archivo.write(
    reinterpret_cast<char*>(&a),
    sizeof(a)
);
```

graba en el archivo un registro completo de tipo Alumno.

Lectura de un registro completo

Para recuperar ese registro se utiliza read().
Por ejemplo:

```
#include <iostream>
#include <fstream>
using namespace std;

struct Alumno {
    int legajo;
    char nombre[30];
    int edad;
    float nota;
};
```

```
int main() {
   ifstream archivo;
   Alumno a;

   archivo.open("alumnos.dat", ios::binary);

   if (archivo.is_open()) {

      archivo.read(reinterpret_cast<char*>(&a),sizeof(a) );

      cout << "Legajo: " << a.legajo << endl;
      cout << "Nombre: " << a.nombre << endl;
      cout << "Edad: " << a.edad << endl;
      cout << "Nota: " << a.nota << endl;
      archivo.close();
   }
   else {
      cout << "No se pudo abrir el archivo" << endl;
   }

   return 0;
}
```

En este caso:
archivo.read(reinterpret_cast<char*>(&a), sizeof(a));
copia desde el archivo tantos bytes como ocupa un registro Alumno y los coloca en la variable a.
Una vez realizada la lectura, pueden utilizarse normalmente sus campos:

a.legajo
a.nombre
a.edad
a.nota

Por lo tanto, con archivos binarios podemos considerar al registro completo como la unidad que se escribe y se lee:

WRITE → graba un registro
READ → recupera un registro

Archivos binarios con varios registros

Normalmente un archivo contiene una gran cantidad de registros del mismo tipo.
Por ejemplo, si tenemos la siguiente estructura:

```
struct Alumno {
    int legajo;
    char nombre[30];
    int edad;
    float nota;
};
```

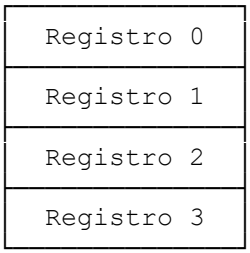
podríamos almacenar en un mismo archivo los datos de muchos alumnos.
Conceptualmente, el archivo podría representarse de la siguiente manera:

```
Registro 0  → Alumno 1
Registro 1  → Alumno 2
Registro 2  → Alumno 3
Registro 3  → Alumno 4
```

...

Cada registro ocupa una determinada cantidad de bytes y se almacena inmediatamente después del registro anterior.
Por ejemplo:

ARCHIVO alumnos.dat



Todos estos registros tienen el mismo tamaño, ya que todos corresponden a la misma estructura `Alumno`.

Grabación de varios registros

Para grabar varios registros podemos utilizar repetidamente el método:

```
write()
```

Por ejemplo:

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

struct Alumno {
    int legajo;
    char nombre[30];
    int edad;
    float nota;
};

int main() {
    ofstream archivo;
    Alumno a;

    archivo.open("alumnos.dat", ios::binary);

    if (archivo.is_open()) {

        a.legajo = 125;
        strcpy(a.nombre, "Juan Perez");
        a.edad = 20;
        a.nota = 8.5;

        archivo.write(
            reinterpret_cast<char*>(&a),
            sizeof(a)
        );
```

```
        a.legajo = 230;
        strcpy(a.nombre, "Maria Gomez");
        a.edad = 22;
        a.nota = 7;

        archivo.write(
            reinterpret_cast<char*>(&a),
            sizeof(a)
        );

        a.legajo = 415;
        strcpy(a.nombre, "Pedro Sanchez");
        a.edad = 19;
        a.nota = 9;

        archivo.write(
            reinterpret_cast<char*>(&a),
            sizeof(a)
        );

        archivo.close();
    }
    else {
        cout << "No se pudo abrir el archivo" << endl;
    }

    return 0;
}
```

Cada vez que se ejecuta:

```
archivo.write(
    reinterpret_cast<char*>(&a),
    sizeof(a)
);
```

se graba un nuevo registro en el archivo.
Después de grabar el primer alumno, la posición de escritura avanza automáticamente.
Por lo tanto, el siguiente registro se escribe a continuación del anterior.
El archivo quedaría formado por:

```
Registro 0 → Juan Perez
Registro 1 → Maria Gomez
Registro 2 → Pedro Sanchez
```

No es necesario indicarle al programa dónde debe escribir cada nuevo registro cuando se realiza una escritura secuencial. La posición de escritura avanza automáticamente.

Carga de registros desde teclado

Naturalmente, los registros también pueden cargarse desde teclado.
Por ejemplo:

```
#include <iostream>
#include <fstream>
```

```
using namespace std;

struct Alumno {
    int legajo;
    char nombre[30];
    int edad;
    float nota;
};

int main() {
    ofstream archivo;
    Alumno a;
    int cantidad;

    archivo.open("alumnos.dat", ios::binary);

    if (archivo.is_open()) {

        cout << "Cantidad de alumnos: ";
        cin >> cantidad;

        for (int i = 0; i < cantidad; i++) {

            cout << "Legajo: ";
            cin >> a.legajo;

            cout << "Nombre: ";
            cin.ignore();
            cin.getline(a.nombre, 30);

            cout << "Edad: ";
            cin >> a.edad;

            cout << "Nota: ";
            cin >> a.nota;

            archivo.write(
                reinterpret_cast<char*>(&a),
                sizeof(a)
            );
        }

        archivo.close();
    }
    else {
        cout << "No se pudo abrir el archivo" << endl;
    }

    return 0;
}
```

En cada repetición del ciclo se carga un alumno en la variable: `a` y luego se graba ese registro en el archivo.
La misma variable `a` puede reutilizarse para todos los alumnos, porque una vez que su contenido fue copiado al archivo, puede cargarse nuevamente con los datos del siguiente alumno.

Este concepto es importante: **el programa puede procesar un archivo registro por registro utilizando una única variable de memoria**.

Lectura de varios registros

Para leer todos los registros de un archivo binario podemos utilizar repetidamente:

```
read()
```

Por ejemplo:

```
#include <iostream>
#include <fstream>
using namespace std;

struct Alumno {
    int legajo;
    char nombre[30];
    int edad;
    float nota;
};

int main() {
    ifstream archivo;
    Alumno a;

    archivo.open("alumnos.dat", ios::binary);

    if (archivo.is_open()) {

        while (
            archivo.read(
                reinterpret_cast<char*>(&a),
                sizeof(a)
            )
        ) {

            cout << "Legajo: " << a.legajo << endl;
            cout << "Nombre: " << a.nombre << endl;
            cout << "Edad: " << a.edad << endl;
            cout << "Nota: " << a.nota << endl;
            cout << endl;
        }

        archivo.close();
    }
    else {
        cout << "No se pudo abrir el archivo" << endl;
    }

    return 0;
}
```

Mientras pueda leerse correctamente un registro completo desde el archivo, continuar procesando.
En cada repetición:

1. se leen del archivo tantos bytes como ocupa un `Alumno`;

2. esos bytes se copian en la variable `a`;
3. se procesan los campos del registro;
4. la posición de lectura avanza al siguiente registro.

Agregar nuevos registros sin borrar los existentes

Supongamos que `alumnos.dat` ya contiene:

```
Registro 0 → Juan
Registro 1 → Maria
Registro 2 → Pedro
```

Si queremos incorporar un nuevo alumno sin eliminar los anteriores, podemos abrir el archivo utilizando: `ios::app`
Por ejemplo:

```
ofstream archivo;

archivo.open("alumnos.dat",ios::binary | ios::app);

Alumno a;

a.legajo = 500;
strcpy(a.nombre, "Lucia Fernandez");
a.edad = 21;
a.nota = 8;

archivo.write(
    reinterpret_cast<char*>(&a),
    sizeof(a)
);
```

El archivo quedará:

```
Registro 0 → Juan
Registro 1 → Maria
Registro 2 → Pedro
Registro 3 → Lucia
```

El nuevo registro se agrega al final.
Por lo tanto, para realizar un alta de un nuevo registro podemos abrir el archivo de la siguiente manera:

```
ofstream archivo("alumnos.dat",ios::binary | ios::app);
```

y escribir el nuevo registro mediante `write()`.

Acceso directo o aleatorio a un archivo

Hasta ahora vimos que un archivo puede recorrerse de manera secuencial, es decir, leyendo un registro después de otro desde el comienzo hasta llegar al registro deseado.

Sin embargo, cuando todos los registros tienen el mismo tamaño, como ocurre normalmente en un archivo binario de estructuras de tamaño fijo, es posible calcular directamente la posición de un registro determinado.

A esta forma de acceso se la denomina acceso directo o acceso aleatorio.

NÚMERO RELATIVO DE REGISTRO

Como vimos al comienzo del capítulo, podemos considerar que los registros de un archivo están numerados a partir de 0:

Registro 0
Registro 1
Registro 2
Registro 3

Este número se denomina también **número relativo de registro**.

Es importante recordar que este número no está necesariamente almacenado dentro del archivo. Es simplemente una forma de identificar la posición que ocupa cada registro respecto del comienzo del archivo.

Por ejemplo:

N° relativo	Registro
0	Alumno 1
1	Alumno 2
2	Alumno 3
3	Alumno 4

CÁLCULO DE LA POSICIÓN DE UN REGISTRO

Supongamos que tenemos:

```
struct Alumno {
   int legajo;
   char nombre[30];
   int edad;
   float nota;
};
```

Todos los registros `Alumno` ocupan la misma cantidad de bytes.

Podemos conocer ese tamaño mediante:

sizeof(Alumno)

Si queremos acceder al registro número `n`, su posición dentro del archivo se puede calcular mediante:

n * sizeof(Alumno)

Por ejemplo, si cada registro ocupara 40 bytes:

Registro 0 → comienza en byte 0
Registro 1 → comienza en byte 40
Registro 2 → comienza en byte 80
Registro 3 → comienza en byte 120
Registro 4 → comienza en byte 160
```
```

seekg(): mover la posición de lectura

Para mover la posición desde la cual se realizará la próxima lectura se utiliza:

```
seekg()
```

Por ejemplo:

```
archivo.seekg(5 * sizeof(Alumno), ios::beg);
```

Esta instrucción mueve la posición de lectura hasta el comienzo del registro número 5.

`ios::beg` significa que el desplazamiento se calcula desde el **comienzo del archivo**.

EJEMPLO DE LECTURA DIRECTA DE UN REGISTRO

Supongamos que queremos leer directamente el registro número 3.

```
#include <iostream>
#include <fstream>
using namespace std;

struct Alumno {
   int legajo;
   char nombre[30];
   int edad;
   float nota;
};

int main() {
   ifstream archivo;
   Alumno a;
   int nroRegistro = 3;

   archivo.open("alumnos.dat", ios::binary);

   if (archivo.is_open()) {

      archivo.seekg(
         nroRegistro * sizeof(Alumno),
         ios::beg
      );

      archivo.read(
         reinterpret_cast<char*>(&a),
         sizeof(a)
      );

      if (archivo) {
         cout << "Legajo: " << a.legajo << endl;
         cout << "Nombre: " << a.nombre << endl;
```

```
            cout << "Edad: " << a.edad << endl;
            cout << "Nota: " << a.nota << endl;
        }
        else {
            cout << "El registro no existe" << endl;
        }

        archivo.close();
    }
    else {
        cout << "No se pudo abrir el archivo" << endl;
    }

    return 0;
}
```

OTRAS POSICIONES DE REFERENCIA

Además de ios::beg existen otras posiciones desde las cuales puede calcularse un desplazamiento.

- ios::beg indica el comienzo del archivo.
- ios::cur indica la posición actual.
- ios::end indica el final del archivo.

seekp(): mover la posición de escritura

Así como `seekg()` mueve la posición de lectura, existe: seekp()

que mueve la posición donde se realizará la próxima escritura.

```
archivo.seekp( 3 * sizeof(Alumno), ios::beg);
```

coloca la posición de escritura en el registro número 3.

Esto permite sobrescribir o modificar un registro determinado sin tener que reescribir todo el archivo.

Para realizar lectura y escritura sobre el mismo archivo suele utilizarse:

```
fstream


fstream archivo;

archivo.open("alumnos.dat", ios::in | ios::out | ios::binary);
```

CONOCER LA POSICIÓN ACTUAL DENTRO DEL ARCHIVO

Existen funciones que permiten conocer en qué posición se encuentra actualmente el archivo.

Estas funciones son: tellg() y tellp()

- tellg(): devuelve la posición actual, expresada en bytes, desde la cual se realizará la próxima lectura.
- tellp():permite conocer la posición, expresada en bytes, donde se realizará la próxima escritura.

La función remove()

La función `remove()` permite eliminar un archivo.

Por ejemplo:

```
remove("alumnos.dat");
```

elimina el archivo denominado `alumnos.dat`.

Para utilizarla puede incluirse:

```
#include <cstdio>
```

Es importante comprender que aquí `remove()` elimina **el archivo completo**, no un registro individual dentro de él.

La función rename()

La función: `rename()` permite cambiar el nombre de un archivo.

Por ejemplo:

```
rename("auxiliar.dat", "alumnos.dat");
```

cambia el nombre:

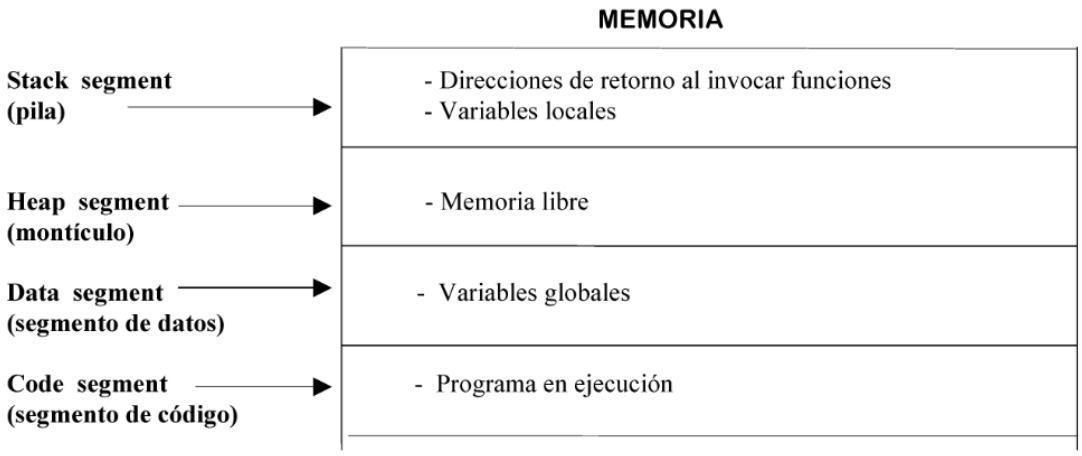
```
auxiliar.dat
```

por:

```
alumnos.dat
```

CAPITULO 22. Gestión Dinámica de Variables

Se entiende por gestión de variables en un lenguaje de programación al conjunto de técnicas que usa ese lenguaje para ubicar en memoria una variable, y mantenerla allí, a partir del momento de su declaración.
Funcionalmente, la memoria disponible para la ejecución de un programa se divide en cuatro segmentos, cada uno de los cuales tiene una utilidad específica. Gráficamente, esta es la segmentación realizada en la memoria al ejecutar un programa:

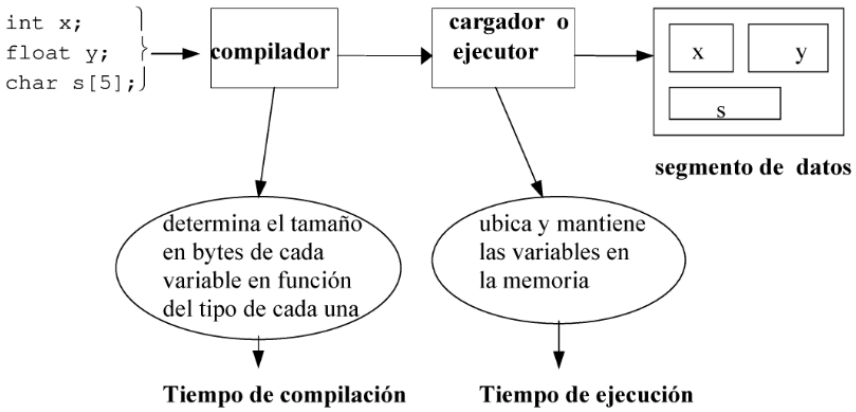


El segmento designado como Heap Segment es la zona de memoria que queda sin usar cuando un programa se ejecuta. No obstante, existen situaciones en donde un programador podría tener necesidad de acceder a la memoria libre y ubicar allí alguna estructura de datos. Esto puede hacerse a través del uso de punteros, y recurriendo a técnicas de manejo de variables que se designan en su conjunto como gestión dinámica de variables.

GESTION DE VARIABLES

Existen dos técnicas básicas de gestión: la gestión estática y la gestión dinámica de variables.

GESTIÓN ESTÁTICA.
La técnica estática es la más elemental de las dos. Se basa en ubicar variables en el segmento de datos (si las variables son globales) o en el stack (si las variables son locales)



Cuando se declara una variable, el compilador determina el tamaño de la misma de acuerdo a su tipo. Luego, al ejecutar el programa, el programa cargador ubica la variable en el segmento de datos o en el stack segment, y allí la mantiene hasta que termina la ejecución del módulo o función que la creó, se utilice o no dicha variable. Si la variable era global, permanecerá en el segmento de datos hasta que termine el programa, y si era local, se quedará en el stack hasta que termine la función o bloque de acciones que la creó. Justamente por eso la técnica se llama **estática**: cada variable se ubica en un lugar fijo y no puede ser destruida, ni modificada en su tamaño, mientras no termine el módulo que la declaró.

Observar sin embargo que la característica esencial de esta técnica, es que el **tamaño de cada variable es determinado en tiempo de compilación**, es decir, **antes** que comience a ejecutarse el programa. El compilador requiere que para cada variable se indique el tamaño de la misma sin ambigüedades. Esto provoca inconvenientes para manejar variables compuestas de **crecimiento impredecible**, como las listas, o para manejar variables en situaciones de tamaño difícil de predecir a priori, como los arreglos en ciertos problemas. Como se sabe, al definir un arreglo debe indicarse con una constante el tamaño del mismo para que el compilador pueda determinar los bytes que va a usar. Al comenzar el programa puede ocurrir que se usen menos elementos que los reservados, en cuyo caso el resto se desperdicia, o puede suceder que se requieran más elementos que los pedidos, y en este caso se debe interrumpir el programa, cambiar el tamaño y volver a compilar.

GESTIÓN DINAMICA.

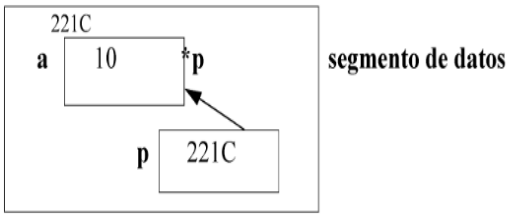
Esta técnica permite resolver los problemas de la gestión estática. La idea central es evitar que el compilador se haga cargo de la gestión de una variable cuando se desconoce a priori el tamaño de la misma o su forma de crecimiento. La única manera de hacer esto es no declarar la variable y de esta forma, el compilador no intentará gestionar la misma, pero ahora el problema es cómo usar una variable que no se declaró.
Entonces, se espera a que el programa comience a ejecutarse, y recién entonces se solicita memoria para la variable que no se declaró. Es decir, la gestión de la variable se hará en tiempo de ejecución y no ya en tiempo de compilación. Esto tiene sentido, porque es sólo en tiempo de ejecución cuando se conoce realmente el tamaño de un arreglo (por ejemplo) o se puede saber cómo crecerá una lista. La memoria pedida para la variable no declarada, solo puede venir del heap segment, que es el área de memoria que quedó libre al comenzar a ejecutarse el programa. De esta forma, las variables gestionadas en forma estática por el compilador se ubicarán en el segmento de datos o en el stack segment, y las variables dinámicas (gestionadas en tiempo de ejecución) se ubicarán en el heap segment. Una ventaja adicional al usar variables dinámicas, se da cuando alguna de ellas deja de ser utilizada y por ello deja de ser necesaria: en ese momento el programador puede destruirla, liberando así la memoria que ocupaba y devolviéndola al heap segment.

CAPITULO 23. Punteros

Básicamente, un puntero es una variable preparada para contener direcciones de memoria. En ese sentido, un puntero puede contener la dirección de otra variable ya alojada en memoria. En el siguiente ejemplo, se supone declarada una variable global **a** y un puntero **p**, y se asigna en el puntero la dirección donde está ubicada la variable **a**.

int a , *p
a←10
p ← &a

```
int a,  *p;
a= 10;
p= &a;
```



Para declarar una variable puntero, debe tenerse en cuenta que un puntero puede contener direcciones, pero sólo de variables del mismo tipo: No puede usarse el mismo puntero para "apuntar" a una variable de tipo int y luego a una variable de tipo float. Ese tipo se llama tipo base del puntero. En la declaración de un puntero, se escribe primero el tipo base del mismo y luego el nombre del puntero, pero precedido por un asterisco (*****). En el ejemplo anterior, se declaró un **puntero p** cuyo tipo base es int.

Para tomar la dirección de otra variable y cargarla en el puntero se puede usar el operador referencia (**&**)
La instrucción p = &a del ejemplo, toma la dirección de la variable **a** y la asigna en el puntero **p**. Una vez que un puntero **p** fue cargado con la dirección de otra variable **a**, se puede usar el identificador ***p** (se lee "asterisco p") para acceder a la variable **a** (es decir: los identificadores **a** y ***p** son equivalentes a partir de la instrucción **p = &a**).

Importante!!		
p	es el puntero con la dirección de **a**	(tipo **puntero**, **4** bytes)
***p**	es la variable apuntada por p	(tipo **int**, **2** bytes)

Otro ejemplo

```
int a, *p, *q, *r;
a= 10;
p= &a;
q=p;
*q= 2;
cout << a; // muestra 2
cout << *p;
// muestra 2
r = NULL; // anula el puntero
*r= 2; // error: *r ya no existe.
```

int a , *p, *q, *r
a←10
p ← &a
q ← p
*q ← 2
MOSTRAR a //Muestra 2
MOSTRAR *p //Muestra 2
r ← null // el puntero apunta a null
*r ← 2 // error: *r ya no existe

Observar que, en los ejemplos anteriores, todas las variables usadas eran estáticas: todas fueron declaradas y por lo tanto fueron

gestionadas por el compilador (incluso los punteros). Sin embargo, usando punteros se pueden gestionar variables en forma dinámica, para lo cual se usa el operador **new**.
Este operador busca en el **heap segment** un conjunto de bytes libres, en donde pueda alojar una variable del tipo pedido. Si encuentra lugar, marca los bytes como ocupados y retorna la dirección de ese lugar. Si no encuentra lugar, retorna NULL.
Cualquier variable creada con **new** puede ser destruida por el programador cuando lo crea necesario, usando **delete**. Este operador "desmarca" los bytes que ocupaba la variable, liberando el lugar que ocupaba y retornándolo al heap segment.
Debe observarse que **delete** no borra la dirección contenida en el puntero, ni asigna NULL en ese puntero: sólo desmarca los bytes, dejándolos libres. La dirección contenida en el puntero, pasa a considerarse como dirección residual ("basura"), y, por lo tanto, es un error de concepto intentar usar la variable apuntada por el puntero, ya que esa variable en la práctica no existe.

int *p, *q
p ← new int
*p ← 7
q ← new int
delete p
*p ← 7 // Error, p ya no existe

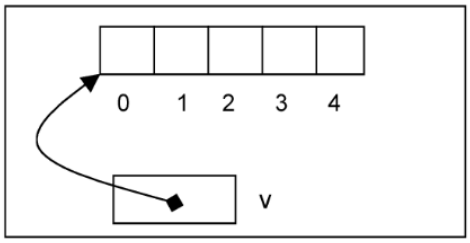
Si bien podría pensarse que **delete p**; es equivalente a hacer p = NULL; debe tenerse cuidado de comprender que esto no es así: usando delete se liberan los bytes usados, y cualquier new posterior podrá reusar esos bytes. En cambio, usando p=NULL; sólo se borra la dirección, pero al no liberar los bytes ningún new posteriormente usado podrá disponer de esos bytes.

Gestión dinámica de arreglos. Relación entre punteros y arreglos

Para entender la forma en que puede gestionarse un arreglo dinámicamente, veamos con un poco más de detalle lo que ocurre cuando se declara un arreglo en forma estática.
Al hacer una declaración del tipo siguiente

int v [5];

se gestiona en el **data segment** cinco lugares de tipo int (es decir, diez bytes por lo general) a modo de un arreglo unidimensional. Pero el hecho importante es que además de esos cinco lugares de tipo int, aloja en memoria un puntero de tipo int (es decir, un puntero al tipo base del arreglo), y es a este puntero al que realmente se designa con el identificador **v**. Además, se carga en ese puntero la dirección del primer elemento del arreglo gestionado (o sea, la dirección del primero de los diez bytes pedidos en la declaración del ejemplo).



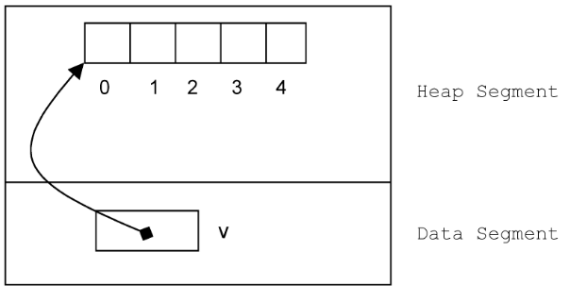
Data Segment

Para hacer que el arreglo se gestione en forma dinámica, declaramos el puntero y luego gestionamos el arreglo con **new.** Si declaramos un puntero, el compilador no tiene porqué saber para qué se usará dicho puntero, y por lo tanto no estamos obligados a indicar el tamaño del arreglo que luego usaremos. Por cierto que el compilador soló reservará memoria para el puntero sin crear el arreglo,

pero esto no es inconveniente por cuanto el operador **new** nos permitirá pedir más tarde la memoria para el arreglo, guardando la dirección en el puntero.

```
int *v, n
INGRESAR n
v ← new int[n]
v[1] ← 18
```

Notemos que el resultado final es el mismo que en la declaración estática: aparece un arreglo y junto a él un puntero conteniendo la dirección del primer elemento del arreglo. La diferencia es que ahora el arreglo propiamente dicho está en el **heap segment** (y no en el **data segment** ni en el **stack segment**). Por lo demás, como se ve en el ejemplo, sigue valiendo lo anterior: el nombre del puntero puede usarse de allí en más, "como si fuera un arreglo" en forma transparente. El gráfico siguiente ilustra lo dicho:



Notar que si al usar un arreglo (gestionado en forma dinámica o en forma estática) lo que realmente nos da el lenguaje es un puntero, entonces al enviar como parámetro el arreglo a una función lo que en realidad hace el lenguaje es enviar como parámetro al puntero: el arreglo en sí mismo no es enviado como parámetro porque ocuparía mucho lugar "duplicando" los datos... Lo que realmente toma la función es una copia del puntero al arreglo.

Punteros y registros

Hasta ahora vimos **punteros** (variables que guardan una dirección de memoria) y **registros** (tipos de datos estructurados, con varios campos de distinto tipo agrupados bajo un mismo nombre). Por separado, cada uno resuelve un problema distinto:

- El **registro** nos permite modelar una entidad del mundo real con varios atributos (por ejemplo, un alumno con nombre, legajo y nota).
- El **puntero** nos permite referenciar una posición de memoria en lugar de copiar el dato.

La combinación de ambos es la base de casi todas las estructuras de datos dinámicas que van a ver más adelante en la materia (listas enlazadas, pilas, colas, árboles).

Dada la siguiente definición:

```
Registro reg{
   int c1,
   float c2
}
```

struct reg { int c1; float c2; };

a partir de ella se pueden declarar por una parte variables de tipo registro, y por la otra punteros capaces de apuntar al tipo registro.

```
Registro reg{
   int c1,
   float c2
}
```

La instrucción p <- &a hará que el puntero p apunte a la variable a, que es un registro ubicado en memoria:

```
reg a, *p
p ← & a
```

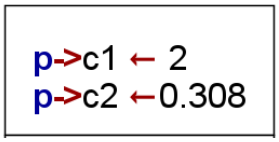
Si se quiere acceder a los campos del registro **a** usando el identificador **a**, se hace en la forma normal, a través del operador punto (.):

a.c1 = 2; a.c2 = 0.308;

Dado que p es un puntero que contiene la dirección de a, podría pensarse que el acceso a los campos de a usando el puntero p sería así:

**p.c1 = 2; *p.c2 = 0.308;*

Sin embargo, lo anterior **no es válido**, ya que en C++, pues provoca una mala interpretación del orden en que deben aplicarse los operadores "asterisco" (*) y "punto" (.).
Para acceder a los campos de un registro apuntado por un puntero p, lo normal no es hacerlo a través de la expresión *p ni con el operador punto (.). Se usa directamente el identificador p y el operador "flecha" (->) como se ve en el ejemplo siguiente:



p -> c1 = 2;
p -> c2 = 0.308;

Igualmente, el uso de *p y el operador punto (.) puede hacerse siempre y cuando *p se encierre entre paréntesis, para evitar problemas de precendencia de operadores: (*p).c1 = 2; (*p).c2 = 0.308;

Para el diagrama de CHAPIN en la materia usaremos el operador flecha ->

CAPITULO 24. Estructuras de datos abstracta

Todas las estructuras de datos que se estudiaron hasta ahora, eran estructuras nativas. Esto quiere decir que el lenguaje (C++ en este caso) las provee ya implementadas y listas para usar. No es necesario hacer ninguna declaración previa para que el lenguaje "entienda" qué es un arreglo, un registro o un archivo. Sin embargo, existen muchas otras estructuras de datos que no son soportadas por C++ (ni por otros lenguajes) que resultan imprescindibles para la solución de ciertos problemas.
Esas estructuras de datos que se definen en forma teórica pero no vienen implementadas por un lenguaje, se designan en general como "estructuras de datos abstractas". Ahora que se sabe lo suficiente sobre la gestión dinámica de variables, se puede hacer un estudio más detallado de las estructuras de datos, incluyendo el análisis de la forma en que pueden implementarse las estructuras abstractas más comunes, como son las listas, las pilas, las colas, los árboles y los grafos.

Básicamente, y sin importar si son nativas o abstractas, las estructuras de datos pueden dividirse en dos grupos, de acuerdo a la forma de su organización interna.
Las más típicas son las siguientes:

Estructuras lineales:
- Arreglos unidimensionales y cadenas de caracteres
- Archivos secuenciales.
- Listas.
- Pilas
- Colas.

Estructuras no lineales:
- Arboles.
- Grafos

Una estructura se dice lineal si para todo elemento de la misma (excepto posiblemente para los elementos de los extremos) existe un único sucesor inmediato y un único antecesor inmediato, lo cual facilita los recorridos secuenciales de la estructura.
En cambio, una estructura se dice no lineal si algún elemento de la misma puede tener más de un antecesor y/o sucesor.
Por ejemplo, un arreglo unidimensional es lineal, ya que cualquier elemento v [i] del arreglo, tiene como único sucesor inmediato a v [i + 1], y como único antecesor inmediato a v [i – 1]. El primer elemento de un arreglo no tiene antecesor (lo cual es válido) y el último elemento no tiene sucesor (lo cual también es válido).

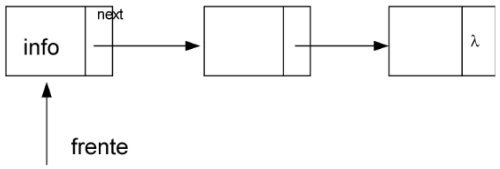
CAPITULO 25. Listas

Lista Simple

Una lista simplemente vinculada (o lista simple) es una estructura lineal compuesta por una serie de elementos individuales llamados nodos. Cada nodo es un registro formado, mínimamente, por **dos campos**:

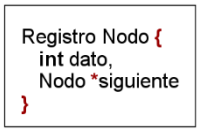
- uno de ellos (que llamaremos **info**) contiene el valor que se almacena en el nodo;
- el otro (que llamaremos **next**) es un puntero con la dirección del nodo sucesor.

El campo next del último nodo, normalmente vale NULL, indicando que el nodo no tiene sucesor. La dirección del primer nodo de la lista se almacena en un puntero separado, al que podemos llamar frente.



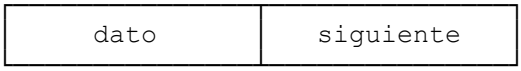
Para declarar y manipular una lista simple, se puede aplicar gestión dinámica.
La idea es que sólo se declara en forma estática el puntero frente, que contendrá la dirección del primer nodo de la lista, y con ese puntero se puede llamar al operador **new** y crear un nodo en forma dinámica.
El nuevo nodo contiene un campo **next** que también es un **puntero**, y puede usarse a su vez para activar nuevamente el operador **new** y crear un segundo nodo. Cada nodo creado dinámicamente contendrá un puntero **next**, y de esta forma, el proceso de creación de nodos puede repetirse tantas veces como sea necesario, creando a razón de un nodo por vez e incorporando dicho nodo al conjunto.

Por ejemplo:

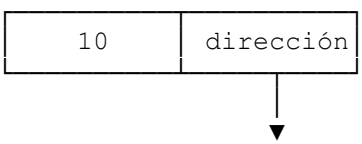



```
struct Nodo {
    int dato;
   Nodo *siguiente;
};
```

Esta estructura merece ser analizada cuidadosamente.El campo dato `int dato` almacena la información que queremos guardar. El campo siguiente `Nodo *siguiente` es un puntero a otro registro del mismo tipo `Nodo`.
Esto significa que puede almacenar la dirección de otro nodo.
Por lo tanto, un nodo puede representarse conceptualmente como:



Por ejemplo:



El campo `siguiente` permite que un nodo conozca dónde se encuentra otro nodo.

Creación dinámica de un nodo

Hasta ahora podríamos declarar un nodo normalmente:

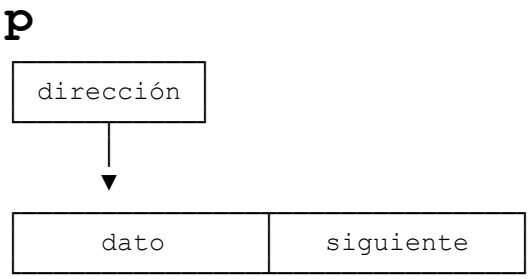
```
Nodo n;
```

Pero para construir estructuras dinámicas necesitamos poder crear nodos durante la ejecución del programa. Para ello utilizamos **new**

Por ejemplo:

```
Nodo *p
p <- new Nodo
```

Primero `Nodo *p` declara un puntero a `Nodo`. Luego `new Nodo` reserva dinámicamente memoria suficiente para almacenar un registro completo de tipo `Nodo`. `new` devuelve la dirección de esa zona de memoria.
Finalmente: `p <- new Nodo` guarda esa dirección en `p`.



Acceso a los campos del nodo

Como p es un puntero a un registro, utilizamos el operador ->
Por ejemplo:

```
p->dato = 10;
```

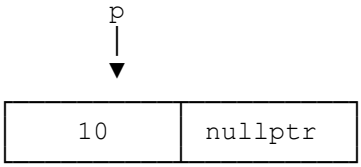
accede al campo dato del nodo apuntado por p.
También podemos escribir:

```
p->siguiente = nullptr;
```

Por lo tanto:

```
Nodo *p
p = new Nodo
p->dato <- 10
p->siguiente <- nullptr
```

crea dinámicamente un nodo que podemos representar como:



¿Qué significa nullptr?

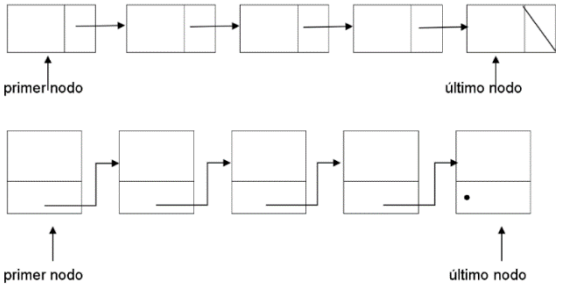
El valor nullptr indica que el puntero **no apunta a ningún objeto válido**.
En nuestro ejemplo:

```
p->siguiente = nullptr;
```

significa que ese nodo no tiene ningún nodo después de él.
Aunque tradicionalmente se utilizó NULL en C++ moderno es preferible utilizar: nullptr

Representación gráfica de una lista

Una lista enlazada es una secuencia de nodos que se interconectan desde el primer nodo hasta el último nodo. En la siguiente gráfica, se evidencia que el primer nodo se enlaza con el segundo, el segundo nodo se enlaza con el tercero y así sucesivamente hasta el último nodo.



Dado que el último nodo no tiene enlace con otro elemento, se le asigna el valor de null.

Recorrido de una lista simplemente enlazada

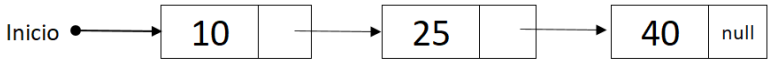
Una vez construida una lista enlazada, una de las operaciones más importantes consiste en **recorrer todos sus nodos**.

Supongamos que tenemos:

```
struct Nodo {
   int dato;
   Nodo *siguiente;
};
```

```
Registro Nodo {
   int dato,
   Nodo *siguiente
}
```

y que el puntero Nodo *inicio contiene la dirección del primer nodo de la lista.
Podemos representar la estructura de la siguiente manera:

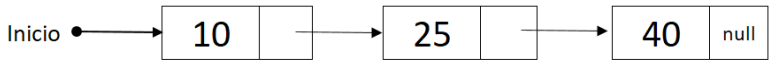


El puntero inicio es especialmente importante porque representa el punto de entrada a toda la estructura.

A partir del primer nodo podemos llegar al segundo mediante: **inicio->siguiente** y desde el segundo podemos llegar al tercero, y así sucesivamente.

¿Cómo avanzar de un nodo al siguiente?

Supongamos:
inicio



El primer nodo contiene dato = 10 y en su campo siguiente se encuentra la dirección del segundo nodo.
Por lo tanto:

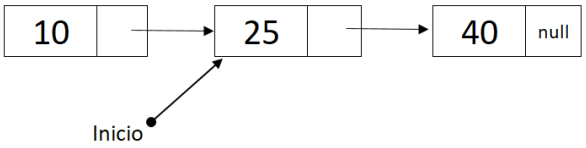
inicio->siguiente

representa la dirección del segundo nodo. Podríamos pensar entonces en hacer:

inicio <- inicio->siguiente;

Después de esta instrucción:

Ahora inicio apunta al nodo que contiene 25.
Si volvemos a hacer:
inicio = inicio->siguiente;
quedará:



El problema de modificar inicio
Aunque el procedimiento anterior permite avanzar por los nodos, tiene un problema muy importante.
Al terminar el recorrido hemos perdido la dirección del primer nodo.
Los nodos continúan existiendo en memoria, pero ya no tenemos la dirección del primero.
Por lo tanto, ya no podemos volver a acceder a la estructura a través de inicio.
Esta situación debe evitarse.

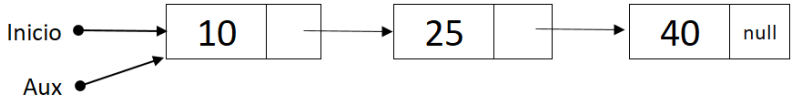
Utilización de un puntero auxiliar
Para recorrer una lista sin modificar inicio, se utiliza otro puntero.
Por ejemplo:

*Nodo *aux*

Luego hacemos:

aux <- inicio

Ahora ambos punteros contienen inicialmente la dirección del primer nodo.

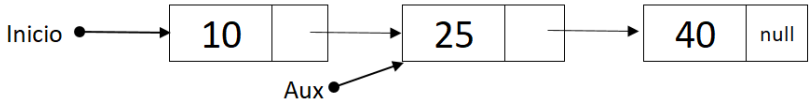


Pero durante el recorrido solamente modificaremos aux.

Primer avance
Ejecutamos:

aux <- aux->siguiente;

Entonces:

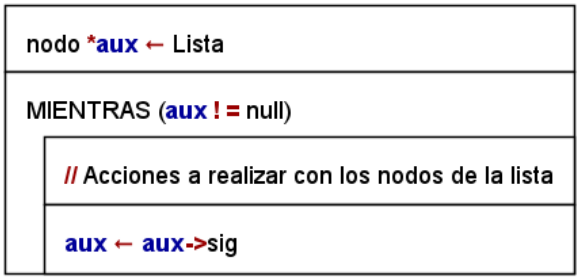


inicio continúa apuntando al primer nodo.
Por lo tanto, no perdimos la lista.

Recorrer toda la lista

Como no sabemos necesariamente cuántos nodos contiene la lista, resulta natural utilizar un ciclo MIENTRAS.
Por ejemplo:

Procedimiento recorrer(nodo *Lista)
//Se recorre con un puntero auxiliar
para no perder el puntero al inicio de la lista




```
void recorrer(nodo *Lista) {
    Nodo *aux = Lista;

    while (aux != nullptr) {

       // Acciones

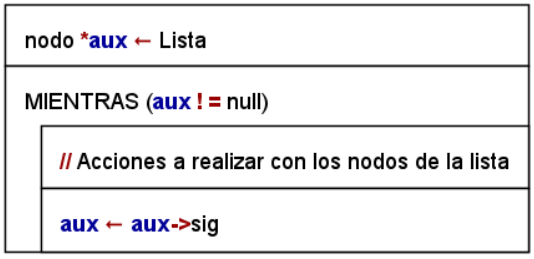
       aux = aux->sig;
    }
}
```

Este es uno de los algoritmos fundamentales para trabajar con listas enlazadas.
Error frecuente
No deberíamos utilizar while (aux->siguiente != nullptr) si queremos procesar todos los nodos de la lista.

Lista vacía

Una lista vacía se representa mediante: Lista <- null

Procedimiento recorrer(Lista)
//Se recorre con un puntero auxiliar
para no perder el puntero al inicio de la lista



el ciclo no se ejecutará ninguna vez. Esto es correcto, ya que no existen nodos para procesar.

Inserción de nodos en una lista simplemente enlazada
Una de las operaciones fundamentales sobre una lista enlazada consiste en **agregar nuevos nodos**.

Comenzaremos por el caso más sencillo:

- insertar el primer nodo en una lista vacía;
- insertar un nuevo nodo al comienzo de una lista que ya contiene elementos.

Inserción del primer nodo

Supongamos que tenemos definida la estructura:

```
Registro Nodo {
   int dato,
   Nodo *siguiente
}
```

y el puntero **Nodo *inicio**
Si la lista está vacía inicio <- null

Para agregar el primer elemento necesitamos crear dinámicamente un nodo.
Por ejemplo:

*Nodo *nuevo*

nuevo <- new Nodo

Ahora nuevo contiene la dirección de un nodo creado dinámicamente.

Luego cargamos el dato:

nuevo->dato <- 10

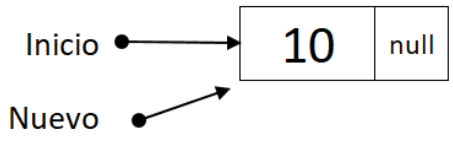
y como este nodo será por el momento el último de la lista:

nuevo->siguiente <- null

Pero todavía inicio continúa valiendo **null**
Por lo tanto, necesitamos hacer que inicio apunte al nuevo nodo:

inicio <- nuevo

El resultado será:



Ahora la lista contiene un elemento.

Algoritmo para insertar al inicio

Procedimiento insertarNodoalInicio(nodo *Lista por referencia, int valor)
// 1-Se crea un nuevo nodo.
// 2-Su campo sig apunta al antiguo primer nodo.
// 3-El puntero Lista ahora apunta al nuevo nodo.

nodo *nodoIns ← new nodo
nodoIns->dato ← valor
nodoIns->sig ← Lista
Lista ← nodoIns

```
void insertarInicio(Nodo *&inicio, int valor) {

   Nodo *nuevo;

   nuevo = new Nodo;

   nuevo->dato = valor;

   nuevo->siguiente = inicio;

   inicio = nuevo;
}
```

Luego:

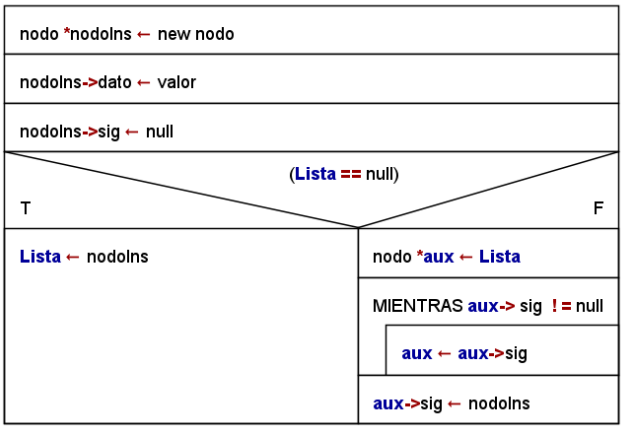
```
insertarInicio(inicio, 10);
```

Como inicio se recibe por referencia, la función puede modificar el puntero original.

Inserción al final de una lista

Otra operación frecuente consiste en agregar un nuevo nodo **al final de la lista**.

Procedimiento insertarNodoalFinal(nodo * Lista por referencia, int valor)
// 1-Se recorre la lista hasta el último nodo (nodo.sig = NULO).
// 2-El último nodo apunta al nuevo.



Algoritmo básico

```
void insertarNodoalFinal(Nodo *&Lista, int valor) {

    Nodo *nuevo;
    Nodo *aux;

    nuevo = new Nodo;

    nuevo->dato = valor;
    nuevo->siguiente = nullptr;

    if (Lista == nullptr) {

        Lista = nuevo;

    }
    else {

        aux = Lista;

        while (aux->siguiente != nullptr) {
            aux = aux->siguiente;
        }

        aux->siguiente = nuevo;
    }
}
```

UNA MEJORA POSIBLE: CONSERVAR TAMBIÉN PUNTERO AL ÚLTIMO NODO

Si se realizan muchas inserciones al final, recorrer toda la lista cada vez puede resultar innecesario. Una alternativa consiste en conservar dos punteros:

Nodo *inicio;
Nodo *fin;

donde:

inicio → primer nodo
fin → último nodo

Entonces, para agregar un nuevo nodo al final, no sería necesario recorrer la lista.

Podríamos hacer:

fin->siguiente <- nuevo
fin <- nuevo

Esta estrategia será especialmente importante cuando estudiemos colas, donde normalmente se conservan punteros al principio y al final de la estructura.

Inserción en una posición intermedia

Hasta ahora vimos cómo insertar un nodo:
- al comienzo de una lista;
- al final de una lista.

Ahora veremos cómo insertar un nuevo nodo **entre dos nodos existentes**.

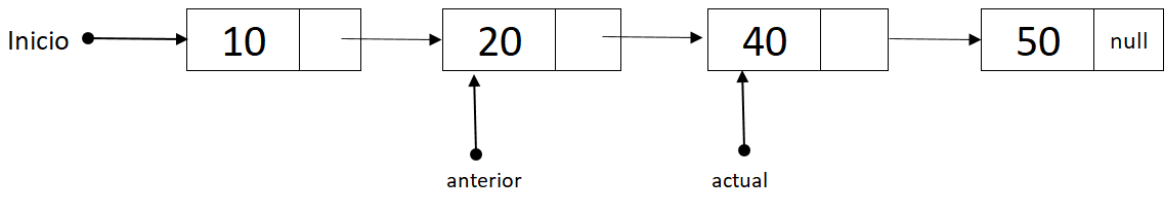
¿Qué necesitamos para insertar entre dos nodos?
Supongamos que queremos insertar un nuevo nodo entre dos nodos

Para realizar correctamente la inserción necesitamos conocer:
- el nodo que quedará **antes** del nuevo nodo;
- el nodo que quedará **después** del nuevo nodo.

Utilizaremos dos punteros:

*Nodo *anterior*
*Nodo *actual*

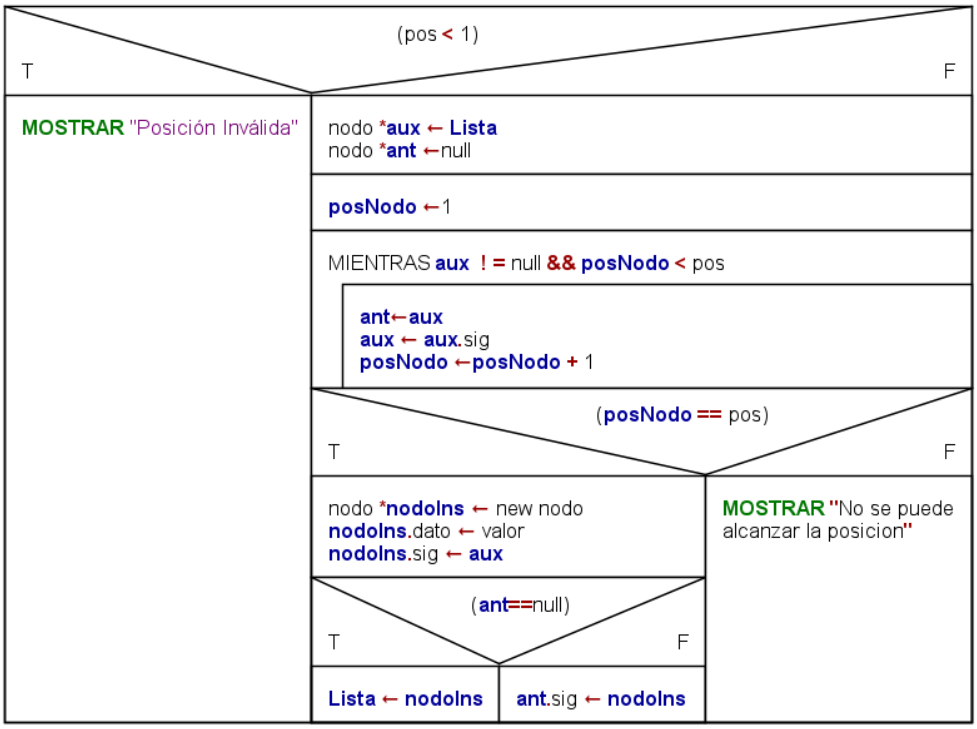
Podemos representarlos así:



Queremos insertar: [30] entre ambos.

Inserción general en una posición intermedia

```
Procedimiento insertarEnPosicion(nodo *Lista por Ref, int pos, int valor)
// 1 - Verifica que la posición solicitada sea válida (mayor o igual a 1).
// 2 - Recorre la lista hasta alcanzar la posición solicitada
//     o hasta que no existan más nodos.
// 3 - Si la posición puede alcanzarse, crea y enlaza el nuevo nodo.
//     En caso contrario, informa que la posición no puede alcanzarse.
// En este algoritmo consideraremos que la primera posición de la lista es la posición 1.
```




```
void insertarEnPosicion(Nodo *&lista, int pos, int valor)
{
   Nodo *aux;
   Nodo *ant;
   Nodo *nodoIns;

   int posNodo;

   if (pos < 1)
   {
      cout << "Posicion invalida" << endl;
   }
   else
   {
      aux = lista;
      ant = nullptr;
      posNodo = 1;
```

```
      while (aux != nullptr && posNodo < pos)
      {
         ant = aux;
         aux = aux->sig;
         posNodo++;
      }

      if (posNodo == pos)
      {
         nodoIns = new Nodo;

         nodoIns->dato = valor;
         nodoIns->sig = aux;

         if (ant == nullptr)
         {
            lista = nodoIns;
         }
         else
         {
            ant->sig = nodoIns;
         }
      }
      else
      {
         cout << "No se puede alcanzar la posicion" << endl;
      }
   }
}
```

Eliminación de nodos

Otra operación fundamental sobre una lista simplemente enlazada consiste en **eliminar uno de sus nodos**.

Para realizar correctamente la eliminación debemos considerar dos tareas:

1. modificar los enlaces para que la lista continúe correctamente conectada;
2. liberar la memoria ocupada por el nodo eliminado.

BÚSQUEDA DEL NODO A ELIMINAR

Supongamos que deseamos eliminar un nodo a partir del valor que contiene.

Utilizaremos dos punteros:

Nodo *aux
Nodo *ant

El puntero **aux** representará el nodo que estamos analizando. Mientras que **ant** conservará la dirección del nodo anterior.

Inicialmente:

```
aux <- Lista
ant <- null
```

Mientras no encontremos el valor buscado, avanzamos:

```
ant <- aux
aux <- aux->sig
```

Es importante realizar primero ant <- aux y después aux <- aux->sig. De esta manera **ant** conserva siempre la dirección del nodo anterior.

ELIMINAR UN NODO INTERMEDIO

Queremos eliminar el nodo apuntado por **aux**. Por lo tanto, hacemos:

```
ant->sig <- aux->sig
```

`

Pero todavía nodo quitado de la lista ocupa memoria. Por lo tanto, debemos liberar esa memoria mediante:

```
delete aux;
```

ELIMINAR EL PRIMER NODO

Existe una situación especial cuando el nodo que queremos eliminar es el primero.

No podemos hacer:

```
ant->sig <- aux->sig
```

porque **ant** vale **null**. En este caso debemos modificar directamente el puntero `Lista`.

Hacemos:

```
Lista <- aux->sig
```

Finalmente:

```
delete aux
```

ELIMINAR EL ÚLTIMO NODO

Supongamos:

En este caso aux->sig <- null. Al hacer:

```
ant->sig <- aux->sig;
```

estamos haciendo realmente:

```
ant->sig <- null
```

Luego **delete aux** libera la memoria ocupada por el antiguo último nodo.

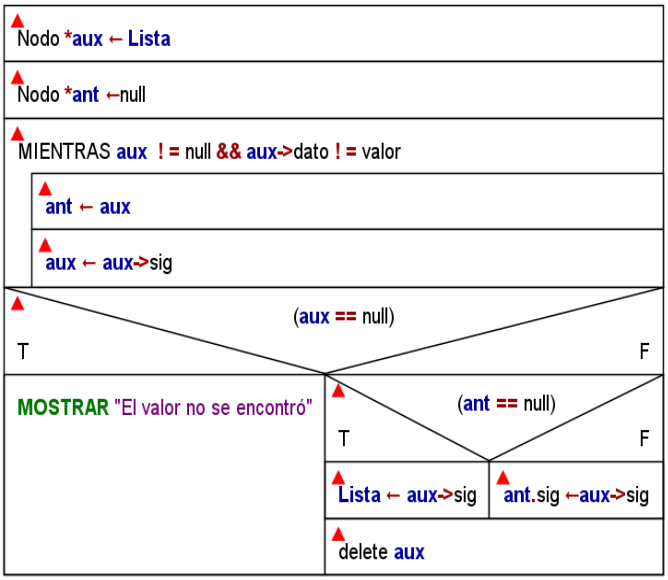
Por lo tanto, **eliminar el último nodo no necesita un caso especial**.

Se resuelve de la misma manera que cualquier nodo que no sea el primero.

ALGORITMO GENERAL DE ELIMINACIÓN

El procedimiento completo puede escribirse:

Procedimiento EliminarNodo(Nodo *Lista por referencia, int valor)
// 1-Se recorre la lista hasta encontrar el valor o llegar al final
// 2-Se lleva el control de dos punteros
// 3- Si se lo encuentra, se lo elimina con la ayuda de estos dos punteros




```
void eliminarNodo(Nodo *&Lista, int valor)
{
   Nodo *aux =  Lista;
   Nodo *ant = nulptr;
```

```
    while (aux != nullptr && aux->dato != valor)
    {
        ant = aux;
        aux = aux->sig;
    }
    if (aux == nullptr)
    {
        cout << "El valor no se encontró" << endl;
    }
    else
    {
        if (ant == nullptr)
        {
            Lista = aux->sig;
        }
        else
        {
            ant->sig = aux->sig;
        }
        delete aux;
    }
}
```

Bibliografía

- Bjarne Stroustrop (2024) Programming: Principles and Practice Using C++, 3rd Edition
- Bjarne Stroustrop (2023) A Tour of C++, 3rd Edition
- C++ Reference. (s. f.). C++ Language: struct. cppreference.com.https://en.cppreference.com/w/cpp/language/struct
- Garrido Carrillo, A. & Valdivia Joaquín, F. (2006). Abstracción y estructura de datos en C++: (ed.). Delta Publicaciones. https://elibro.net/es/lc/utnfrsn/titulos/167034
- Joyanes Aguilar, L. (2005). Programación en C: metodología, algoritmos y estructura de datos: (2 ed.). McGraw-Hill España. https://elibro.net/es/lc/utnfrsn/titulos/50302
- Joyanes Aguilar, L. (2006). Programación en C++: algoritmos, estructuras de datos y objetos: (2 ed.). McGraw-Hill España. https://elibro.net/es/lc/utnfrsn/titulos/50088
- Koenig, A. (s. f.). C++ Programming: From Problem Analysis to Program Design. (Edición universitaria).
- Tymoschuk, J. Fritelli, V. & Guzman, A. (2020). Algoritmos y estructuras de datos: (2 ed.). Jorge Sarmiento Editor - Universitas. https://elibro.net/es/lc/utnfrsn/titulos/175249
- Seminario de ingreso TUP UTN
- Sznajdleder. Pablo – Algoritmos a fondo: con implementación en C y java